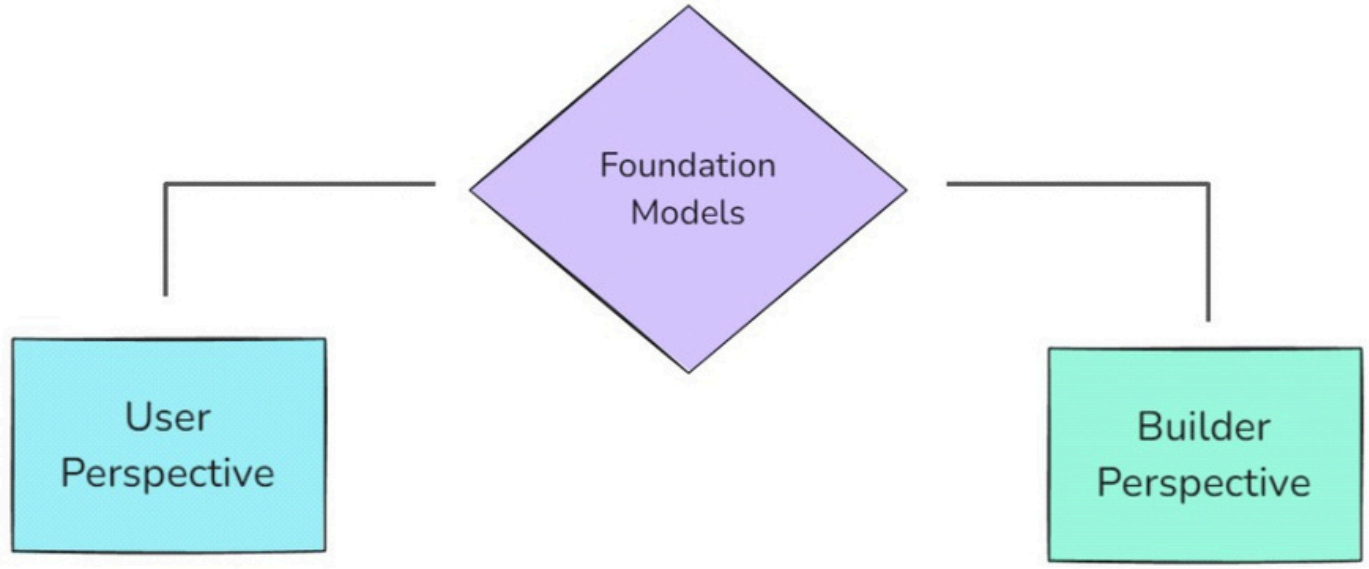
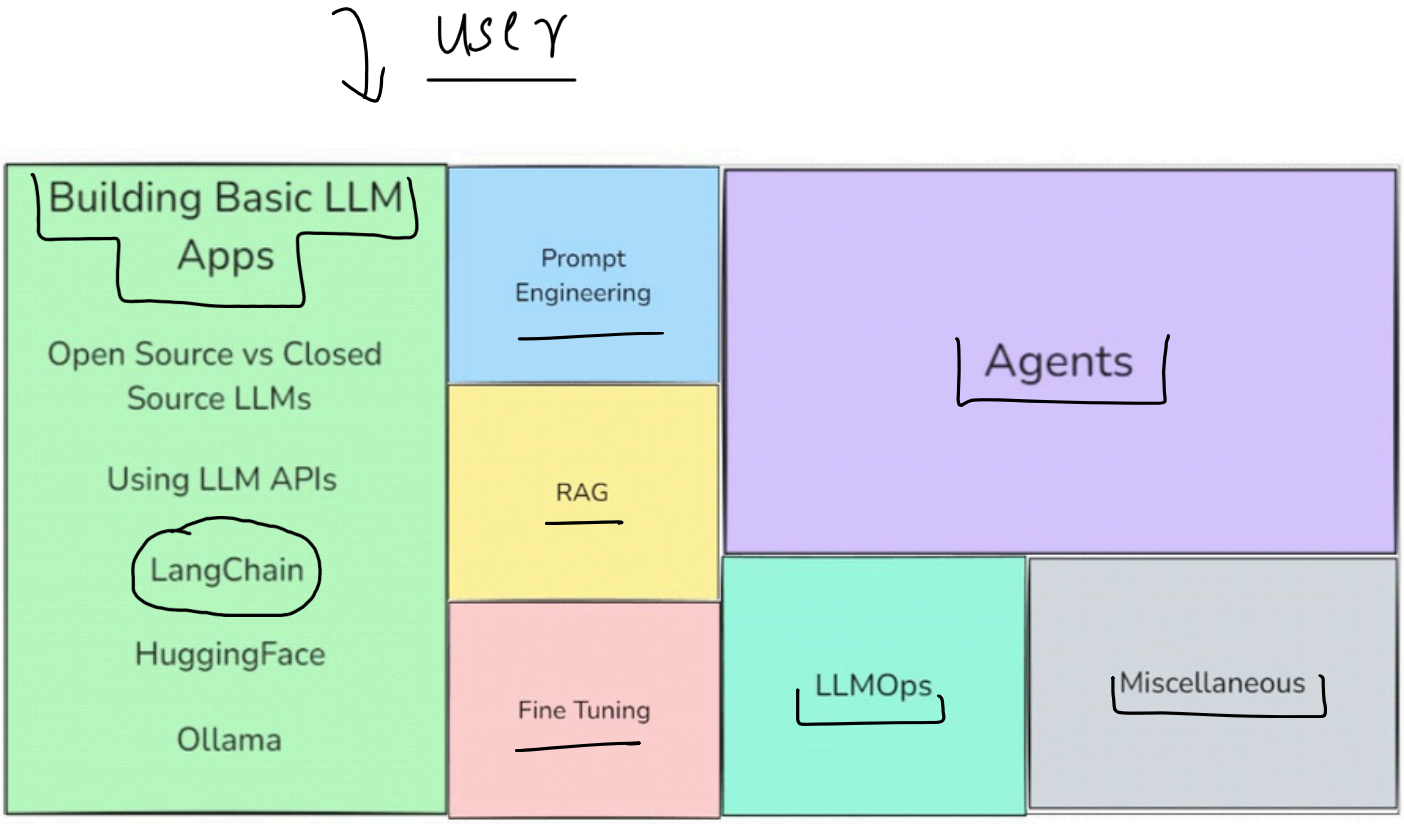
A little background!

29 January 2025 08:28





What is LangChain

29 January 2025 08:35

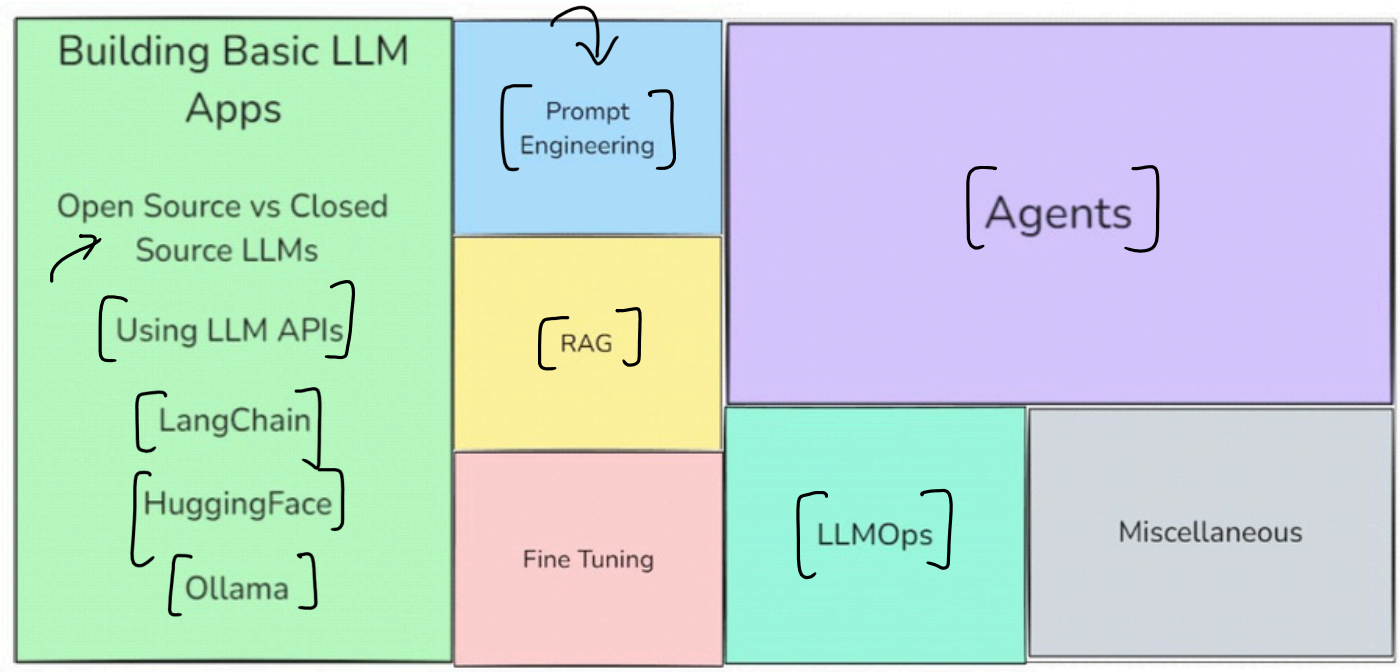
LangChain is an open source framework that helps in building LLM based applications. It provides modular components and end-to-end tools that help developers build complex AI applications, such as chatbots, question-answering systems, retrieval-augmented generation (RAG), autonomous agents, and more.

1.Supports all the major LLMs
2.Simplifies developing LLM based applications
3.Integrations available for all major tools
4.Open source/Free/Actively developed
5.Supports all major GenAI use cases

chains

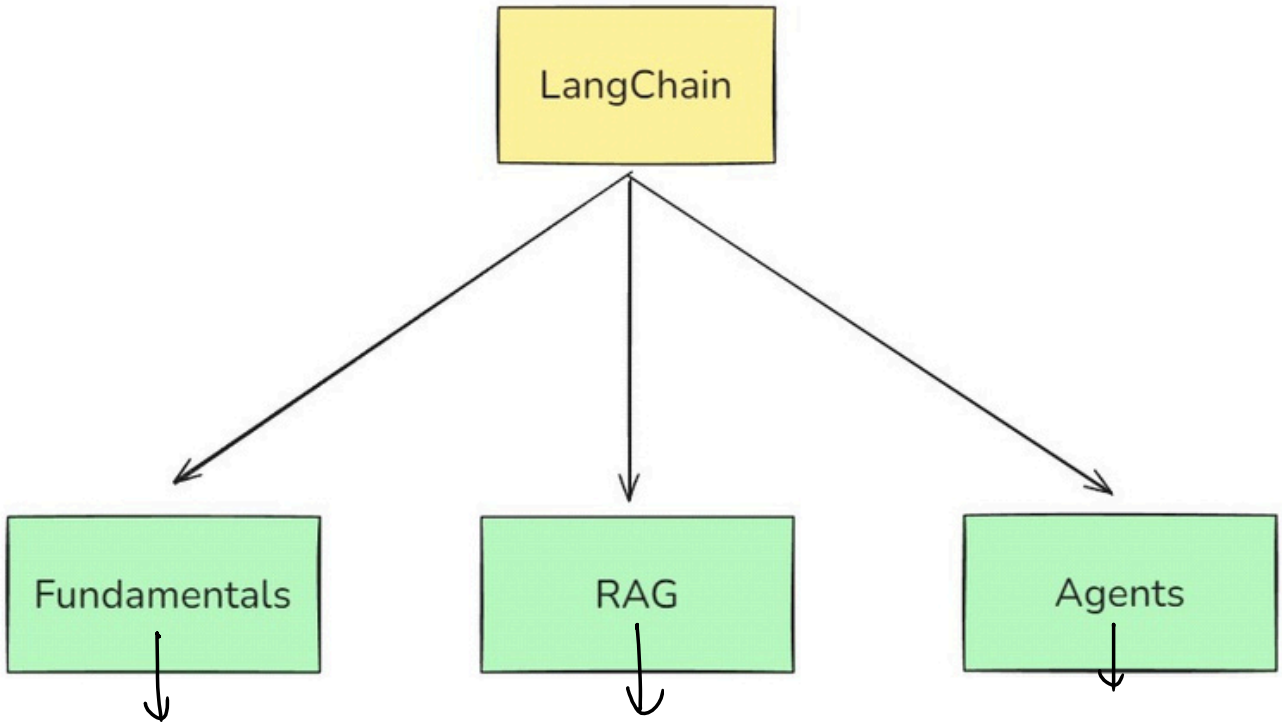
Why LangChain first

29 January 2025 08:35



Curriculum Structure

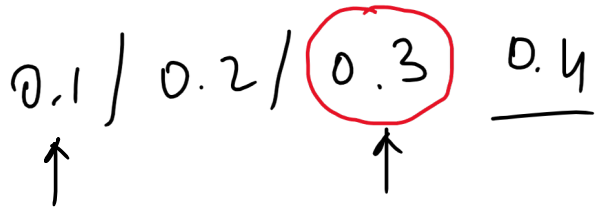
29 January 2025 08:36



My Focus

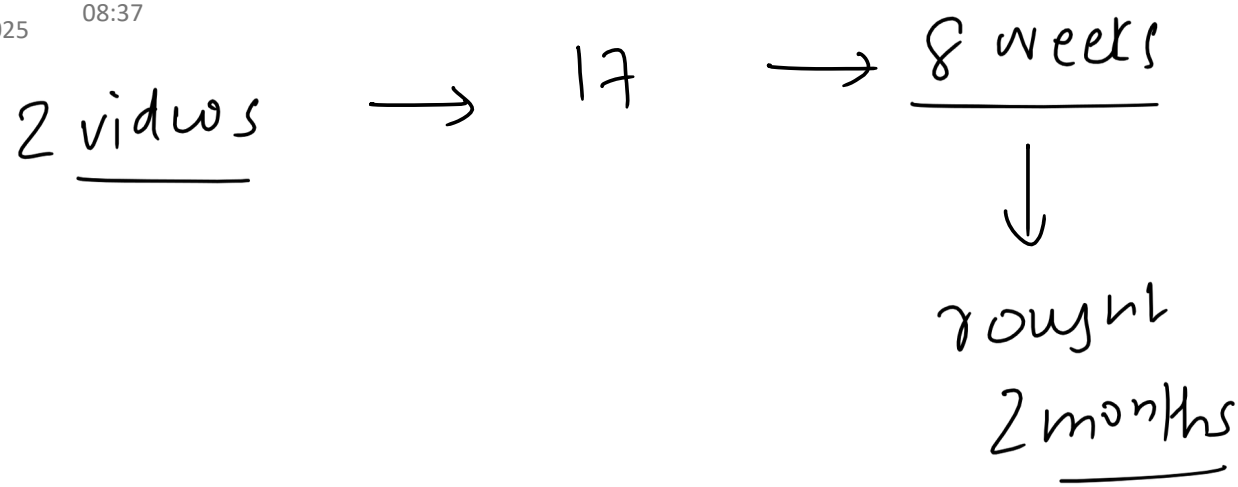
29 January 2025 08:35

1.Updated information
2. Clarity
3. Conceptual understanding
4. The 80percent approach



Timeline

29 January 2025 08:37



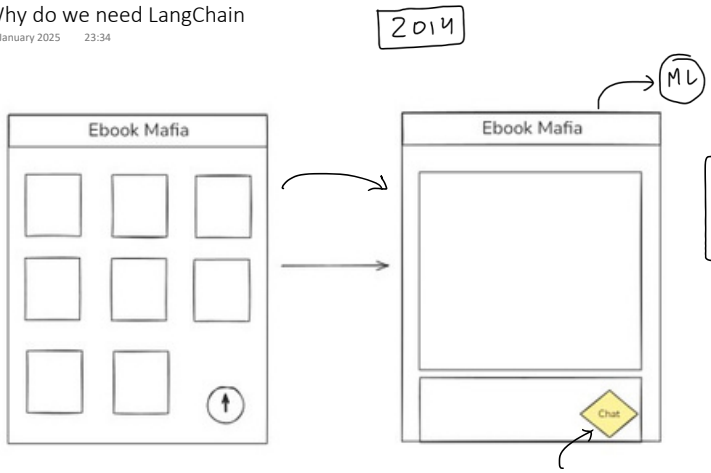
What is LangChain

07 January 2025 23:14

LangChainis an open-source framework for developing applications powered by large language models (LLMs).

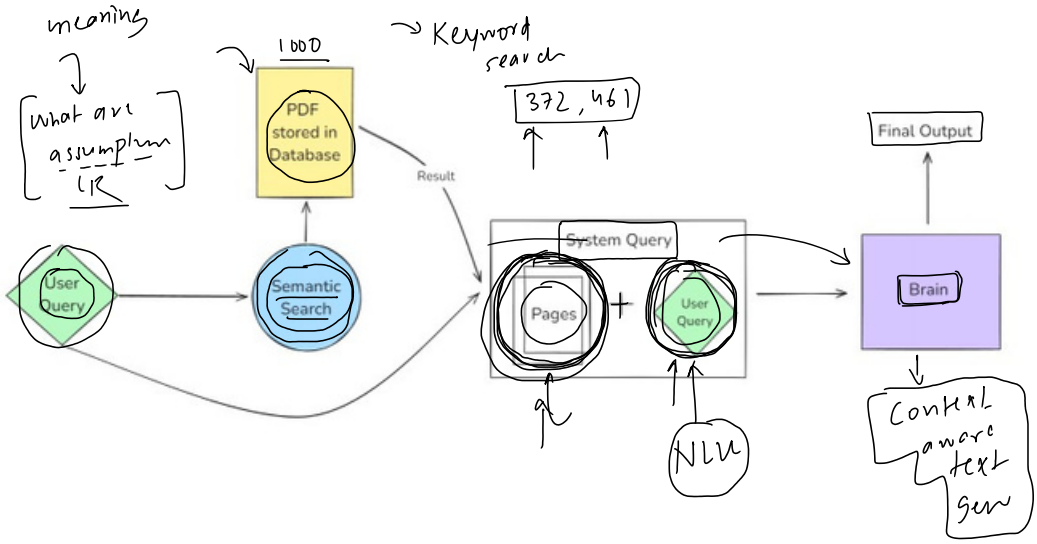
Why do we need LangChain

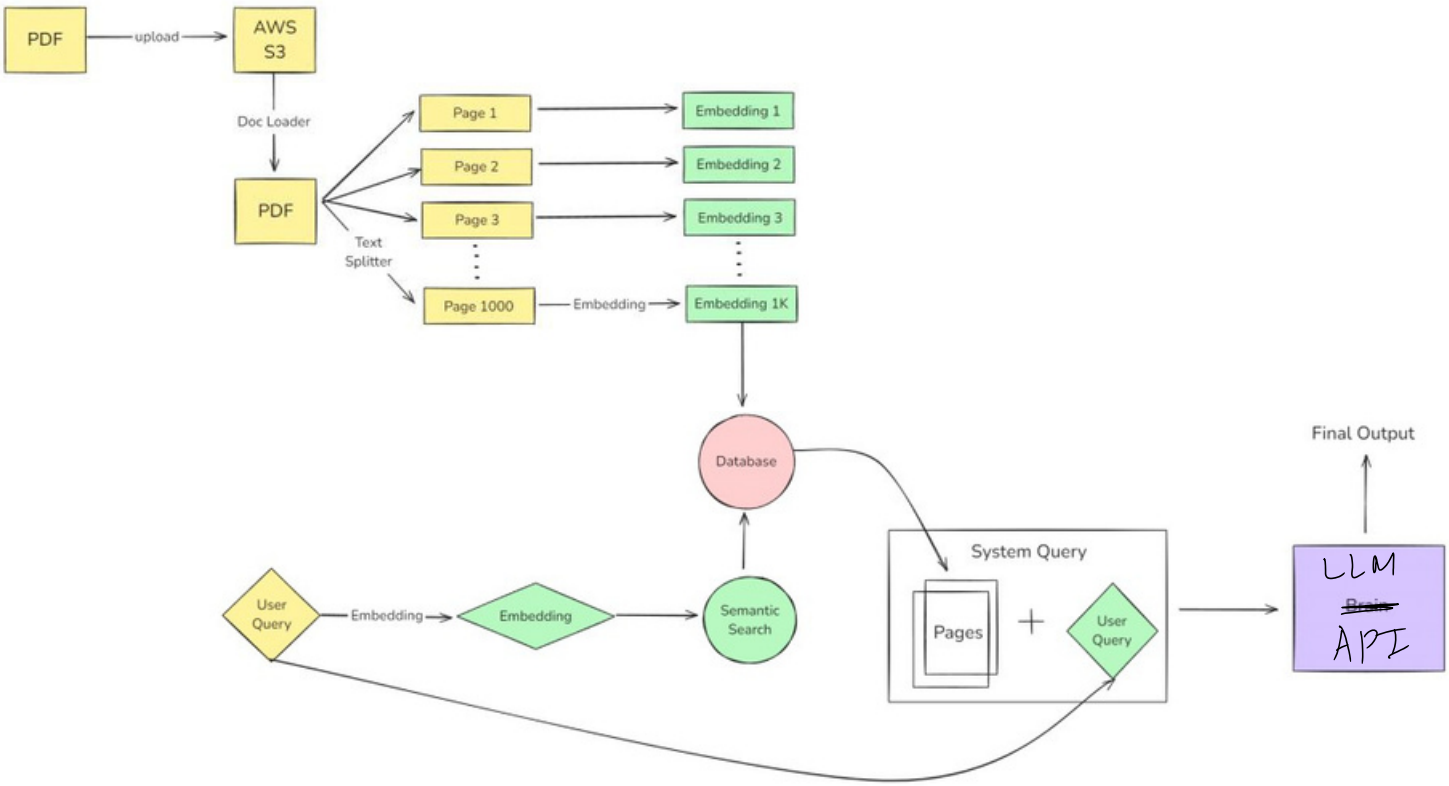
21 January 2025 23:34



Example Queries

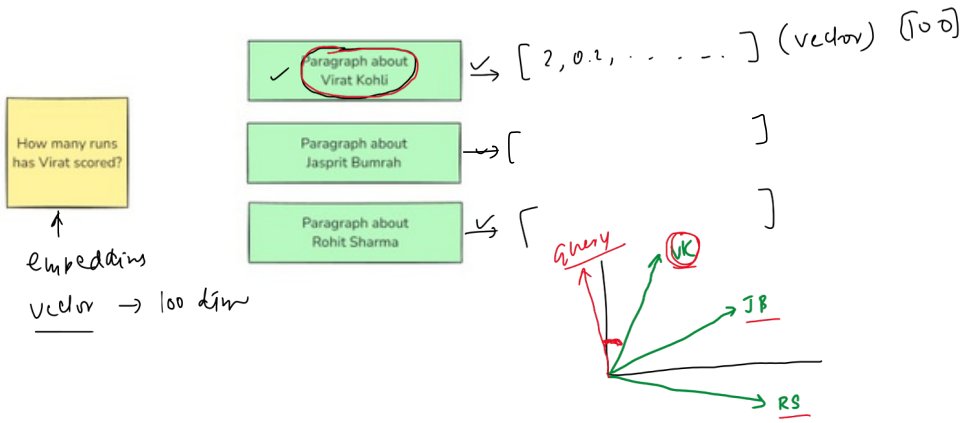
1. Explain page number 5 as if I am a 5 year old
2. Generate a True False exercise on Linear Regression
3. Generatenotes forDecisionTrees





Semantic search

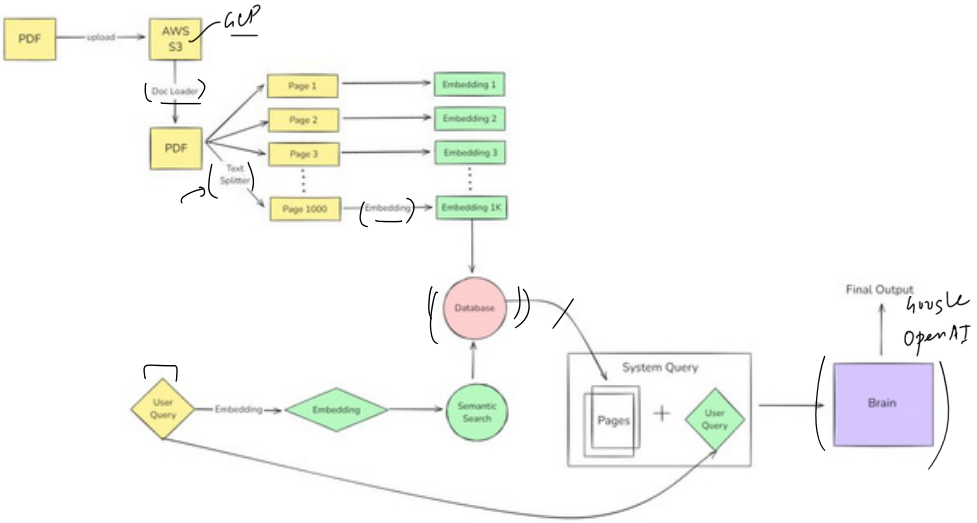
embedding → vector (set of nos.)



Benefits

21 January 2025 23:34

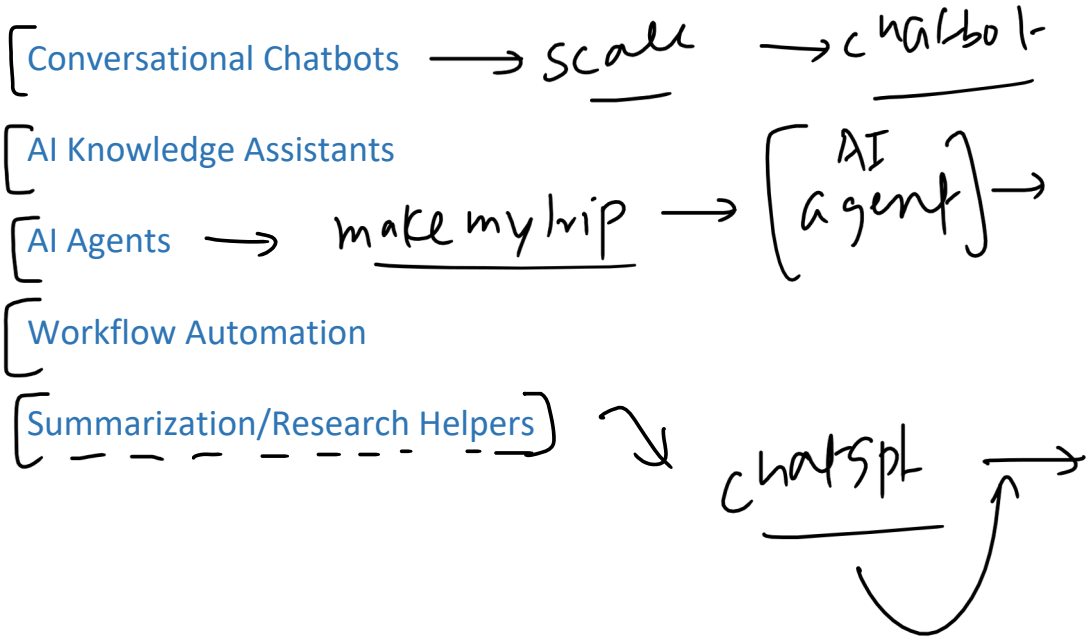
- Concept of chains
- Model Agnostic Development
- Complete ecosystem
- Memory and state handling



LLM
API

What can you build?

21 January 2025 23:34



Alternatives

21 January 2025 23:34

LlamaIndex

Haystack

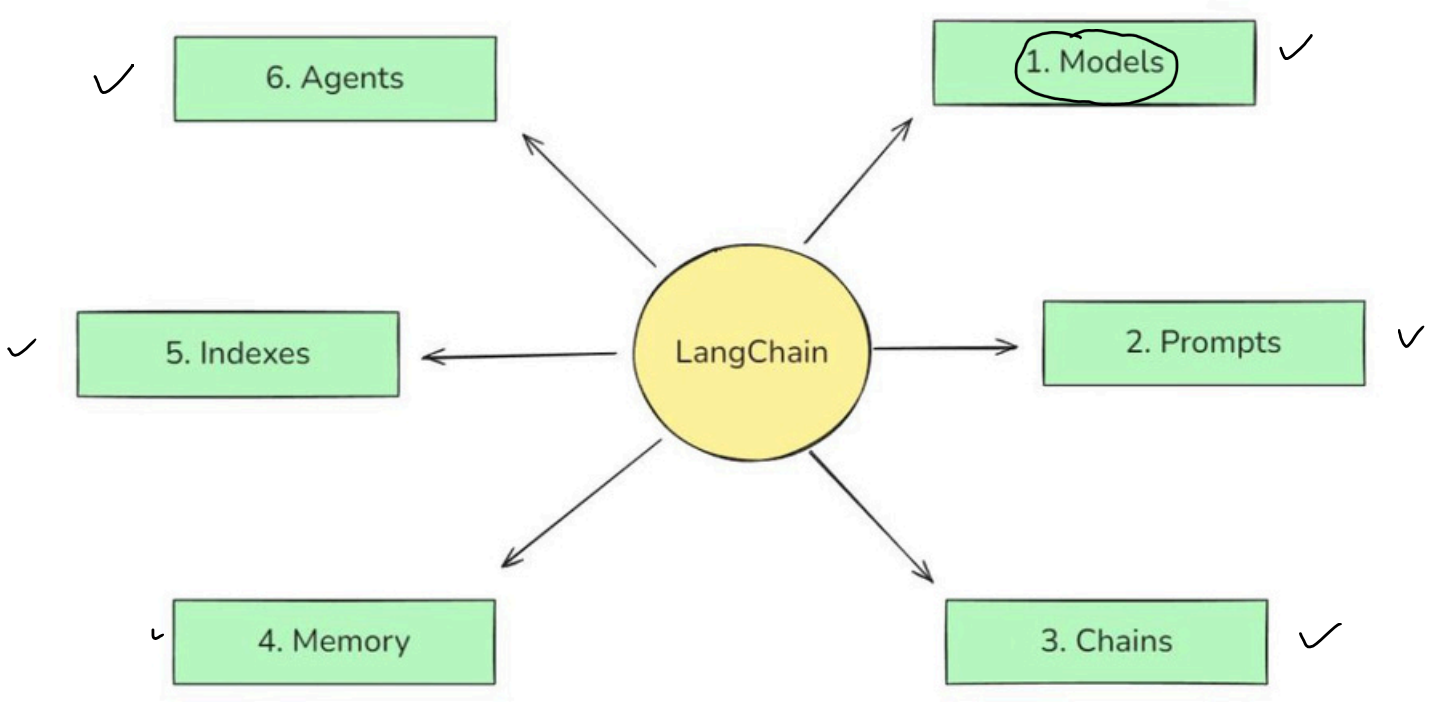
Recap

23 January 2025 10:29

LangChainis an open-source framework for developing applications powered by large language models (LLMs).

LangChain Components

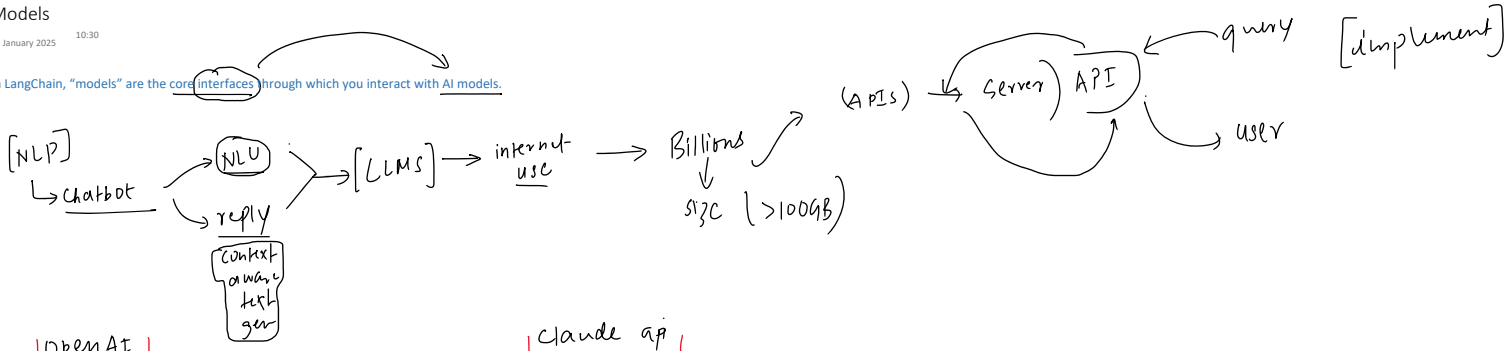
23 January 2025 10:30



Models

23 January 2025 10:30

In LangChain, "models" are the core interfaces through which you interact with AI models.



OpenAI

```
Create a human-like response to a prompt
```

```python
from openai import OpenAI
client = OpenAI()

completion = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {
            "role": "user",
            "content": "Write a haiku about recursion in programming."
        }
    ]
)

print(completion.choices[0].message)
```

Claude api

claude_quickstart.py

```python
import anthropic

client = anthropic.Anthropic()

message = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1000,
    temperature=0,
    system="You are a world-class poet. Respond only with short poems.",
    messages=[
        {
            "role": "user",
            "content": [
                {
                    "type": "text",
                    "text": "Why is the ocean salty?"
                }
            ]
        }
    ]
)
print(message.content)
```

Langchain → API

interface →

OpenAI

```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model='gpt-4', temperature=0)

result = model.invoke("Now divide the result by 1.5")

print(result.content)
```

Langchain

Claude

```python
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv

load_dotenv()

model = ChatAnthropic(model='claude-3-opus-20240229')

result = model.invoke("Hi who are you")

print(result.content)
```

Langchain

[language model]

LLMs

text → text

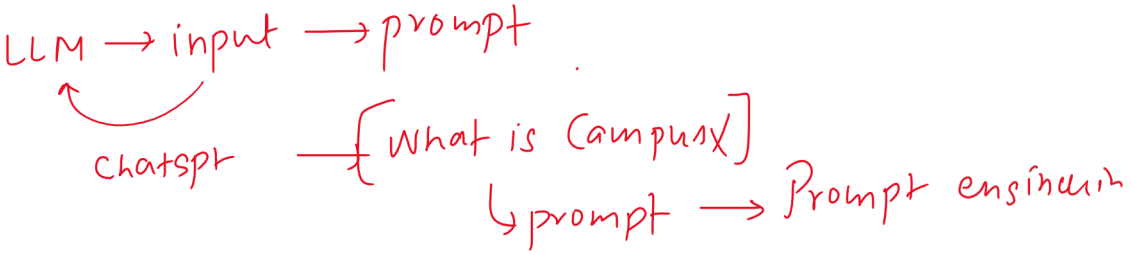
[Embedding model]

text → vector (output)

+ semantic search

Prompts

23 January 2025 10:30



1.Dynamic & Reusable Prompts

```python
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template('Summarize {topic} in {emotion} tone')

print(prompt.format(topic='Cricket', length='fun'))
```

2.Role-Based Prompts

```python
# Define the ChatPromptTemplate using from_template
chat_prompt = ChatPromptTemplate.from_template([
    ("system", "Hi you are a experienced {profession}"),
    ("user", "Tell me about {topic}"),
])

# Format the prompt with the variable
formatted_messages = chat_prompt.format_messages(profession="Doctor", topic="Viral Fever")
```

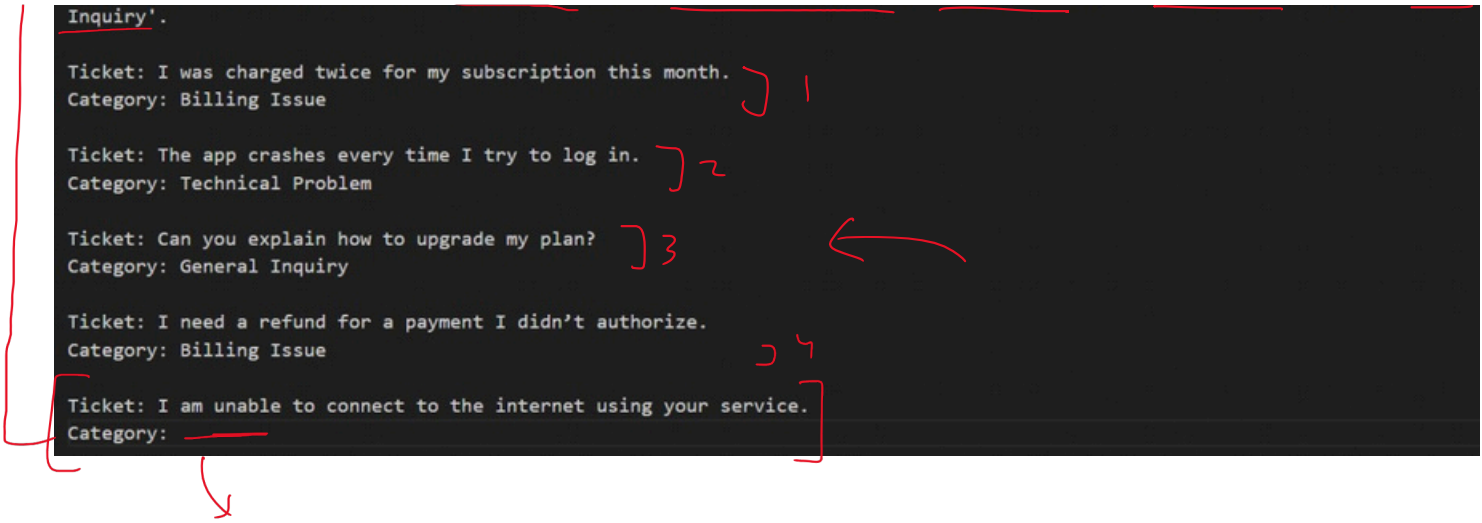
3.Few Shot Prompting

```python
examples = [
    {"input": "I was charged twice for my subscription this month.", "output": "Billing Issue"},
    {"input": "The app crashes every time I try to log in.", "output": "Technical Problem"},
    {"input": "Can you explain how to upgrade my plan?", "output": "General Inquiry"},
    {"input": "I need a refund for a payment I didn't authorize.", "output": "Billing Issue"},
]
```

```python
# Step 2: Create an example template
example_template = """
Ticket: {input}
Category: {output}
"""
```

```python
# Step 3: Build the few-shot prompt template
few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=PromptTemplate(input_variables=["input", "output"], template=example_template),
    prefix="Classify the following customer support tickets into one of the categories: 'Billing Issue', 'Technical Problem', or 'General Inquiry'.\n\n",
    suffix="\nTicket: {user_input}\nCategory:",
    input_variables=["user_input"],
)
```

```
Classify the following customer support tickets into one of the categories: 'Billing Issue', 'Technical Problem', or 'General Inquiry'.

Ticket: I was charged twice for my subscription this month.
Category: Billing Issue
```


Inquiry'.
Ticket: I was charged twice for my subscription this month.
Category: Billing Issue
Ticket: The app crashes every time I try to log in.
Category: Technical Problem
Ticket: Can you explain how to upgrade my plan?
Category: General Inquiry
Ticket: I need a refund for a payment I didn't authorize.
Category: Billing Issue
Ticket: I am unable to connect to the internet using your service.
Category:

Chains

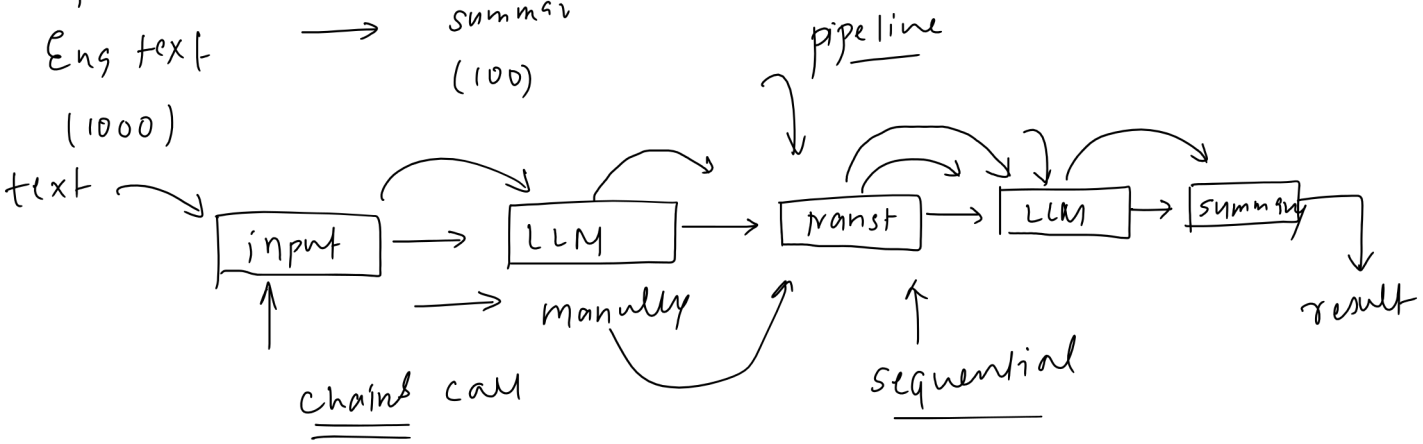
23 January 2025 10:31

Pipelines → LLM + Pipeline

input

Eng text (1000)

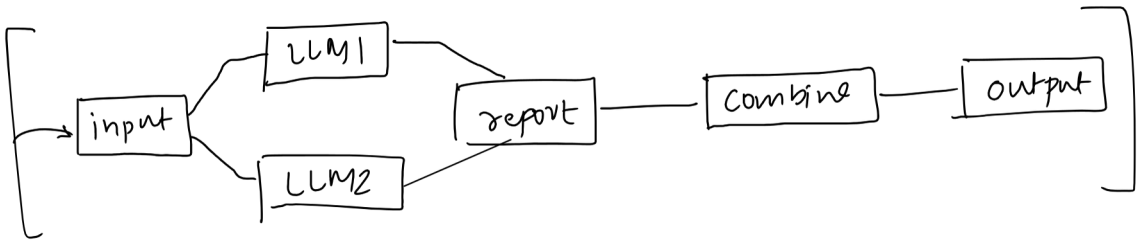
→ Hindi summary (100)



Complex pipeline

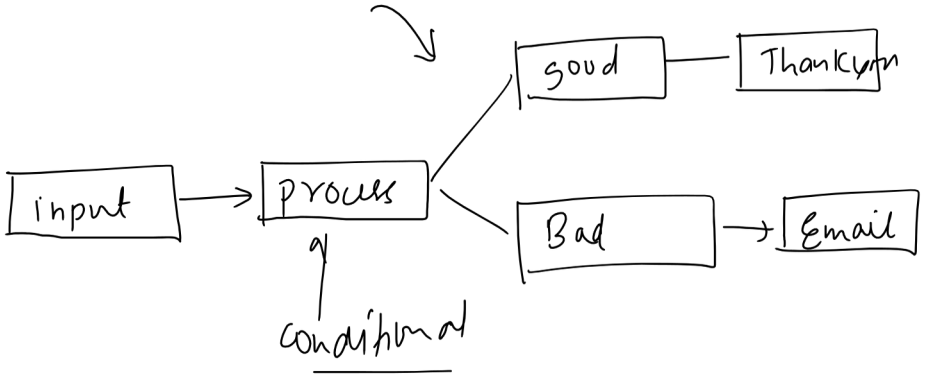
Conditional chains

Parallel chain



AI Agent

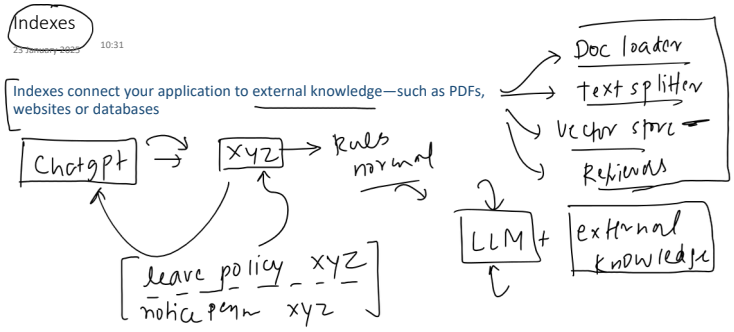
feedback



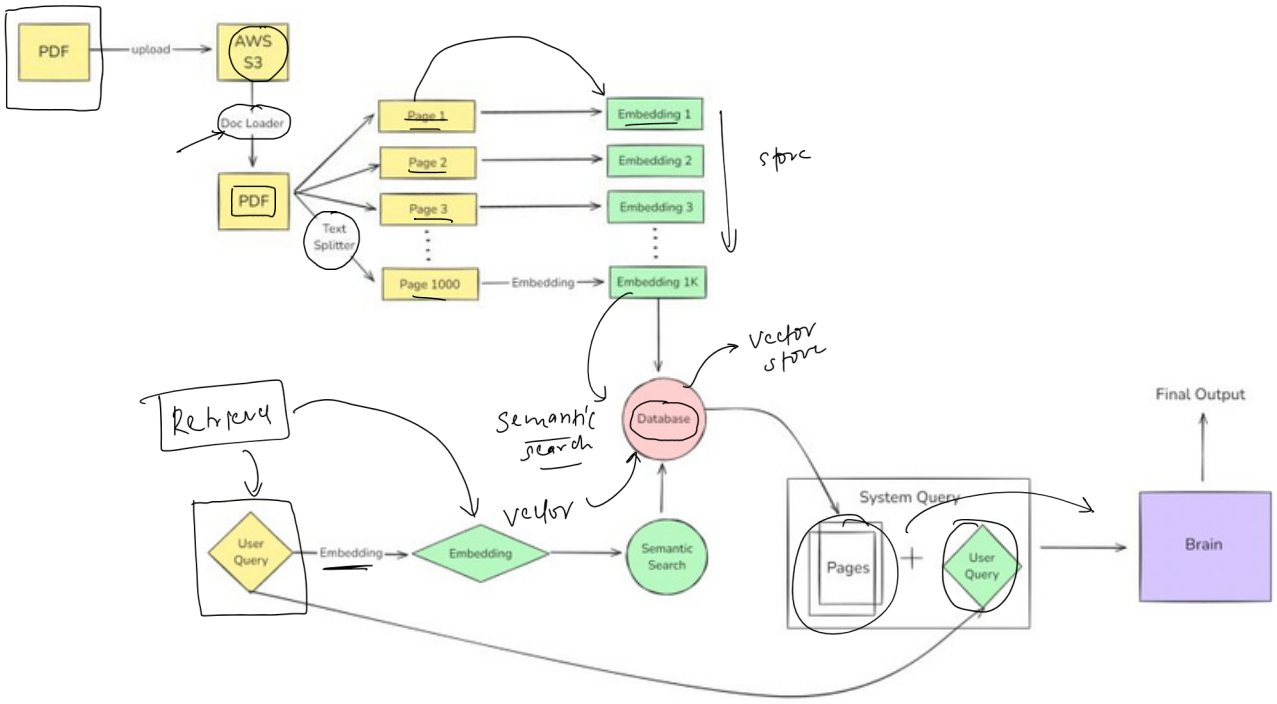
Indexes

23 January 2023 10:31

Indexes connect your application to external knowledge—such as PDFs, websites or databases



Rulebook 1000 pages



LLM = API

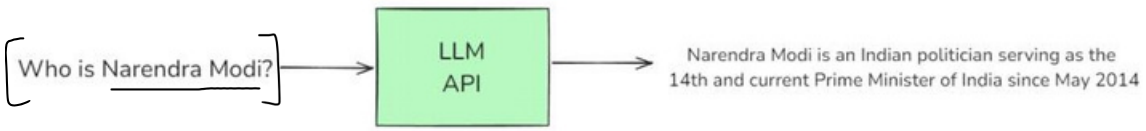
Memory

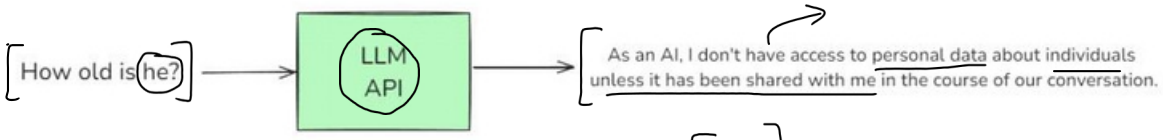
23 January 2025 10:31

LLM API calls are stateless

Chatbot

GPT



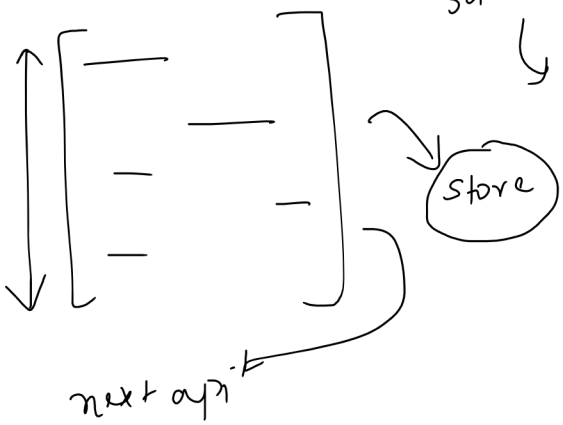


last n intrs

[100]

summary → api call

- ConversationBufferMemory: Stores a transcript of recent messages. Great for short chats but can grow large quickly.
- ConversationBufferWindowMemory: Only keeps the last N interactions to avoid excessive token usage.
- Summarizer-Based Memory: Periodically summarizes older chat segments to keep a condensed memory footprint.
- Custom Memory: For advanced use cases, you can store specialized state (e.g., the user's preferences or key facts about them) in a custom memory class.



Agents

23January 2025 10:31

AI Agents

+ LLMs (NLU + Textgen) → chatbot

+

Chatbot AI agent

↓

Chatbot with super

AI agent

travel website

+ Shimla ~~Manali~~

↓

24th Jan

API ↘ Indigo

↓ Book the flight

AI agent-

+ [Reasoning capability]

+ TOOLS

Tools → Calculator, Weather API

↓

User → AI agent → Can you multiply today's temp of Delhi with 3

+ Chain of thought

→ [Delhi temp]

→ 25°C

→ 25, 3, *

75 → (75)

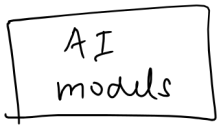
Recap

10 February 2025 09:21

+ Models

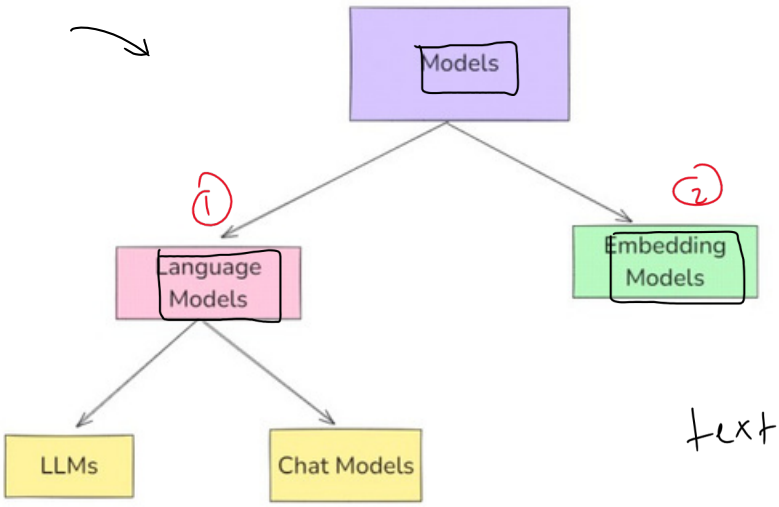
What are Models

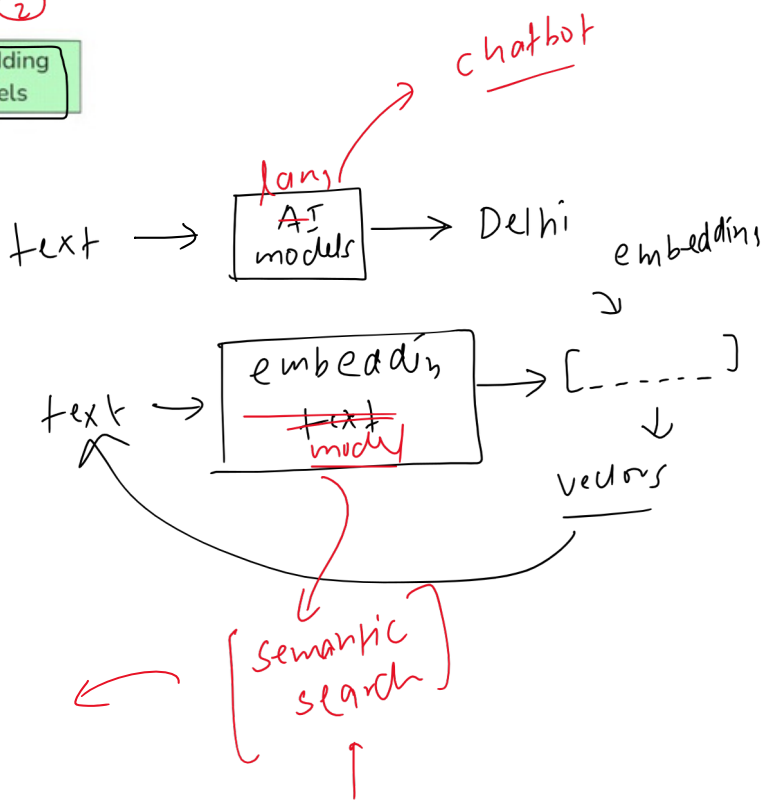
07 January 2025 23:15



The Model Component in LangChain is a crucial part of the framework, designed to facilitate interactions with various language models and embedding models.

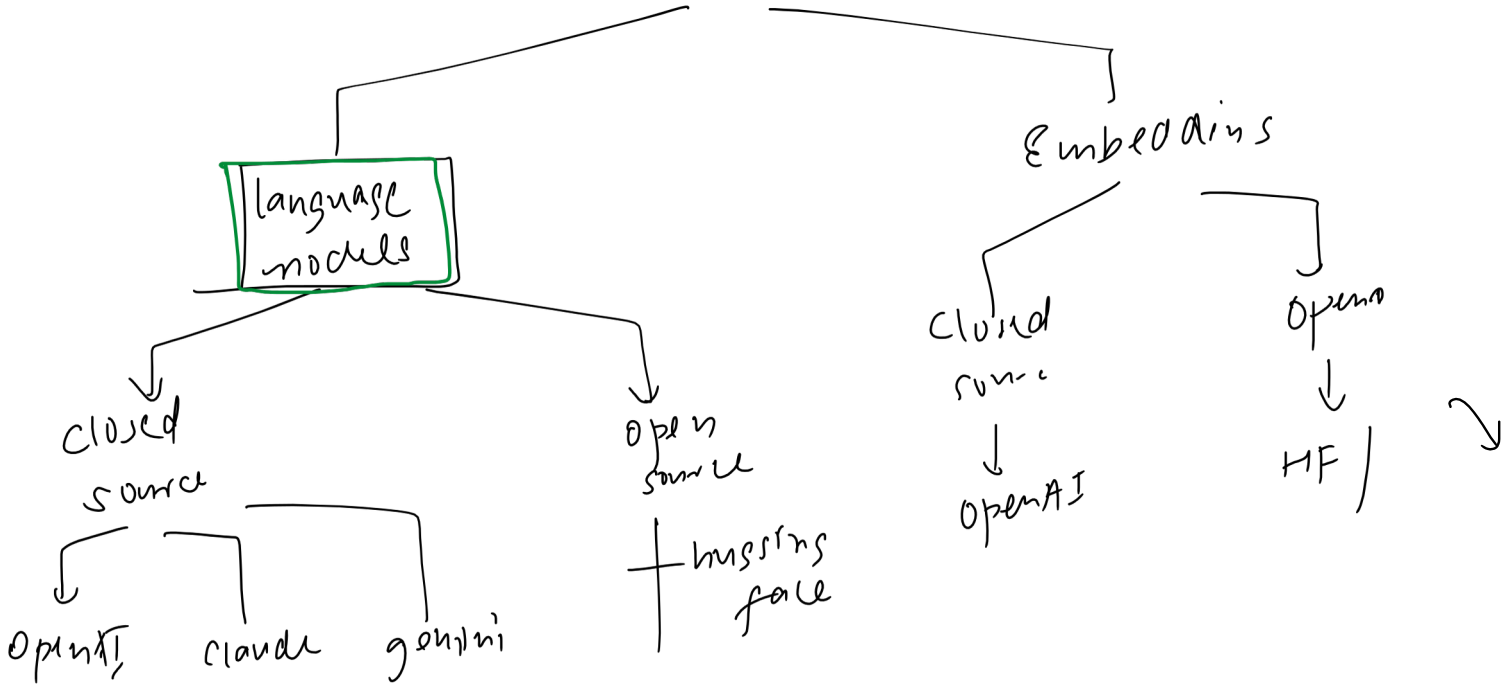
It abstracts the complexity of working directly with different LLMs, chat models, and embedding models, providing a uniform interface to communicate with them. This makes it easier to build applications that rely on AI-generated text, text embeddings for similarity search, and retrieval-augmented generation (RAG).





Plan of Action

10 February 2025 09:21



Open source using huggingface api:

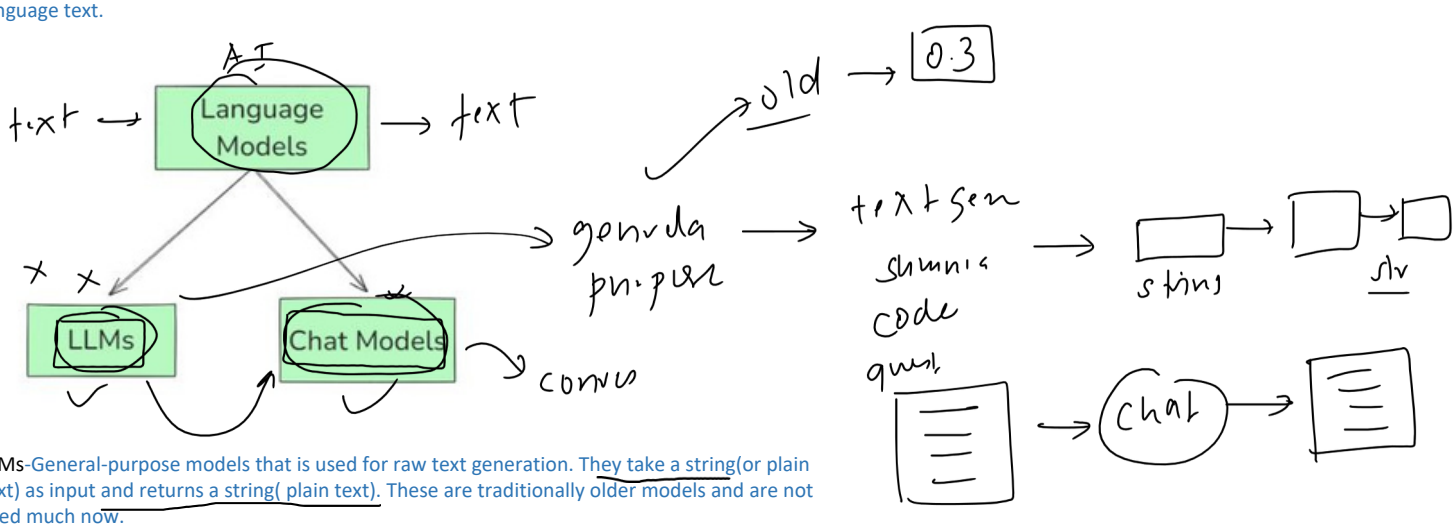
```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
llm = HuggingFaceEndpoint(
    repo_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation"
)

model = ChatHuggingFace(llm=llm)
```

model download from hugging face and run locally.

```
from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline
import os

os.environ['HF_HOME'] = 'D:/huggingface_cache'

llm = HuggingFacePipeline.from_model_id(
    model_id='TinyLlama/TinyLlama-1.1B-Chat-v1.0',
    task='text-generation',
    pipeline_kwargs=dict(
        temperature=0.5,
        max_new_tokens=100
    )
)
model = ChatHuggingFace(llm=llm)
```

Language Models

Language Models are AI systems designed to process, generate, and understand natural language text.



LLMs-General-purpose models that is used for raw text generation. They take a string(or plain text) as input and returns a string(plain text). These are traditionally older models and are not used much now.

Chat Models-Language models that are specialized for conversational tasks. They take a sequence of messages as inputs and return chat messages as outputs (as opposed to using plain text). These are traditionally newer models and used more in comparison to the LLMs.

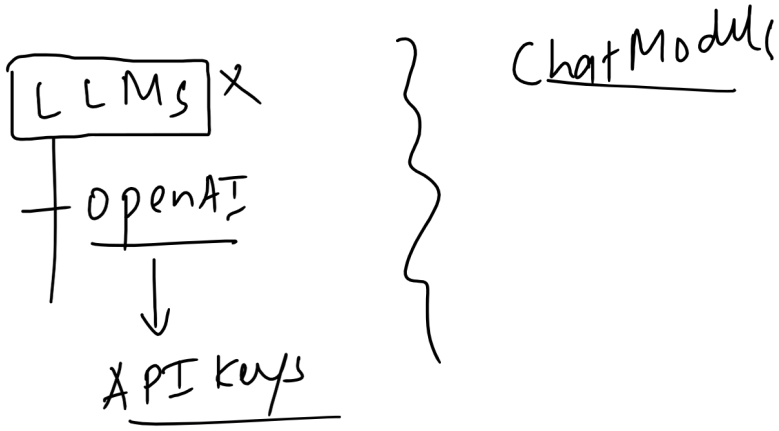
Feature	LLMs (Base Models)	Chat Models (Instruction-Tuned)
Purpose	Free-form text generation	Optimized for multi-turn conversations
Training Data	General text corpora (books, articles)	Fine-tuned on chat datasets (dialogues, user-assistant conversations)
Memory & Context	No built-in memory	Supports structured conversation history
Role Awareness	No understanding of "user" and "assistant" roles	Understands "system", "user", and "assistant" roles
Example Models	GPT-3, Llama-2-7B, Mistral-7B, OPT-1.3B	GPT-4, GPT-3.5-turbo, Llama-2-Chat, Mistral-Instruct, Claude
Use Cases	Text generation, summarization, translation, creative writing, code generation	Conversational AI, chatbots, virtual assistants, customer support, AI tutors

LLM

```python
from langchain_openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

llm = OpenAI(model='gpt-3.5-turbo-instruct')

result = llm.invoke("What is the capital of India")

print(result)
```

Demo

1. OpenAI
2. Anthropic
3. Google
4. HuggingFace



`temperature` is a parameter that **controls the randomness of a language model's output**. It affects how **creative or deterministic** the responses are.

- **Lower values** (`0.0 - 0.3`) → More **deterministic** and predictable.
- **Higher values** (`0.7 - 1.5`) → More **random**, creative, and diverse.

Use Case	Recommended Temperature
Factual answers (math, code, facts)	`0.0 - 0.3`
Balanced response (general QA, explanations)	`0.5 - 0.7`
Creative writing, storytelling, jokes	`0.9 - 1.2`
Maximum randomness (wild ideas, brainstorming)	`1.5+`

Setup

10 February 2025 09:21

Chat Model

```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model='gpt-4', temperature=1.5, max_completion_tokens=10)

result = model.invoke("Write a 5 line poem on cricket")

print(result.content)
```

```python
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv

load_dotenv()

model = ChatAnthropic(model='claude-3-5-sonnet-20241022')

result = model.invoke('What is the capital of India')

print(result.content)
```

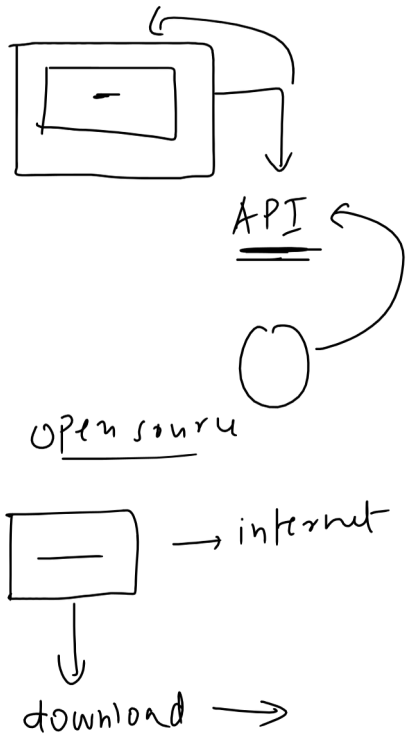
```python
from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv

load_dotenv()

model = ChatGoogleGenerativeAI(model='gemini-1.5-pro')

result = model.invoke('What is the capital of India')

print(result.content)
```

Open Source Models

11 February 2025 08:58

Open-source language models are freely available AI modelsthat can be downloaded, modified, fine-tuned, and deployedwithout restrictions from a central provider. Unlike closed-source models such as OpenAI's GPT-4, Anthropic's Claude, or Google's Gemini, open-source models allow full control and customization.

Feature	Open-Source Models	Closed-Source Models
Cost	Free to use (no API costs)	Paid API usage (e.g., OpenAI charges per token)
Control	Can modify, fine-tune, and deploy anywhere	Locked to provider's infrastructure
Data Privacy	Runs locally (no data sent to external servers)	Sends queries to provider's servers
Customization	Can fine-tune on specific datasets	No access to fine-tuning in most cases
Deployment	Can be deployed on **on-premise** servers or **cloud**	Must use vendor's API



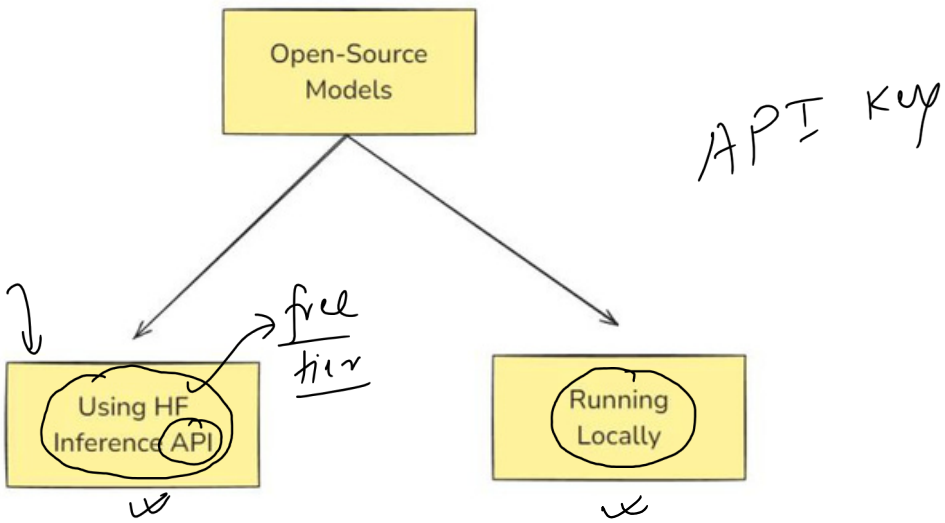
Some Famous Open Source Models

Model	Developer	Parameters	Best Use Case
LLaMA-2-7B/13B/70B	Meta AI	7B - 70B	General-purpose text generation
Mixtral-8x7B	Mistral AI	8x7B (MoE)	Efficient & fast responses
Mistral-7B	Mistral AI	7B	Best small-scale model (outperforms LLaMA-2-13B)
Falcon-7B/40B	TII UAE	7B - 40B	High-speed inference
BLOOM-176B	BigScience	176B	Multilingual text generation
GPT-J-6B	EleutherAI	6B	Lightweight and efficient
GPT-NeoX-20B	EleutherAI	20B	Large-scale applications
StableLM	Stability AI	3B - 7B	Compact models for chatbots

Where to find them?

HuggingFace-The largest repository of open-source LLMs

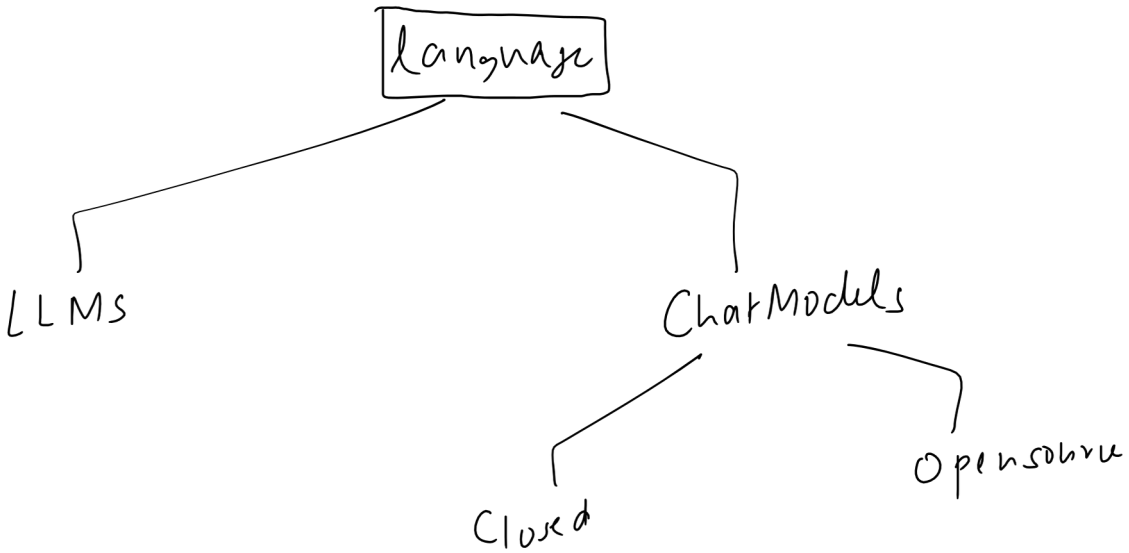
Ways to use Open-source Models

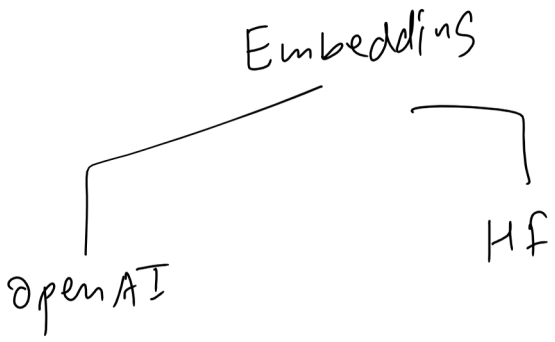


Disadvantages

Disadvantage	Details
High Hardware	Running large models (e.g., LLaMA-2-70B) requires expensive GPUs.

Disadvantage	Details
High Hardware Requirements	Running large models (e.g., LLaMA-2-70B) requires expensive GPUs.
Setup Complexity	Requires installation of dependencies like **PyTorch**, **CUDA**, **transformers**.
Lack of RLHF	Most open-source models don't have **fine-tuning with human feedback**, making them weaker in instruction-following.
Limited Multimodal Abilities	Open models don't support **images, audio, or video** like GPT-4V.





Using a HF inference api

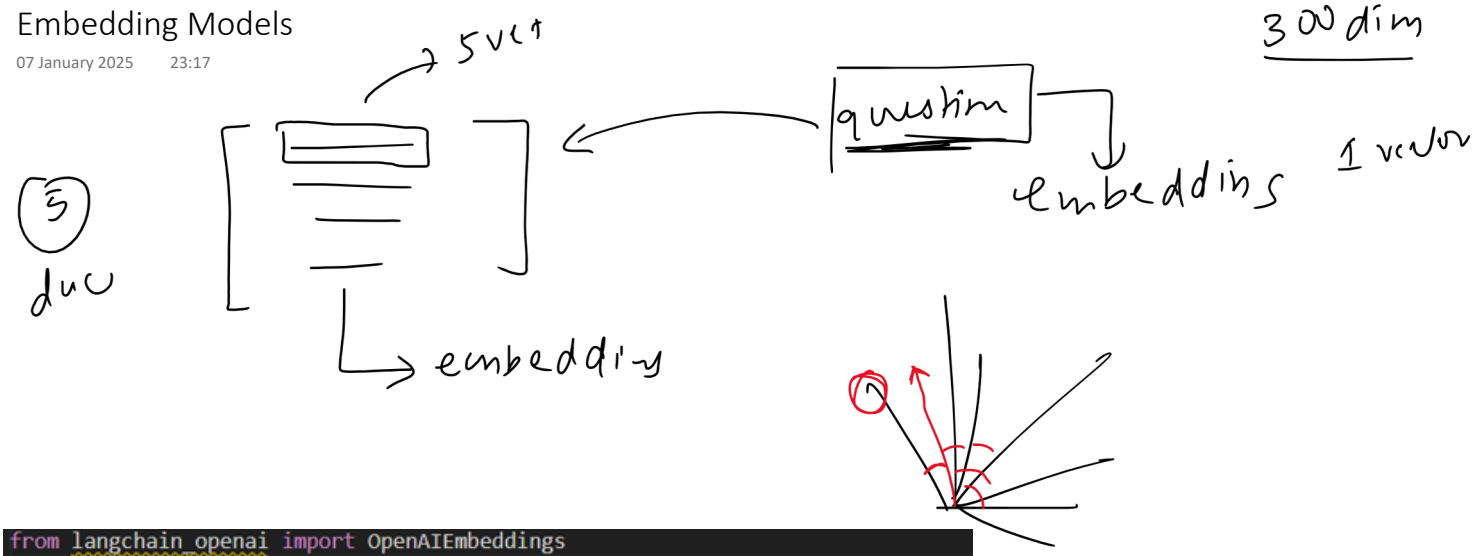
```python
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation"
)

model = ChatHuggingFace(llm=llm)

result = model.invoke("What is the capital of India")

print(result.content)
```

Running Locally

```python
from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline
import os

os.environ['HF_HOME'] = 'D:/huggingface_cache'

llm = HuggingFacePipeline.from_model_id(
    model_id='TinyLlama/TinyLlama-1.1B-Chat-v1.0',
    task='text-generation',
    pipeline_kwargs=dict(
        temperature=0.5,
        max_new_tokens=100
    )
)
model = ChatHuggingFace(llm=llm)

result = model.invoke("What is the capital of India")

print(result.content)
```

Embedding Models

07 January 2025 23:17



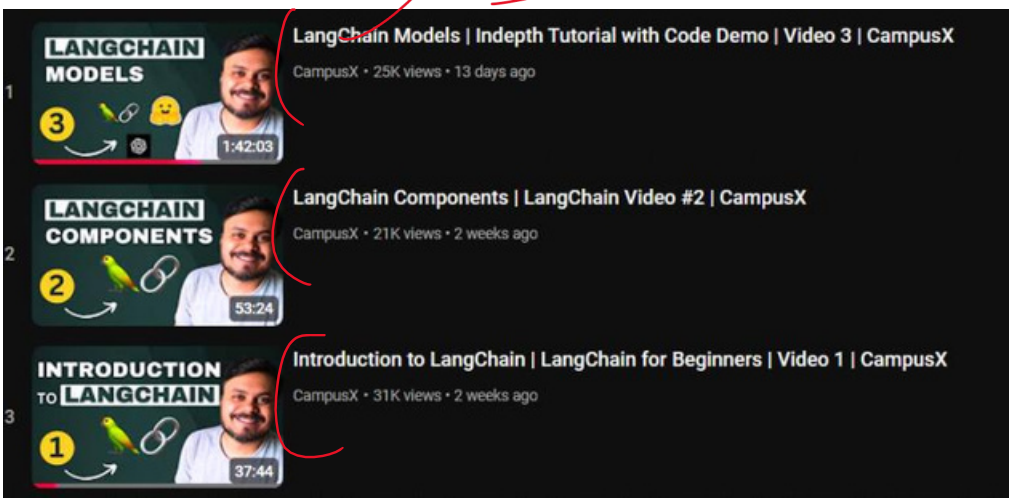

```python
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv

load_dotenv()

embedding = OpenAIEmbeddings(model='text-embedding-3-large', dimensions=32)

result = embedding.embed_query("Delhi is the capital of India")

print(str(result))
```

```python
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv

load_dotenv()

embedding = OpenAIEmbeddings(model='text-embedding-3-large', dimensions=32)

documents = [
    "Delhi is the capital of India",
    "Kolkata is the capital of West Bengal",
    "Paris is the capital of France"
]

result = embedding.embed_documents(documents)

print(str(result))
```

```python
from langchain_huggingface import HuggingFaceEmbeddings

embedding = HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2')

documents = [
    "Delhi is the capital of India",
    "Kolkata is the capital of West Bengal",
    "Paris is the capital of France"
]

vector = embedding.embed_documents(documents)

print(str(vector))
```

Recap

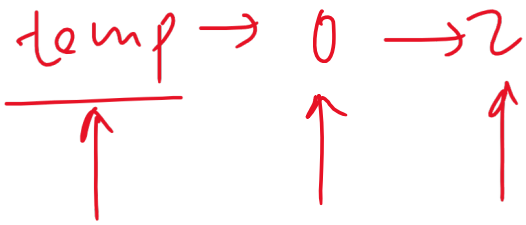
24February 2025 09:42



④ → Prompts

A mistake from my side!

24 February 2025 09:42

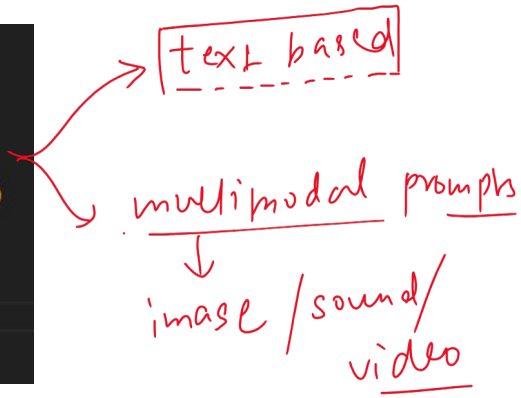


What are Prompts

10 January 2025 08:37

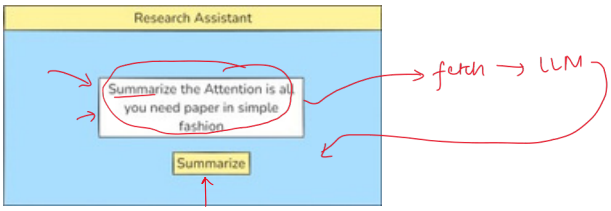
Promptsare the input instructions or queries given to a model to guide its output.

```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model='gpt-4', temperature=1.5, max_completion_tokens=10)

result = model.invoke("Write a 5 line poem on cricket")

print(result.content)
```



Static vs Dynamic Prompts

14 February 2025 00:01



A **PromptTemplate**in LangChain is a structured way to create prompts dynamically by inserting variables into a predefined template. Instead of hardcoding prompts, PromptTemplateallows you to define placeholders that can be filled in at runtime with different inputs.

This makes it reusable, flexible, and easy to manage, especially when working with dynamic user inputs or automated workflows.

```
paper_input = st.selectbox( "Select Research Paper Name", ["Select...", "Attention Is All You Need",
"BERT: Pre-training of Deep Bidirectional Transformers", "GPT-3: Language Models are Few-Shot
Learners", "Diffusion Models Beat GANs on Image Synthesis"] )

style_input = st.selectbox( "Select Explanation Style", ["Beginner-Friendly", "Technical", "Code-
Oriented", "Mathematical"] )

length_input = st.selectbox( "Select Explanation Length", ["Short (1-2 paragraphs)", "Medium (3-5
paragraphs)", "Long (detailed explanation)"] )
```

```python
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI()

# detailed way
template2 = PromptTemplate(
    template='Greet this person in 5 languages. The name of the person is {name}',
    input_variables=['name']
)

# fill the values of the placeholders
prompt = template2.invoke({'name':'nitish'})

result = model.invoke(prompt)

print(result.content)
```

Prompt Template

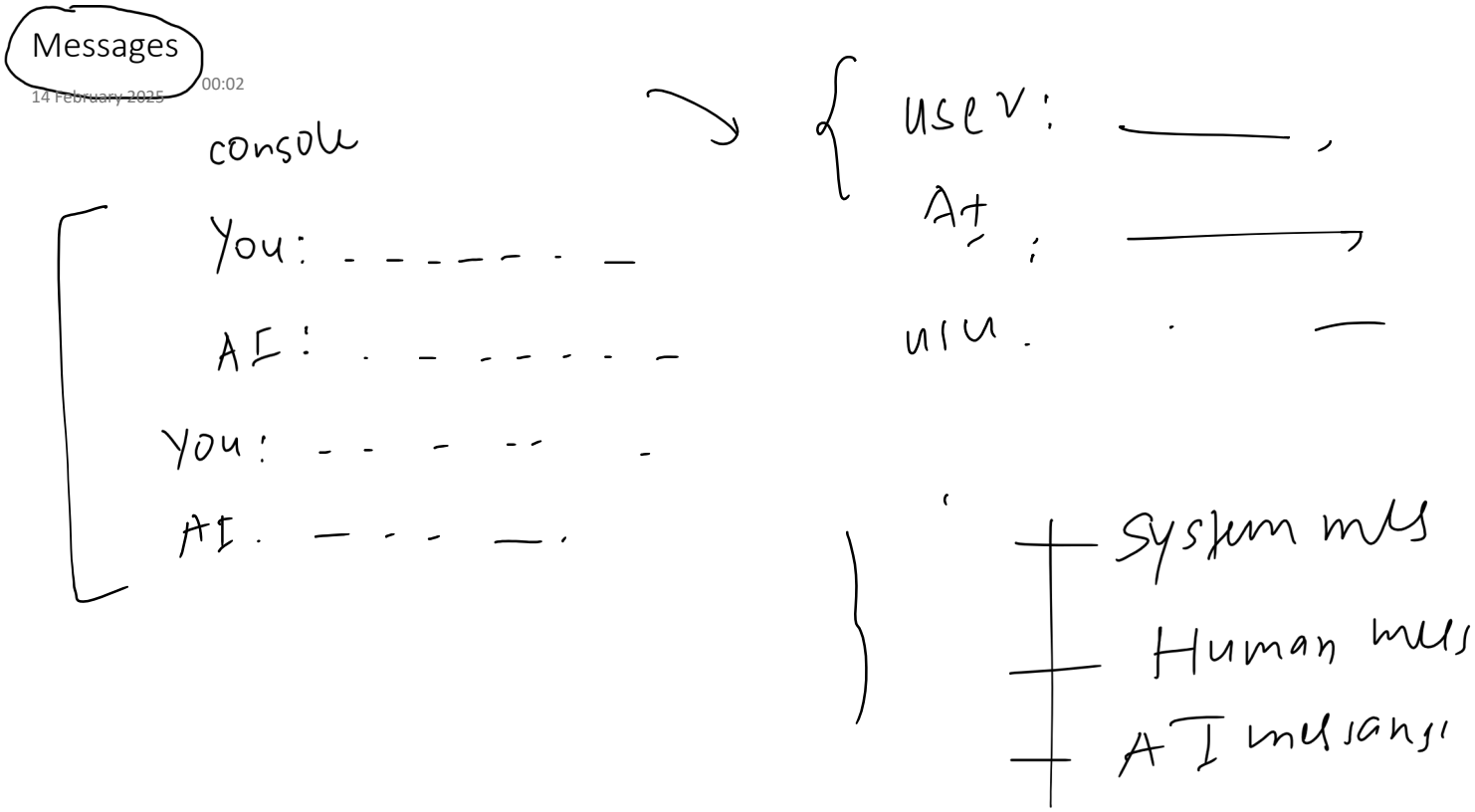
10 January 2025 08:37

A **PromptTemplate**in LangChain is a structured way to create prompts dynamically by inserting variables into a predefined template. Instead of hardcoding prompts, PromptTemplateallows you to define placeholders that can be filled in at runtime with different inputs.

This makes it reusable, flexible, and easy to manage, especially when working with dynamic user inputs or automated workflows.

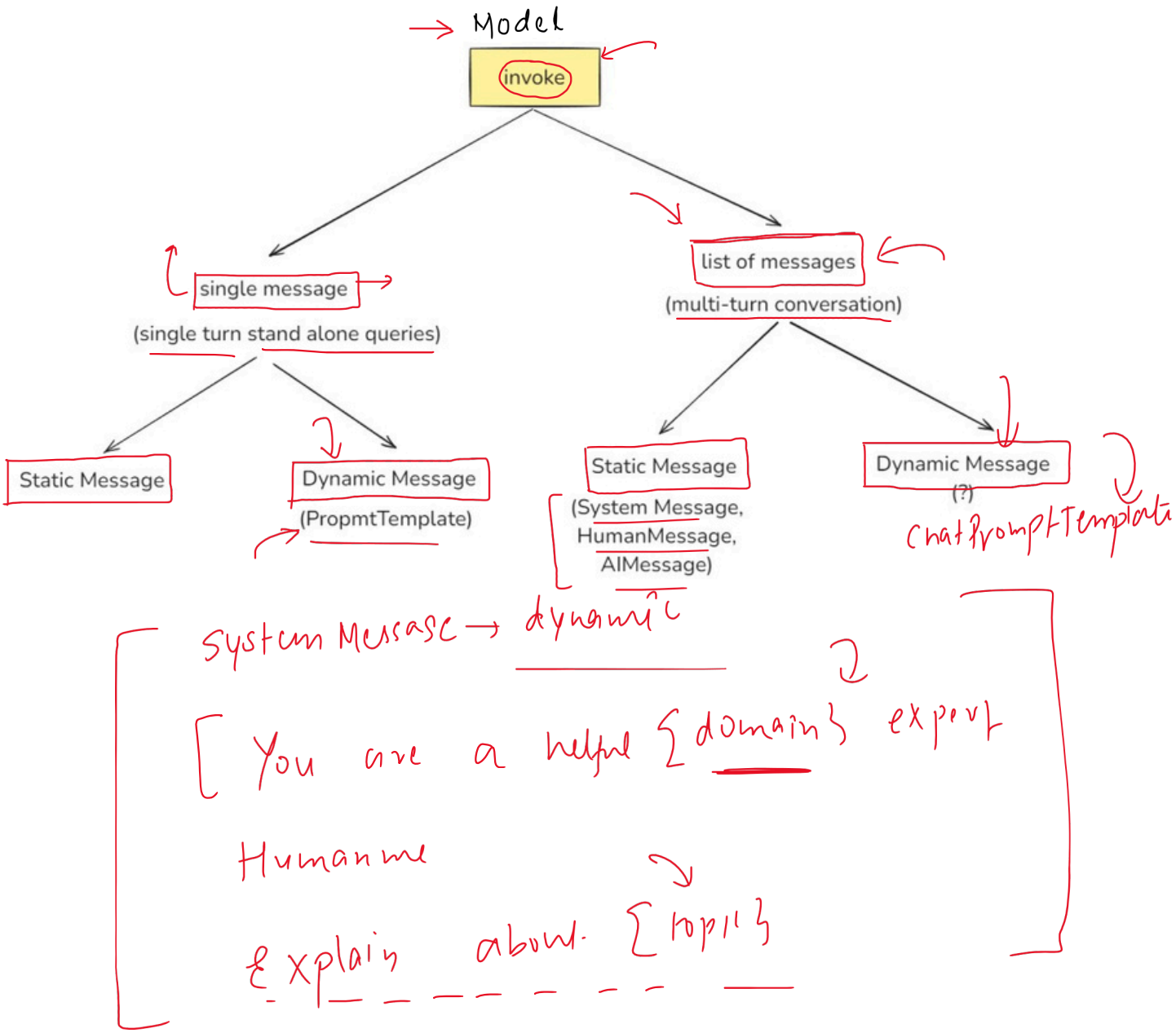
Why use PromptTemplate over f strings?

1.Default validation
2. Reusable
3.LangChain Ecosystem


Messages
14 February 2025
00:02
console
You:
AI:
You:
AI.
user:
AI:
system msg
Human msg
AI instruction

Chat Prompt Templates

14 February 2025 00:02

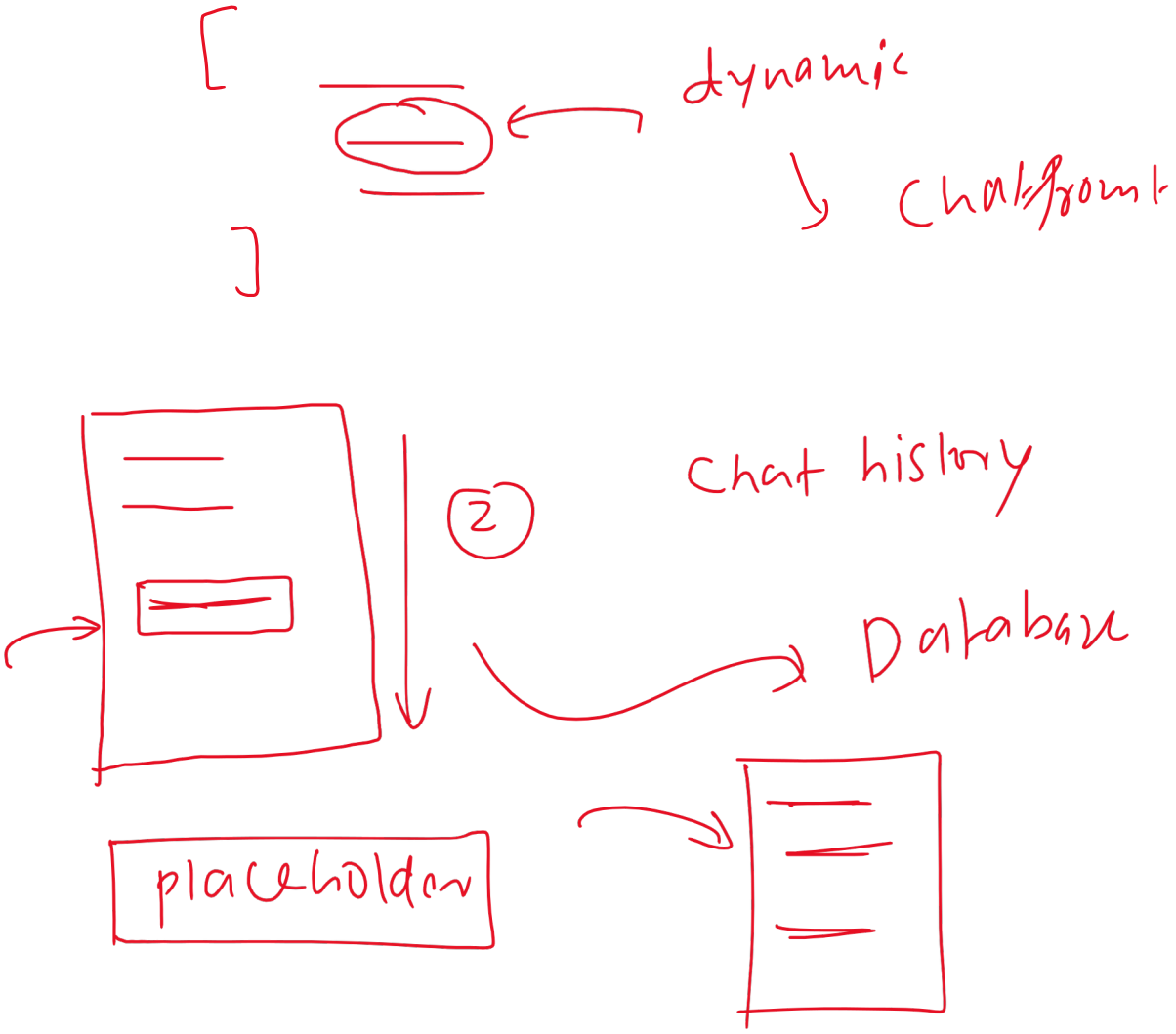



```python
from langchain_core.prompts import ChatPromptTemplate

chat_template = ChatPromptTemplate([
    ('system', 'You are a helpful {domain} expert'),
    ('human', 'Explain in simple terms, what is {topic}')
])

prompt = chat_template.invoke({'domain':'cricket','topic':'Dusra'})

print(prompt)
```

Message Placeholder

14 February 2025 00:09

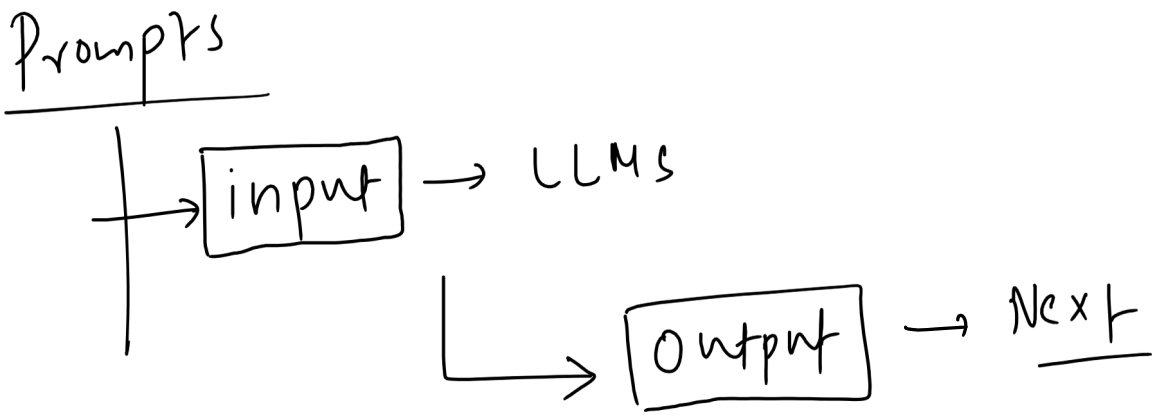
A MessagesPlaceholderin LangChain is a special placeholderused inside a ChatPromptTemplateto dynamically insert chat historyor a list of messagesat runtime.




```python
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
# chat template
chat_template = ChatPromptTemplate([
    ('system','You are a helpful customer support agent'),
    MessagesPlaceholder(variable_name='chat_history'),
    ('human','{query}')
])

chat_history = []
# load chat history
with open('chat_history.txt') as f:
    chat_history.extend(f.readlines())

print(chat_history)

# create prompt
prompt = chat_template.invoke({'chat_history':chat_history, 'query':'Where is my refund'})

print(prompt)
```

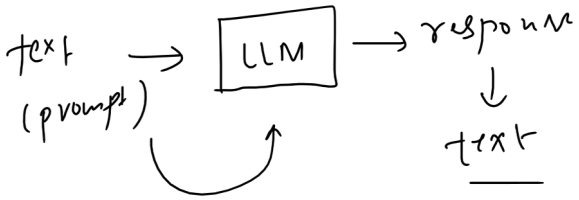
Recap

28 February 2025 00:12



Structured Output

28 February 2025 00:13



In LangChain, structured outputrefers to the practice of having language models return responses in a well-defined data format (for example, JSON), rather than free-form text. This makes the model output easier to parse and work with programmatically.

[Prompt] - *Can you create a one-day travel itinerary for Paris?*

[LLM's Unstructured Response]

Here's a suggested itinerary: *Morning: Visit the Eiffel Tower.*

Afternoon: Walk through the Louvre Museum.

Evening: Enjoy dinner at a Seine riverside café.

[JSON enforced output]

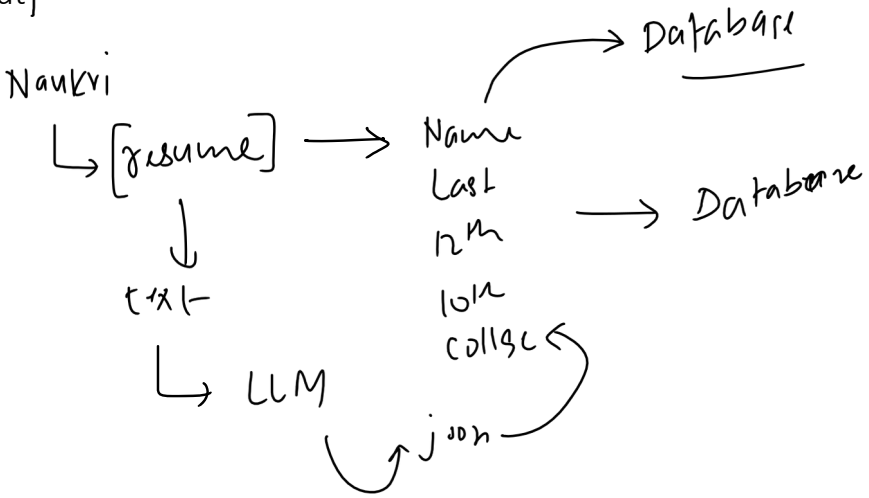
```
[
  {"time": "Morning", "activity": "Visit the Eiffel Tower"},
  {"time": "Afternoon", "activity": "Walk through the Louvre Museum"},
  {"time": "Evening", "activity": "Enjoy dinner at a Seine riverside café"}
]
```

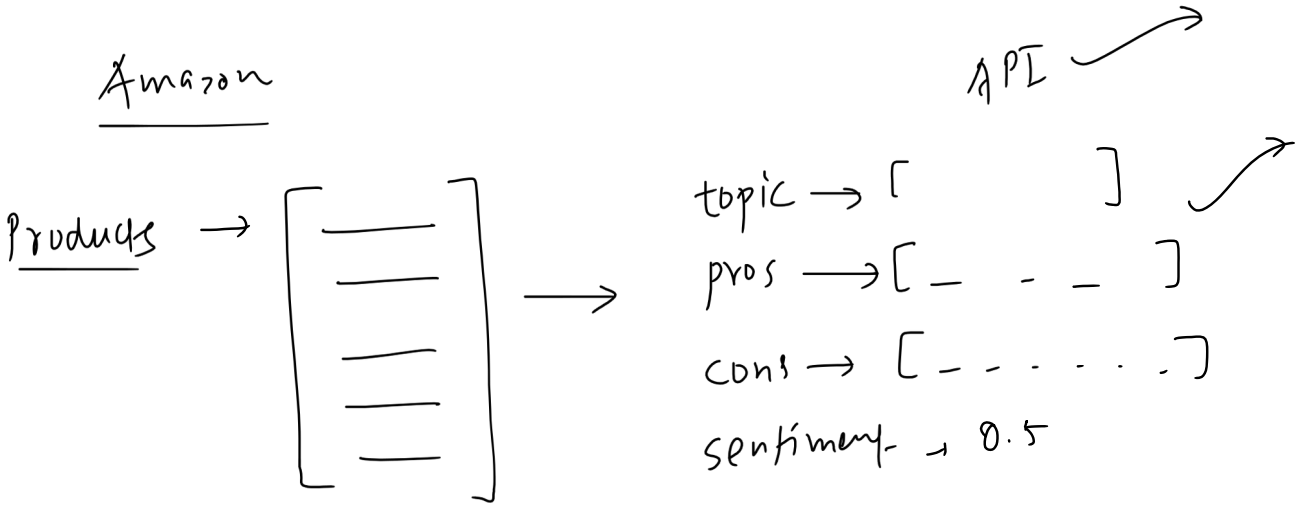
Source -https://www.linkedin.com/pulse/structured-outputs-from-llms-langchain-output-parsers-vijay-chaudhary-wgjqc

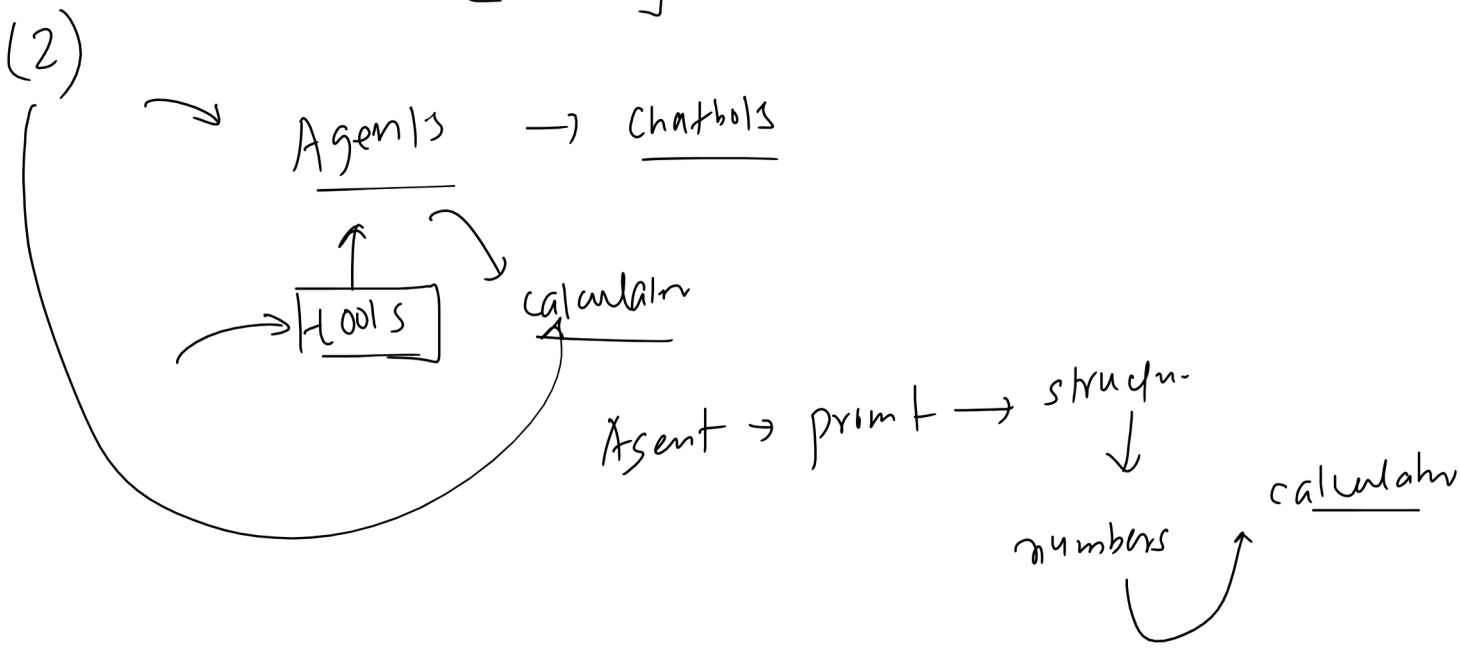
Why do we need Structured Output

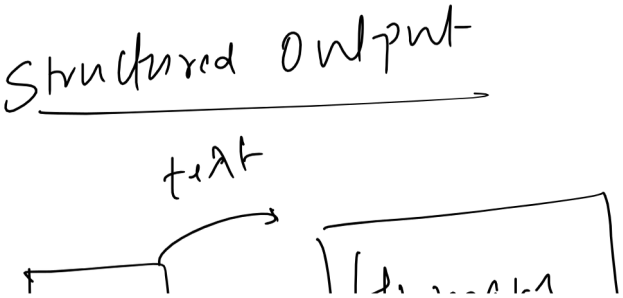
28 February 2025 00:13

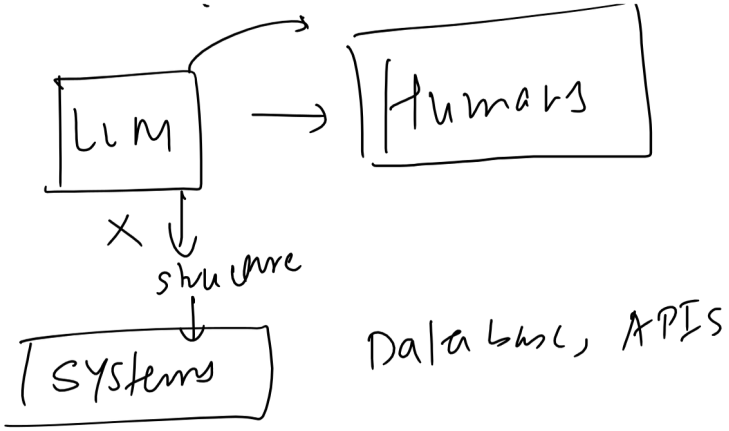
Data Extraction
API building
Agents





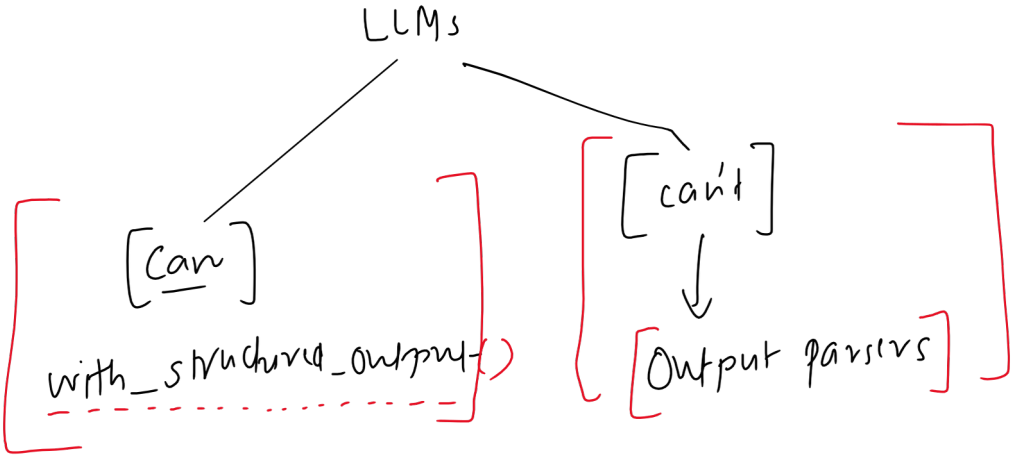





LLM
Humans
structure
Systems
Databases, APIs

Ways to get Structured Output

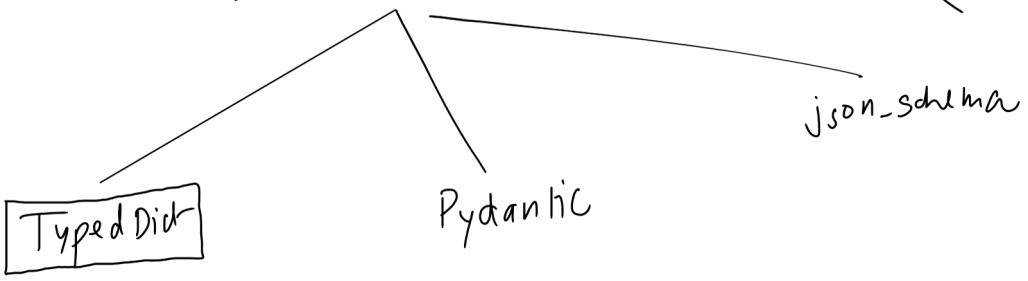
28 February 2025 00:14



with_structured_output → data-format

28 February 2025 00:15

(model_invoke)



You areanAIassistantthatextractsstructured insightsfromtext. Givenaproductreview, extract: -Summary: Abriefoverviewofthemain points. -Sentiment: Overalltoneofthereview (positive, neutral, negative). Returntheresponse in JSON format.

TypedDict

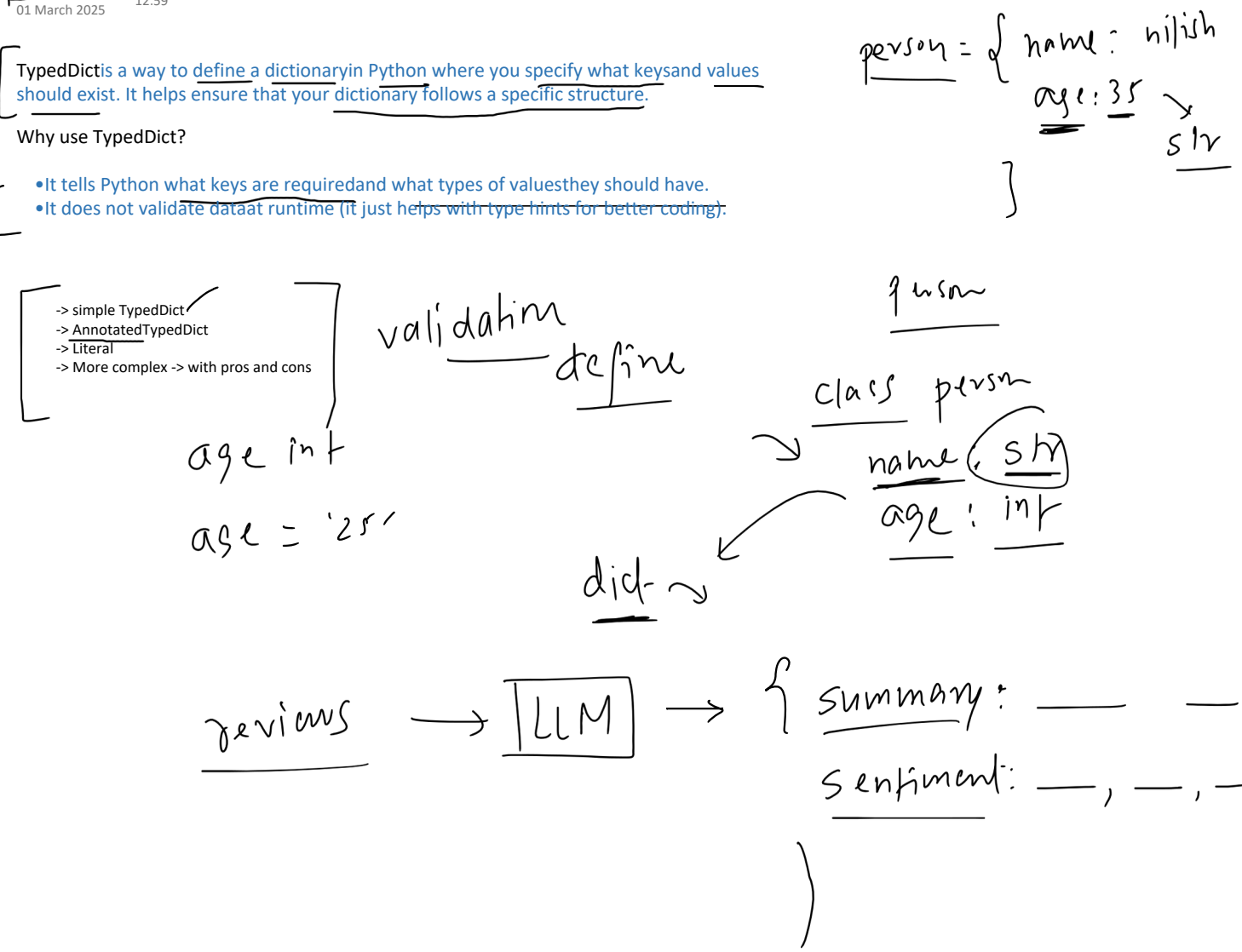
01 March 2025 12:59

TypedDictis a way to define a dictionaryin Python where you specify what keysand values should exist. It helps ensure that your dictionary follows a specific structure.

Why use TypedDict?

- It tells Python what keys are requiredand what types of valuesthey should have.
- It does not validate dataat runtime (it just helps with type hints for better coding).

-> simple TypedDict
-> AnnotatedTypedDict
-> Literal
-> More complex -> with pros and cons




```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
from typing import TypedDict, Annotated, Optional, Literal

load_dotenv()

model = ChatOpenAI()

# schema
class Review(TypedDict):

    key_themes: Annotated[list[str], "Write down all the key themes discussed in the review in a list"]
    summary: Annotated[str, "A brief summary of the review"]
    sentiment: Annotated[Literal["pos", "neg"], "Return sentiment of the review either negative, positive or neutral"]
    pros: Annotated[Optional[list[str]], "Write down all the pros inside a list"]
    cons: Annotated[Optional[list[str]], "Write down all the cons inside a list"]
    name: Annotated[Optional[str], "Write the name of the reviewer"]


structured_model = model.with_structured_output(Review)

result = structured_model.invoke("""I recently upgraded to the Samsung Galaxy S24 Ultra, and I must say, it's an absolute powerhouse! The

The S-Pen integration is a great touch for note-taking and quick sketches, though I don't use it often. What really blew me away is the 2

However, the weight and size make it a bit uncomfortable for one-handed use. Also, Samsung's One UI still comes with bloatware—why do I ne

Pros:
Insanely powerful processor (great for gaming and productivity)
Stunning 200MP camera with incredible zoom capabilities
Long battery life with fast charging
S-Pen support is unique and useful

Review by Nitish Singh
""")

print(result['name'])
```

Pydantic

01 March 2025 12:59

Pydantic is a data validationand data parsinglibrary for Python. It ensures that the data you work with is correct, structured, and type-safe.

Basic example
Default values
Optional fields
Coerce
Builtin validation
Field Function -> default values, constraints, description, regex expressions
Returns pydantic object -> convert to json/dict

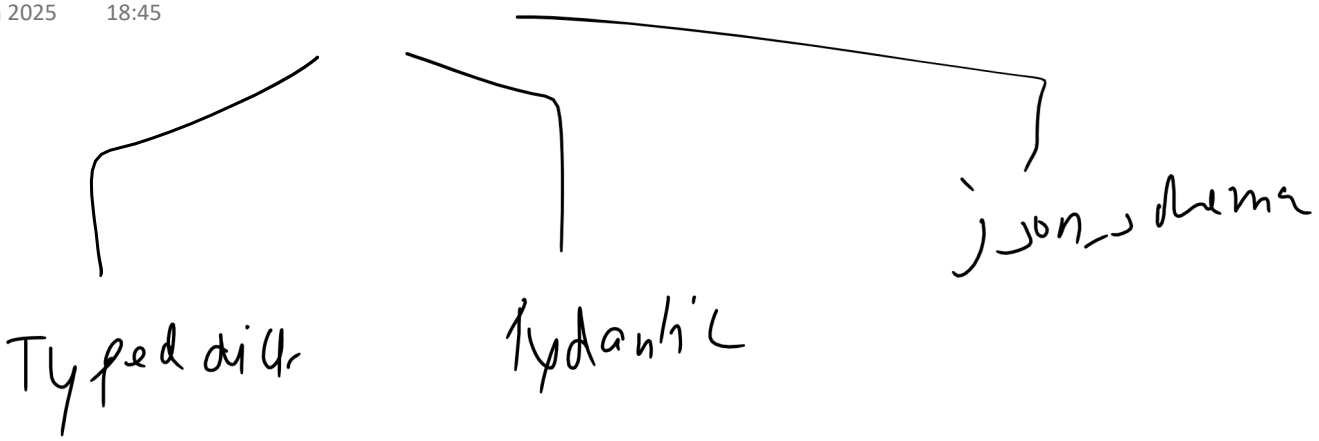
```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
from typing import TypedDict, Annotated, Optional, Literal
from pydantic import BaseModel, Field

load_dotenv()

model = ChatOpenAI()

# schema
class Review(BaseModel):

    key_themes: list[str] = Field(description="Write                    e review in a list")
    summary: str = Field(description="A brief summar
    sentiment: Literal["pos", "neg"] = Field(descrip                    her negative, positive or neutral")
    pros: Optional[list[str]] = Field(default=None, description="Write down all the pros inside a list")
    cons: Optional[list[str]] = Field(default=None, description="Write down all the cons inside a list")
    name: Optional[str] = Field(default=None, description="Write the name of the reviewer")


structured_model = model.with_structured_output(Review)

result = structured_model.invoke("""I recently upgraded to the Samsung Galaxy S24 Ultra, and I must say, it's an absolute powerhouse! The

The S-Pen integration is a great touch for note-taking and quick sketches, though I don't use it often. What really blew me away is the 2

However, the weight and size make it a bit uncomfortable for one-handed use. Also, Samsung's One UI still comes with bloatware—why do I n

Pros:
Insanely powerful processor (great for gaming and productivity)
Stunning 200MP camera with incredible zoom capabilities
Long battery life with fast charging
S-Pen support is unique and useful

Review by Nitish Singh
""")

print(result)
```

(parameter) description: str | None

description: Human-readable description.

Json

01 March 2025 18:45




```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
from typing import TypedDict, Annotated, Optional, Literal
from pydantic import BaseModel, Field

load_dotenv()

model = ChatOpenAI()

# schema
json_schema = {
  "title": "Review",
  "type": "object",
  "properties": {...
  "required": ["key_themes", "summary", "sentiment"]
}


structured_model = model.with_structured_output(json_schema)

result = structured_model.invoke("""I recently upgraded to the Samsung Galaxy S24 Ultra, and I must say, it's an absolute powerhouse! The

The S-Pen integration is a great touch for note-taking and quick sketches, though I don't use it often. What really blew me away is the 20

However, the weight and size make it a bit uncomfortable for one-handed use. Also, Samsung's One UI still comes with bloatware—why do I ne

Pros:
Insanely powerful processor (great for gaming and productivity)
Stunning 200MP camera with incredible zoom capabilities
Long battery life with fast charging
S-Pen support is unique and useful

Review by Nitish Singh
""")

print(result)
```

When to use what?

01 March 2025 12:59

✅ Use `TypedDict` if:

- You **only need type hints** (basic structure enforcement).
- You **don't need validation** (e.g., checking numbers are positive).
- You **trust the LLM** to return correct data.

✅ Use `Pydantic` if:

- You need **data validation** (e.g., sentiment must be `"positive"`, `"neutral"`, or `"negative"`).
- You need **default values** if the LLM **misses fields**.
- You want **automatic type conversion** (e.g., `"100"` → `100`).

✅ Use `JSON Schema` if:

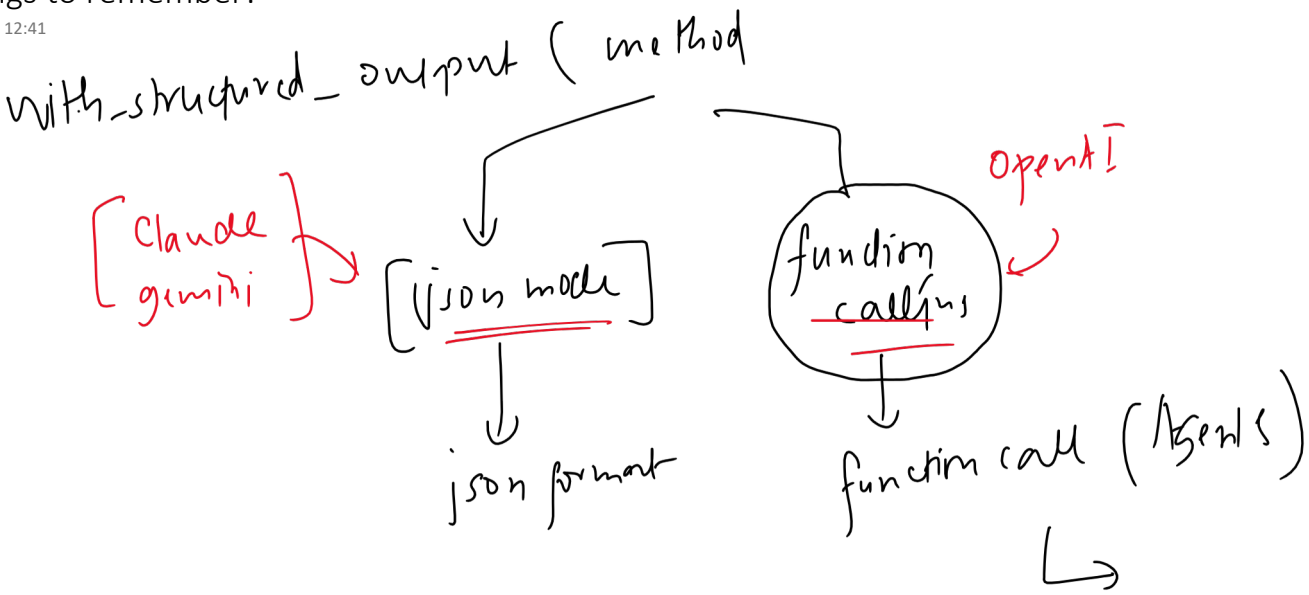
- You **don't want to ~~import extra Python libraries~~** (Pydantic).
- You need **validation but don't need Python objects**.
- You want to define structure **in a standard JSON format**.

🚀 When to Use What?

Feature	TypedDict ✅	Pydantic 🚀	JSON Schema 🌍
Basic structure	✅	✅	✅
Type enforcement	✅	✅	✅
Data validation	❌	✅	✅
Default values	❌	✅	❌
Automatic conversion	❌	✅	❌
Cross-language compatibility	❌	❌	✅

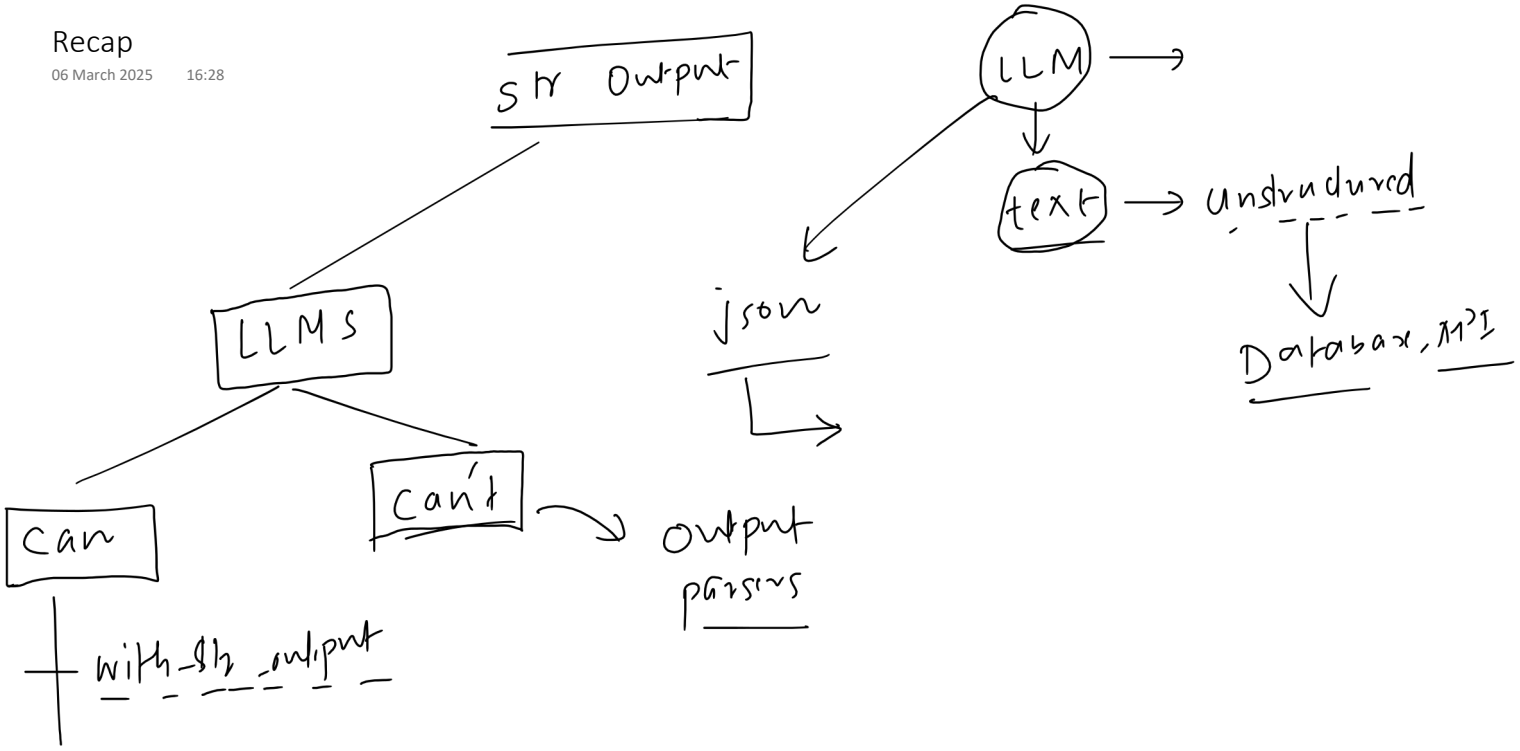
A few things to remember!

03 March 2025 12:41



Recap

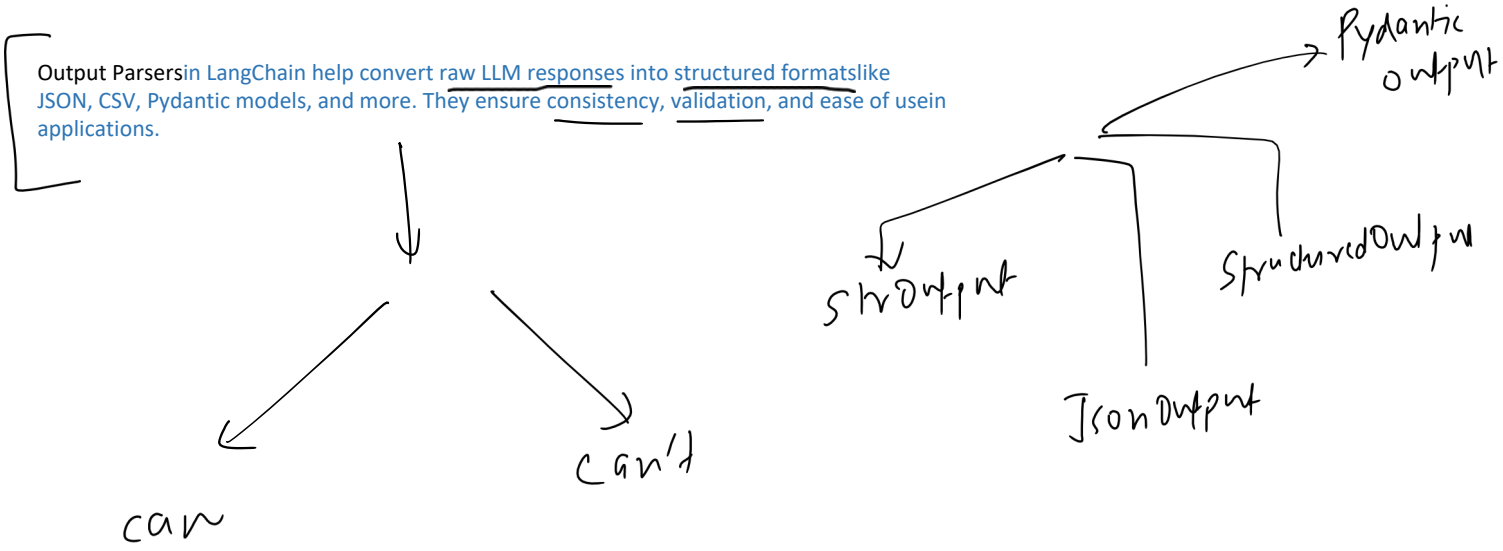
06 March 2025 16:28



Output Parsers

06 March 2025 16:29

Output Parsersin LangChain help convert raw LLM responses into structured formatslike JSON, CSV, Pydantic models, and more. They ensure consistency, validation, and ease of usein applications.



StrOutputParser

06 March 2025 16:29

The StrOutputParseris the simplest output parser in LangChain. It is used to parse the output of a Language Model (LLM) and return it as a plain string.

result.content

```
content='A black hole is a region in space where gravity is so strong that nothing, not e
ven light, can escape its pull. It is formed when a massive star collapses upon itself.'
additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens
': 37, 'prompt_tokens': 15, 'total_tokens': 52, 'completion_tokens_details': {'accepted_p
rediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_toke
ns': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_name':
 'gpt-3.5-turbo-0125', 'system_fingerprint': None, 'finish_reason': 'stop', 'logprobs': N
one} id='run-a7b90203-58f8-47c5-a01b-01184b6aec14-0' usage_metadata={'input_tokens': 15,
'output_tokens': 37, 'total_tokens': 52, 'input_token_details': {'audio': 0, 'cache_read'
: 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}}
```

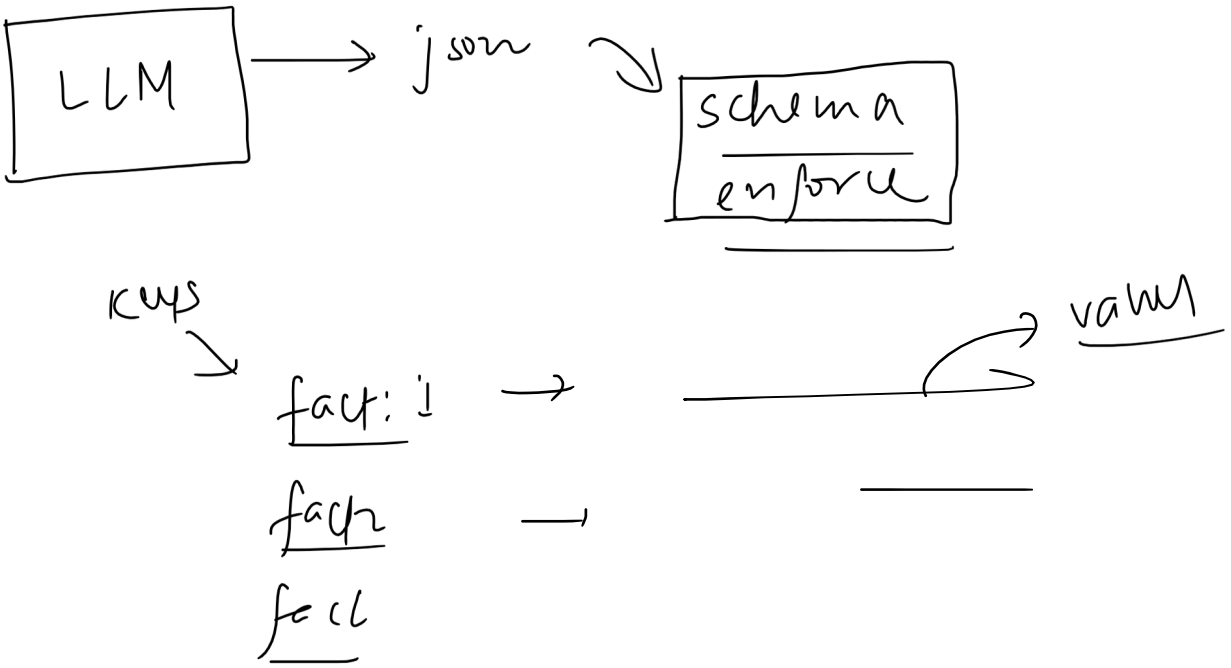
topic → LLM → [Detailed report] → LLM → 5 line summary

```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()


model = ChatOpenAI()

# 1st prompt -> detailed report
template1 = PromptTemplate(
    template='Write a detailed report on {topic}',
    input_variables=['topic']
)

# 2nd prompt -> summary
template2 = PromptTemplate(
    template='Write a 5 line summary on the following text. /n {text}',
    input_variables=['text']
)

parser = StrOutputParser()

chain = template1 | model | parser | template2 | model | parser

result = chain.invoke({'topic':'black hole'})

print(result)
```

(variable) parser: Any

JSONOutputParser

06 March 2025 16:29



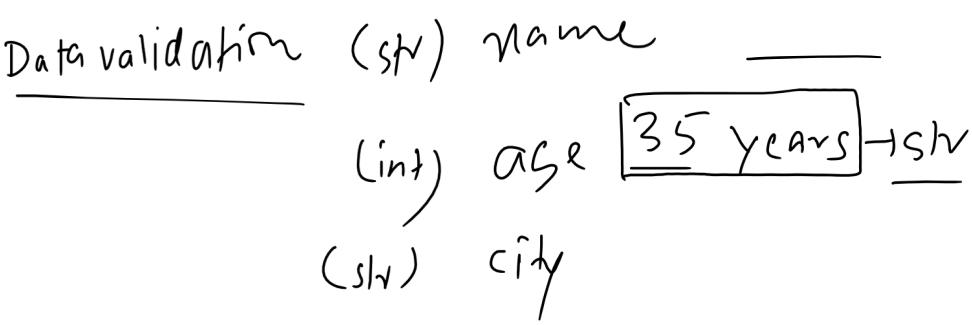

```python
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import JsonOutputParser

load_dotenv()

# Define the model
llm = HuggingFaceEndpoint(
    repo_id="google/gemma-2-2b-it",
    task="text-generation"
)

model = ChatHuggingFace(llm=llm)

parser = JsonOutputParser()

template = PromptTemplate(
    template='Give me 5 facts about {topic} \n {format_instruction}',
    input_variables=['topic'],
    partial_variables={'format_instruction': parser.get_format_instructions()}
)

chain = template | model | parser

result = chain.invoke({'topic':'black hole'})

print(result)
```

StructuredOutputParser

06 March 2025 16:29

StructuredOutputParseris an output parserin LangChain that helps extract structured JSON datafrom LLM responses based on predefined field schemas.

It works by defining a list of fields (ResponseSchema)that the model should return, ensuring the output follows a structured format.




```python
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
from langchain.output_parsers import StructuredOutputParser, ResponseSchema

load_dotenv()

# Define the model
llm = HuggingFaceEndpoint(
    repo_id="google/gemma-2-2b-it",
    task="text-generation"
)

model = ChatHuggingFace(llm=llm)

schema = [
    ResponseSchema(name='fact_1', description='Fact 1 about the topic'),
    ResponseSchema(name='fact_2', description='Fact 2 about the topic'),
    ResponseSchema(name='fact_3', description='Fact 3 about the topic'),
]

parser = StructuredOutputParser.from_response_schemas(schema)

template = PromptTemplate(
    template='Give 3 fact about {topic} \n {format_instruction}',
    input_variables=['topic'],
    partial_variables={'format_instruction':parser.get_format_instructions()}
)

chain = template | model | parser

result = chain.invoke({'topic':'black hole'})

print(result)
```

PydanticOutputParser

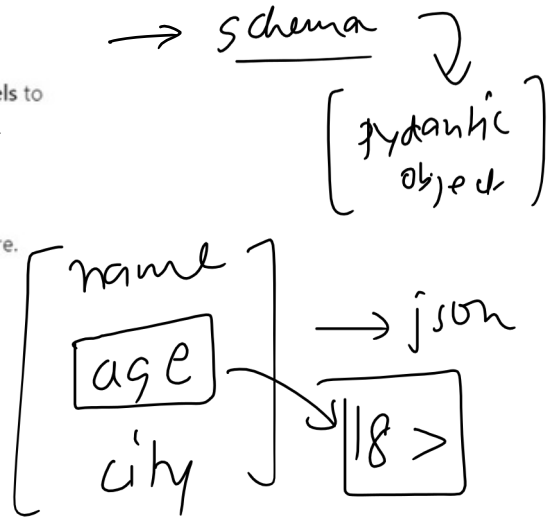
06 March 2025 16:30

🔹 What is `PydanticOutputParser` in LangChain?

`PydanticOutputParser` is a **structured output parser** in LangChain that uses **Pydantic models** to enforce schema validation when processing LLM responses.

🚀 Why Use `PydanticOutputParser`?

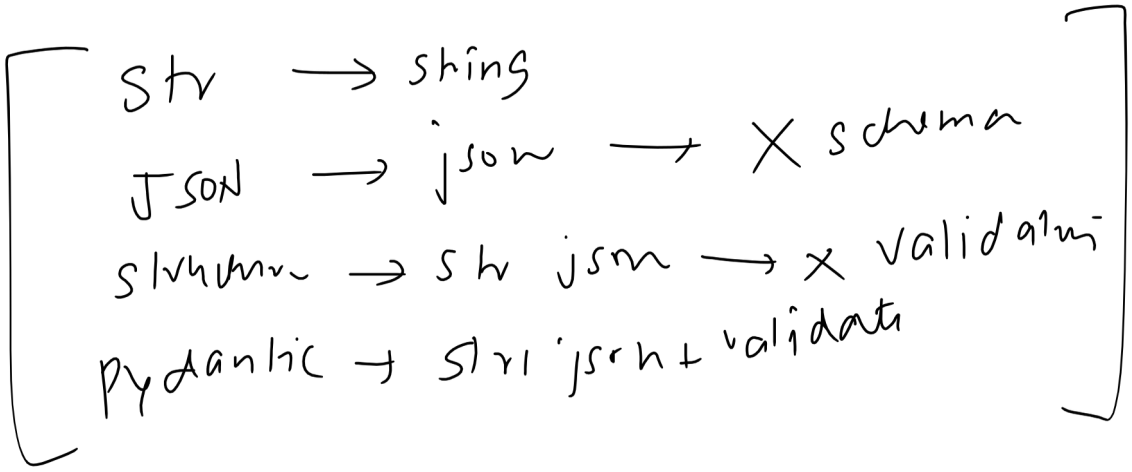
- ✅ **Strict Schema Enforcement** → Ensures that LLM responses follow a well-defined structure.
- ✅ **Type Safety** → Automatically converts LLM outputs into Python objects.
- ✅ **Easy Validation** → Uses Pydantic's built-in validation to catch incorrect or missing data.
- ✅ **Seamless Integration** → Works well with other LangChain components.

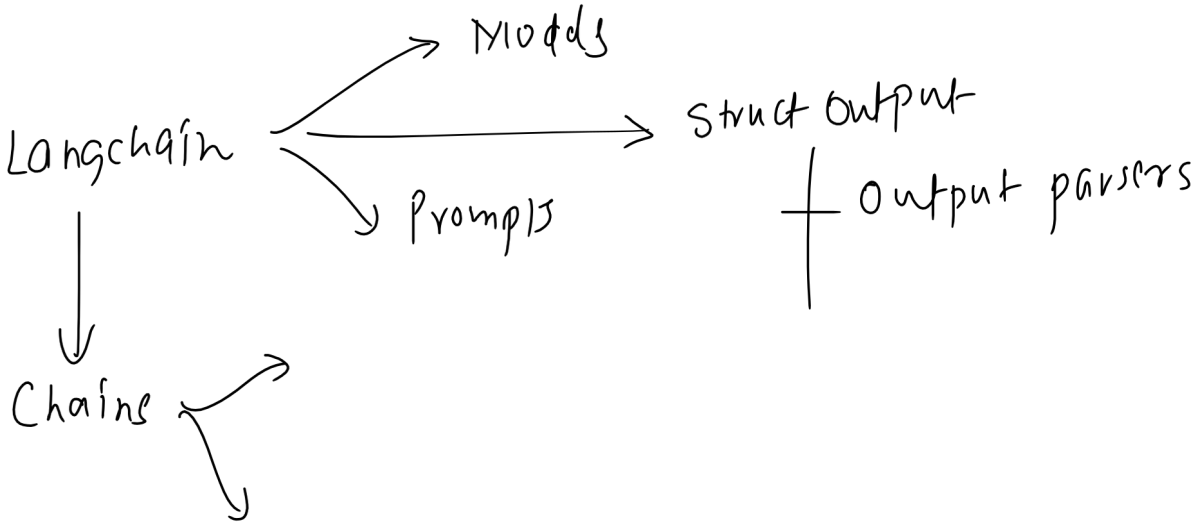



```python
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field

load_dotenv()

# Define the model
llm = HuggingFaceEndpoint(
    repo_id="google/gemma-2-2b-it",
    task="text-generation"
)

model = ChatHuggingFace(llm=llm)

class Person(BaseModel):

    name: str = Field(description='Name of the person')
    age: int = Field(gt=18, description='Age of the person')
    city: str = Field(description='Name of the city the person belongs to')

parser = PydanticOutputParser(pydantic_object=Person)

template = PromptTemplate(
    template='Generate the name, age and city of a fictional {place} person \n {format_instruction}',
    input_variables=['place'],
    partial_variables={'format_instruction':parser.get_format_instructions()}
)

chain = template | model | parser

final_result = chain.invoke({'place':'sri lankan'})

print(final_result)
```

Recap

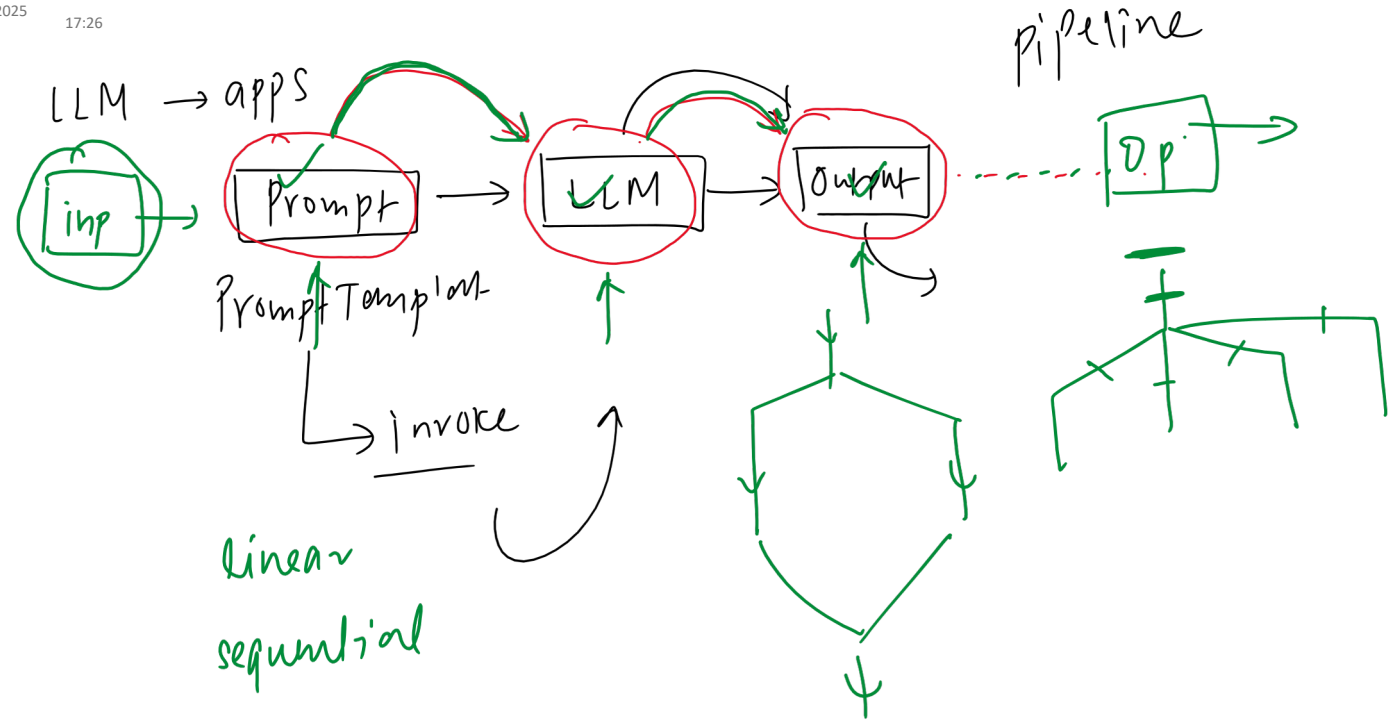
11 March 2025 17:26

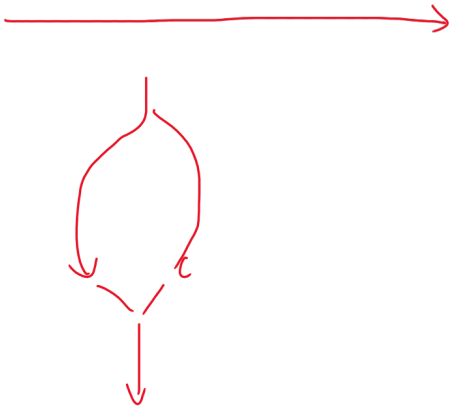




What & Why

11 March 2025 17:26





simple chains

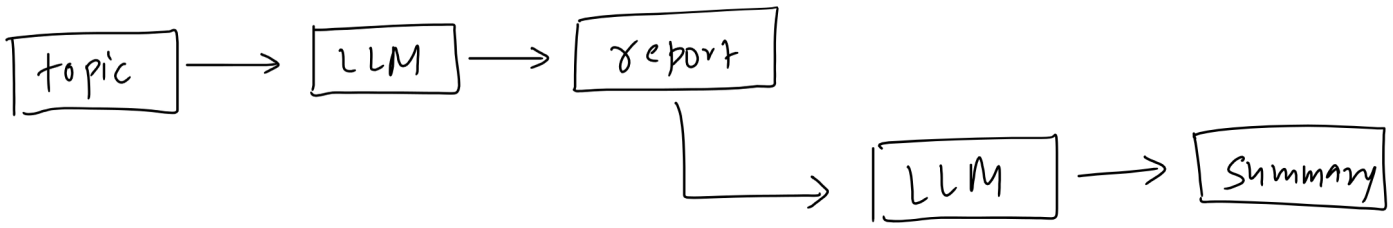
Simple Chain

11 March 2025 17:26

```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

prompt = PromptTemplate(
    template='Generate 5 interesting facts about {topic}',
    input_variables=['topic']
)

model = ChatOpenAI()

parser = StrOutputParser()

chain = prompt | model | parser

result = chain.invoke({'topic':'cricket'})

print(result)

chain.get_graph().print_ascii()
```

Sequential Chain

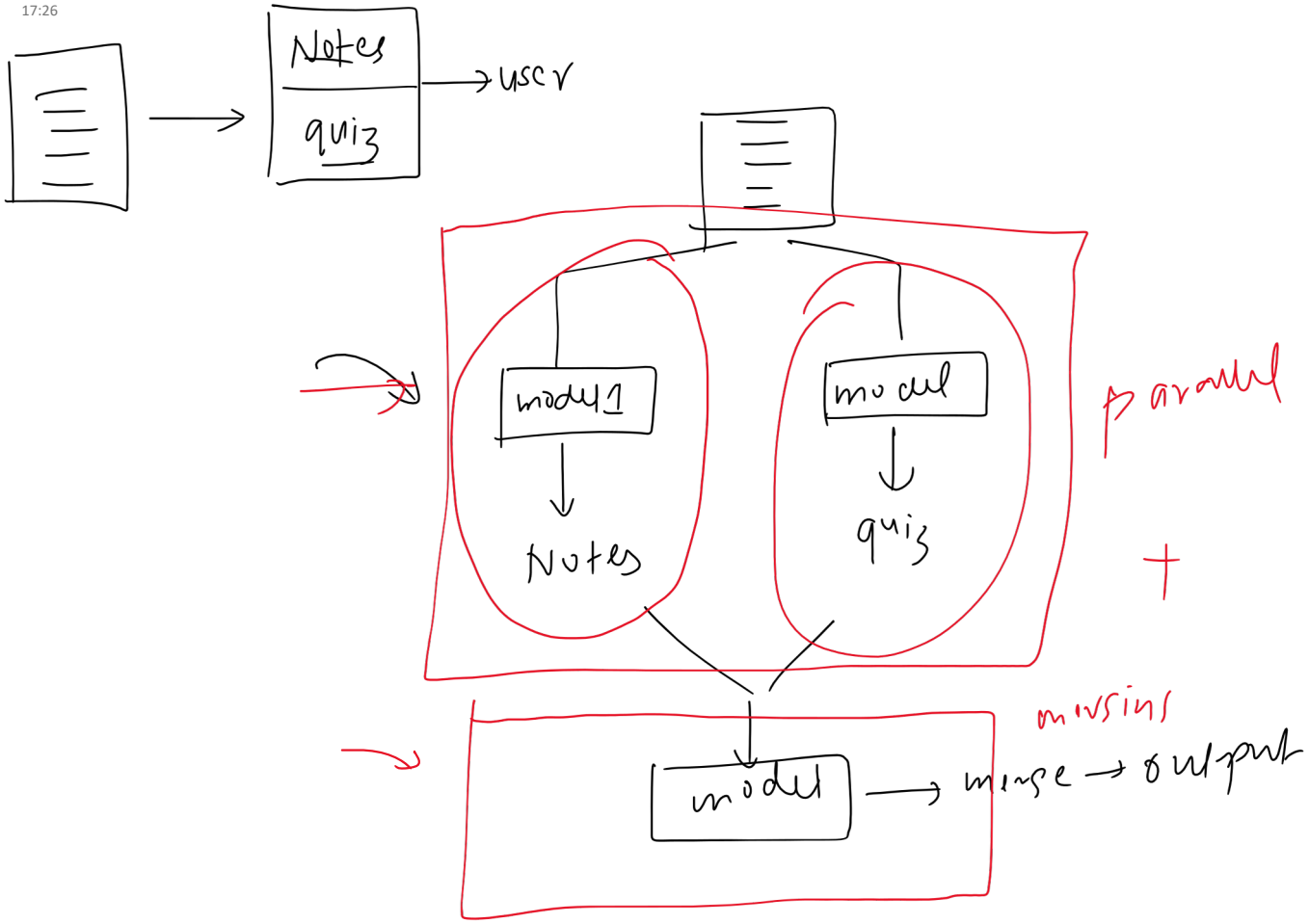
11 March 2025 17:26




```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

prompt1 = PromptTemplate(
    template='Generate a detailed report on {topic}',
    input_variables=['topic']
)

prompt2 = PromptTemplate(
    template='Generate a 5 pointer summary from the following text \n {text}',
    input_variables=['text']
)

model = ChatOpenAI()

parser = StrOutputParser()

chain = prompt1 | model | parser | prompt2 | model | parser

result = chain.invoke({'topic': 'Unemployment in India'})

print(result)

chain.get_graph().print_ascii()
```

Parallel Chain

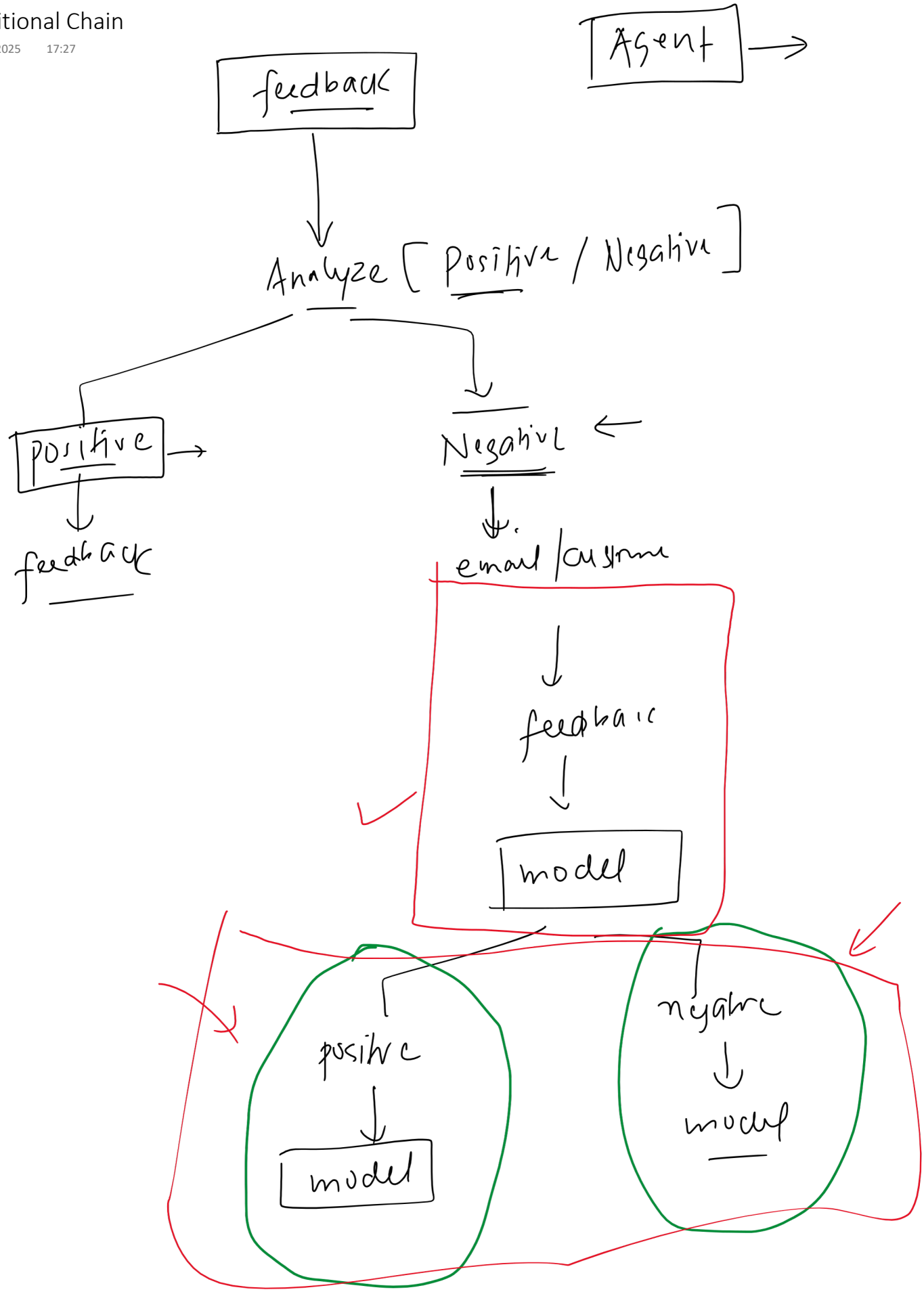
11 March 2025 17:26



```python
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain.schema.runnable import RunnableParallel

load_dotenv()

model1 = ChatOpenAI()

model2 = ChatAnthropic(model_name='claude-3-7-sonnet-20250219')

prompt1 = PromptTemplate(
    template='Generate short and simple notes from the following text \n {text}',
    input_variables=['text']
)

prompt2 = PromptTemplate(
    template='Generate 5 short question answers from the following text \n {text}',
    input_variables=['text']
)

prompt3 = PromptTemplate(
    template='Merge the provided notes and quiz into a single document \n notes -> {notes} and quiz -> {quiz}',
    input_variables=['notes', 'quiz']
)

parser = StrOutputParser()

parallel_chain = RunnableParallel({
    'notes': prompt1 | model1 | parser,
    'quiz': prompt2 | model2 | parser
})

merge_chain = prompt3 | model1 | parser

chain = parallel_chain | merge_chain

text = """...

result = chain.invoke({'text':text})

print(result)

chain.get_graph().print_ascii()
```

Conditional Chain

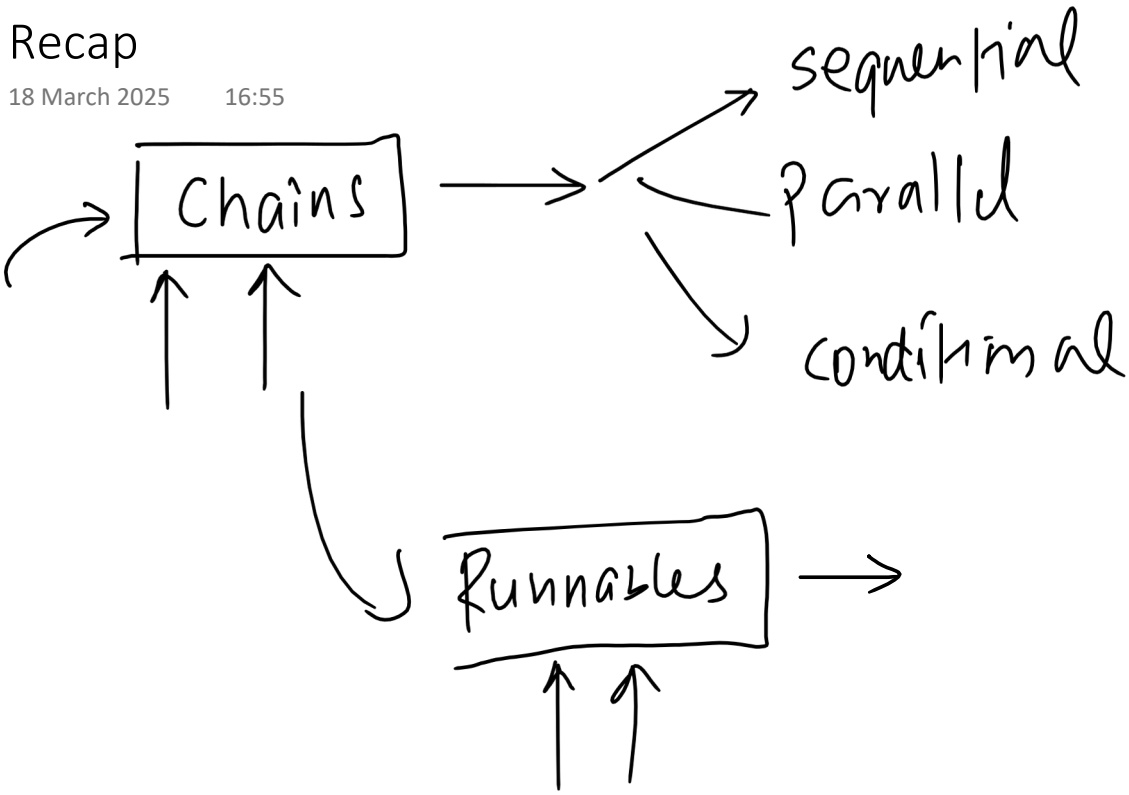
11 March 2025 17:27



```python
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain.schema.runnable import RunnableParallel, RunnableBranch, RunnableLambda
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from typing import Literal

load_dotenv()

model = ChatOpenAI()

parser = StrOutputParser()

class Feedback(BaseModel):

    sentiment: Literal['positive', 'negative'] = Field(description='Give the sentiment of the feedback')

parser2 = PydanticOutputParser(pydantic_object=Feedback)

prompt1 = PromptTemplate(
    template='Classify the sentiment of the following feedback text into postive or negative \n {feedback} \n {format_instruction}',
    input_variables=['feedback'],
    partial_variables={'format_instruction':parser2.get_format_instructions()}
)

classifier_chain = prompt1 | model | parser2

prompt2 = PromptTemplate(
    template='Write an appropriate response to this positive feedback \n {feedback}',
    input_variables=['feedback']
)

prompt3 = PromptTemplate(
    template='Write an appropriate response to this negative feedback \n {feedback}',
    input_variables=['feedback']
)

branch_chain = RunnableBranch(
    (lambda x:x.sentiment == 'positive', prompt2 | model | parser),
    (lambda x:x.sentiment == 'negative', prompt3 | model | parser),
    RunnableLambda(lambda x: "could not find sentiment")
)

chain = classifier_chain | branch_chain

print(chain.invoke({'feedback': 'This is a beautiful phone'}))
```

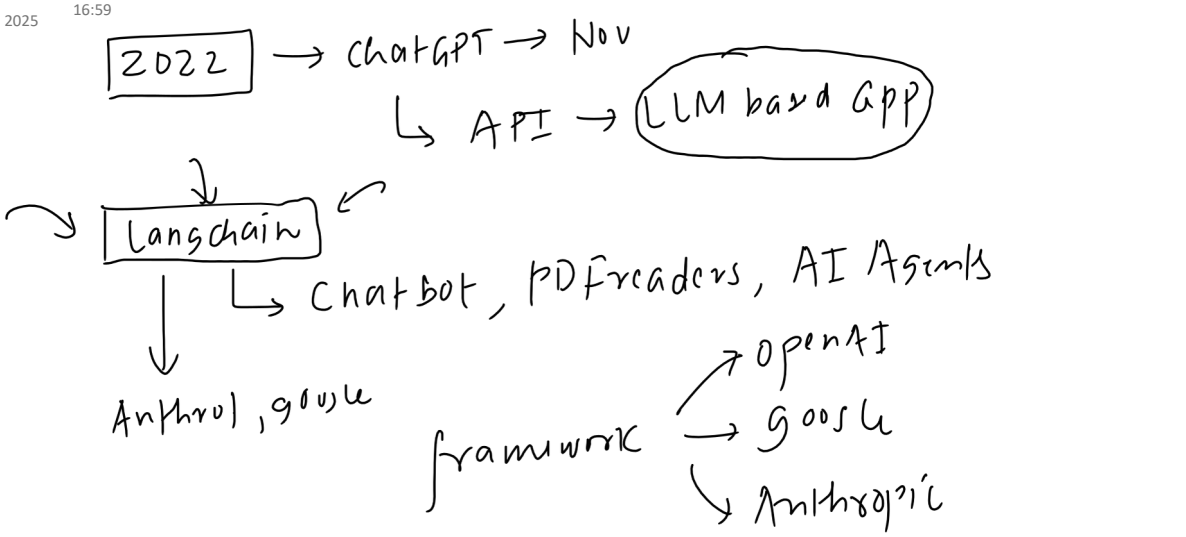
Recap

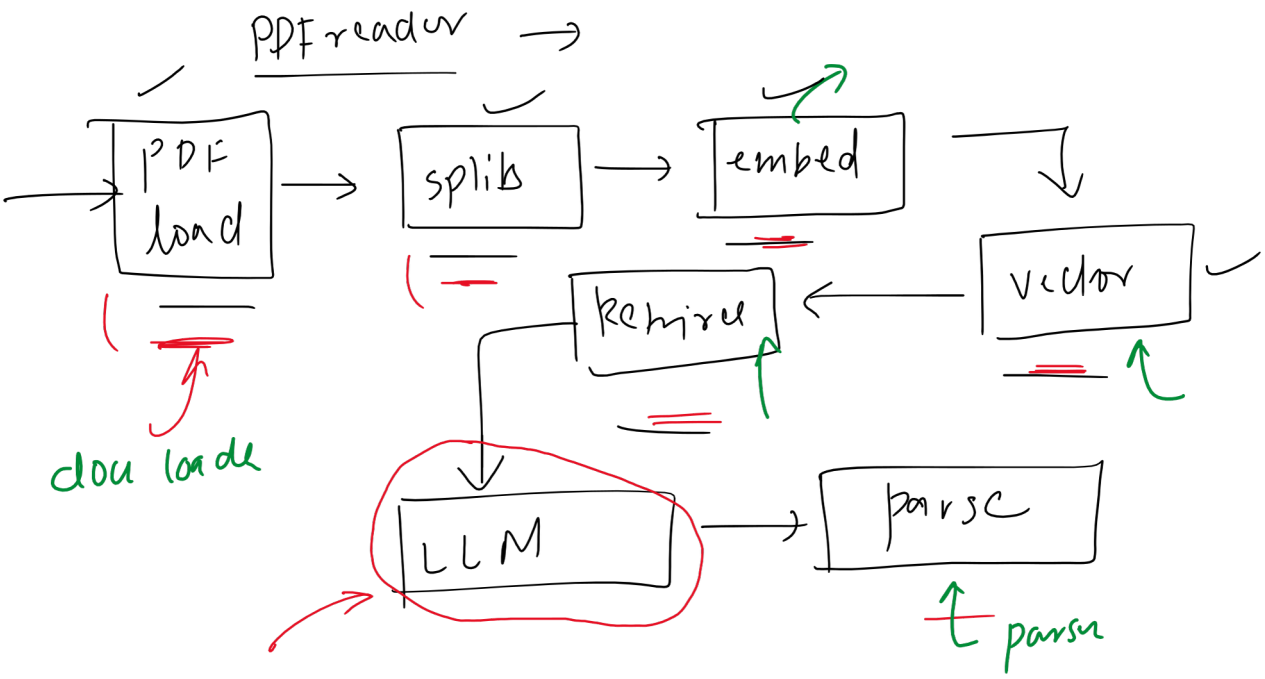
18 March 2025 16:55

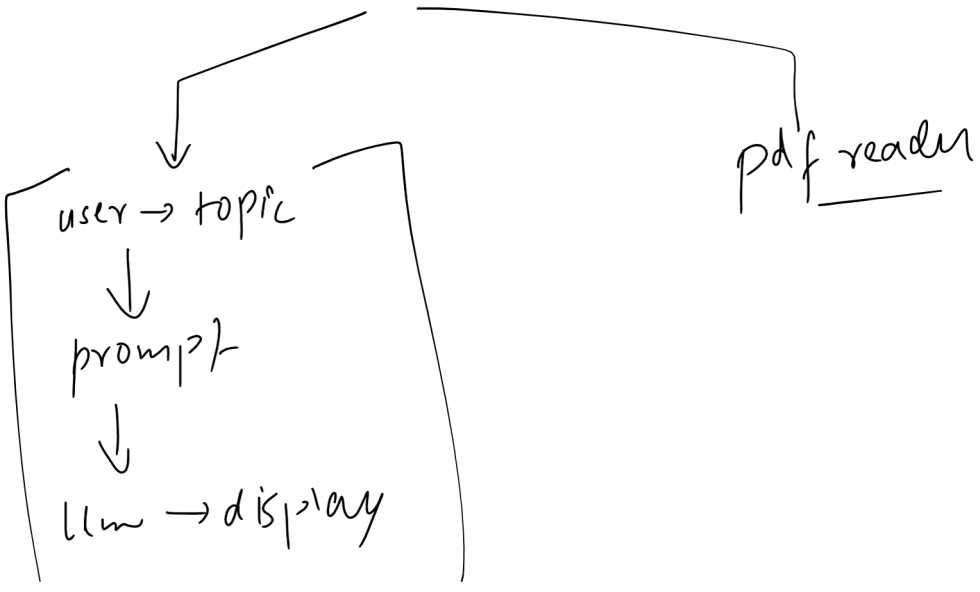


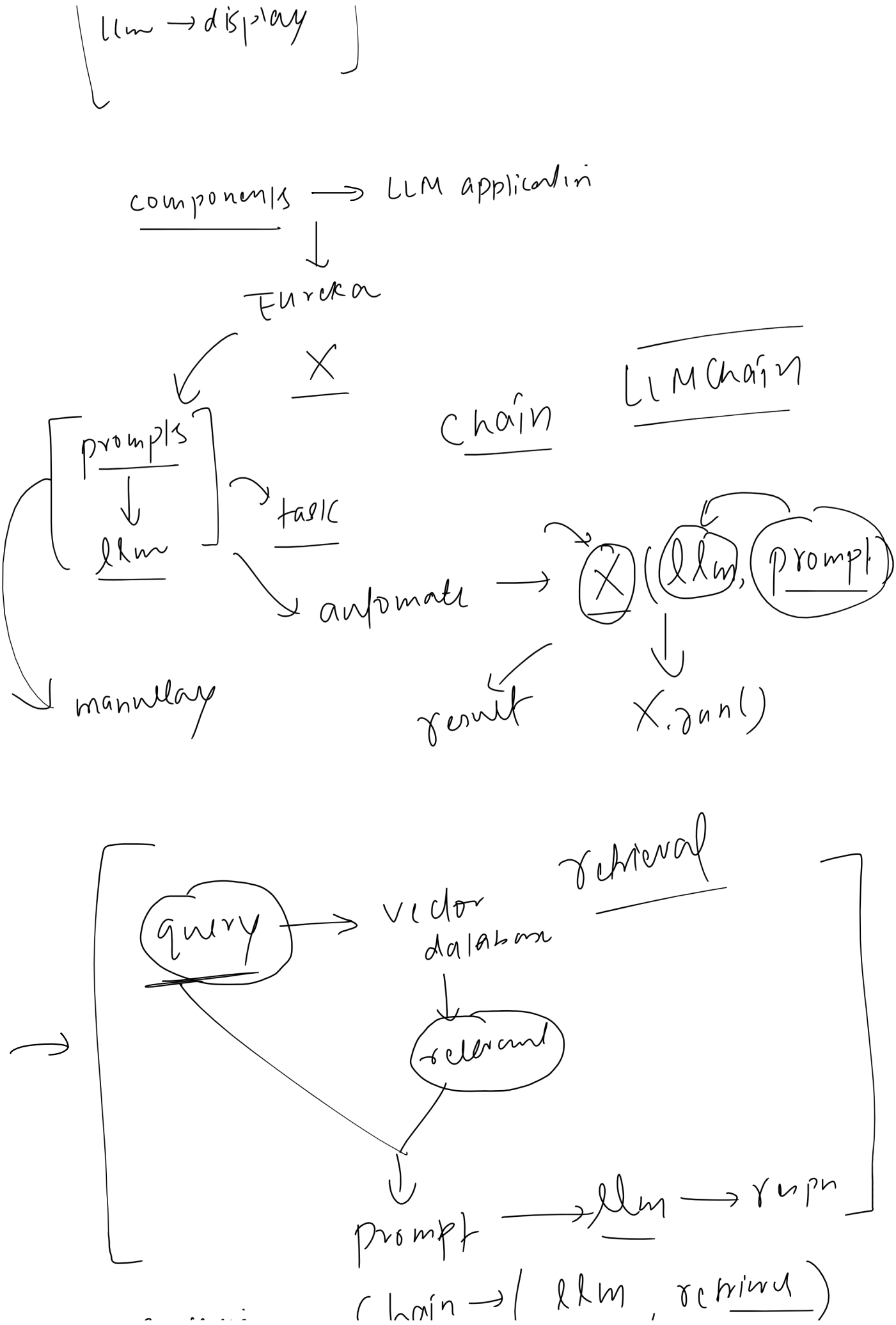
The Why

18 March 2025 16:59








llm → display
components → LLM application
Eureka
X
LLMChain
Chain
prompts
llm
task
automate
manually
X(llm, prompt)
result
X.run()
retrieval
query
vector database
relevant
prompt
llm → resp
Chain → llm, retrieval

RetrieveQAChain Chain → (llm , retriver)

LLMChain

LLMchain ↔ LLMChain

Chain Name	Description
LLMChain	Basic chain that calls an LLM with a prompt template.
SequentialChain	Chains multiple LLM calls in a specific sequence.
SimpleSequentialChain	A simplified version of SequentialChain for easier use.
ConversationalRetrievalChain	Handles conversational Q&A with memory and retrieval.
RetrievalQA	Fetches relevant documents and uses an LLM for question-answering.
RouterChain	Directs user queries to different chains based on intent.
MultiPromptChain	Uses different prompts for different user intents dynamically.
HydeChain (Hypothetical Document Embeddings)	Generates hypothetical answers to improve document retrieval.
AgentExecutorChain	Orchestrates different tools and actions dynamically using an agent.
SQLDatabaseChain	Connects to SQL databases and answers natural language queries.

→ topic → joke → explanation

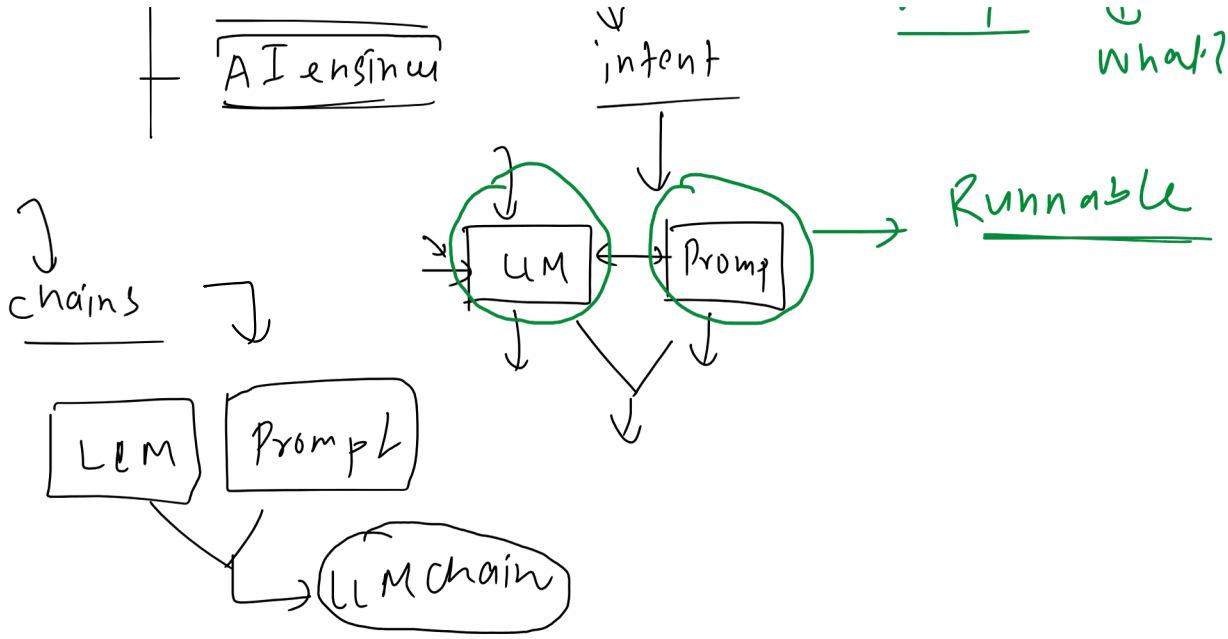
Chains
↓
use cases
↓
resue task
↓
chain

Problem

too many chains
- codebase
- AI ensinu

↓ intent

why? ↓ what?

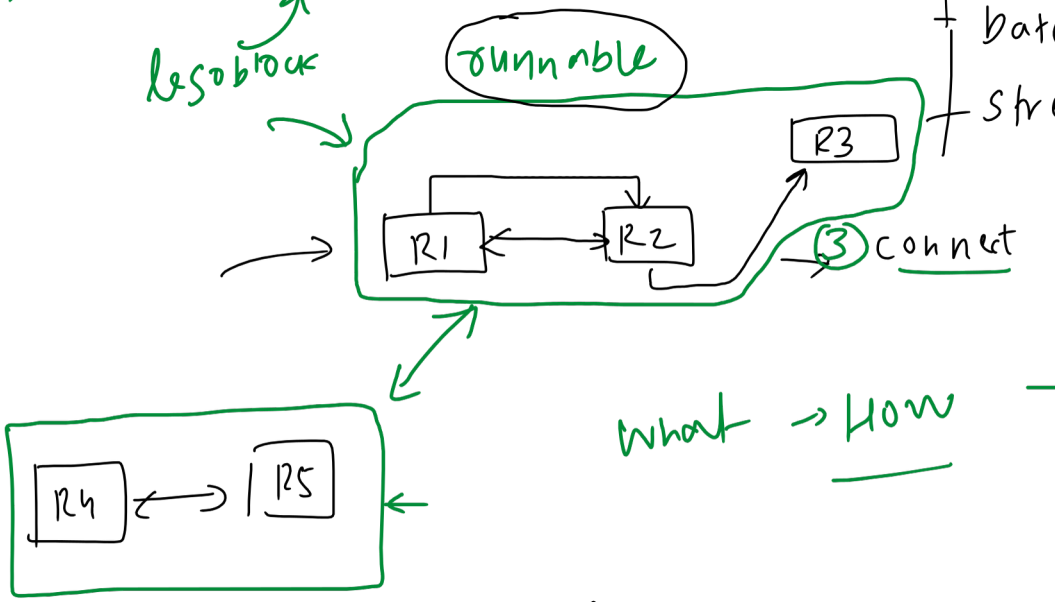

AI ensinu
intent
what?
Runnable
LLM
Prompt
chains
LLM
Prompt
LLMchain

The What

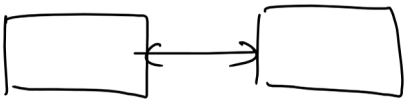
19 March 2025 00:12

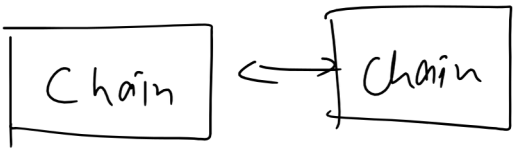
1. unit of work
   - input
   - process
   - output

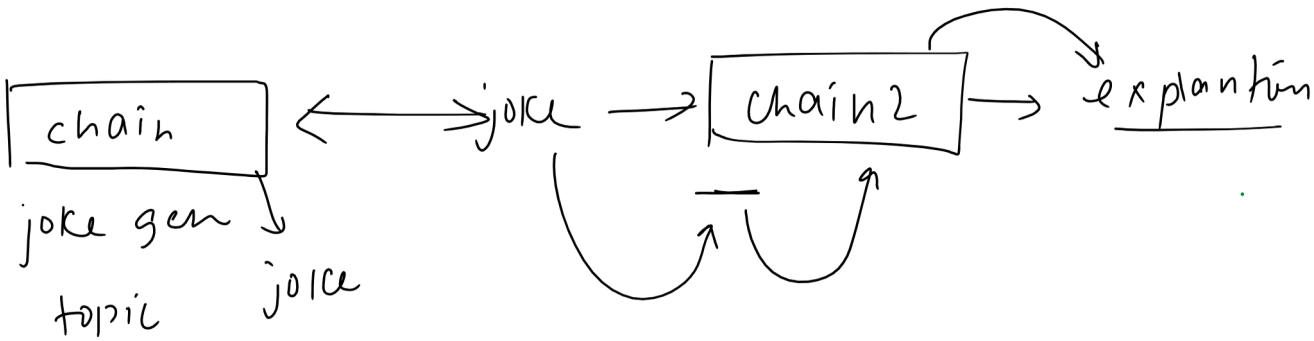
2. common interface
   - invoke()
   - batch()
   - stream()



chains

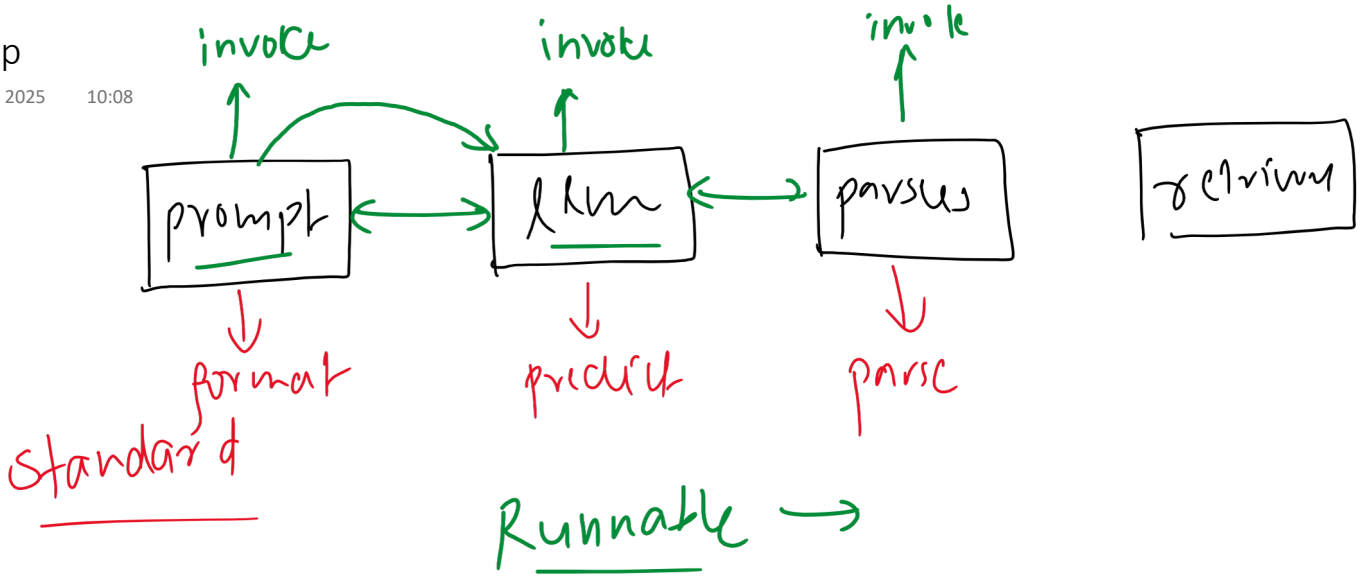






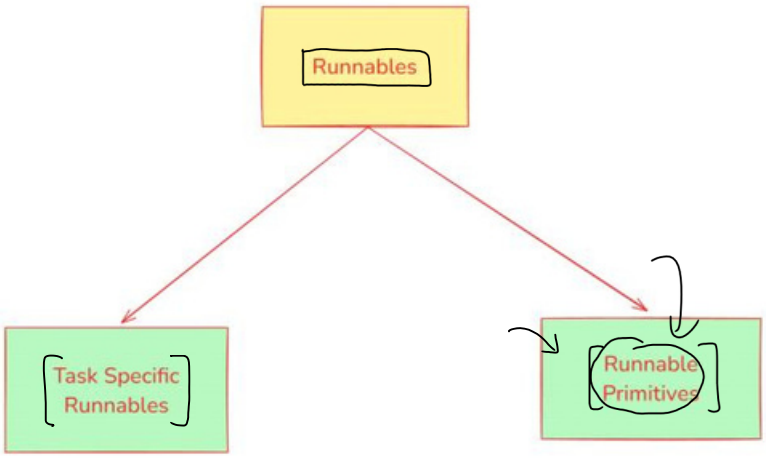
Recap

20 March 2025 10:08



Plan of Action

20 March 2025 11:02



✅ **Definition**: These are **core LangChain components** that have been converted into Runnables so they can be used in pipelines.

✅ **Purpose**: Perform **task-specific** operations like LLM calls, prompting, retrieval, etc.

✅ **Examples**:

- `ChatOpenAI` → Runs an LLM model.
- `PromptTemplate` → Formats prompts dynamically.
- `Retriever` → Retrieves relevant documents.

✅ **Definition**: These are **fundamental building blocks** for structuring execution logic in AI workflows.

✅ **Purpose**: They **help orchestrate execution** by defining how different Runnables interact (sequentially, in parallel, conditionally, etc.).

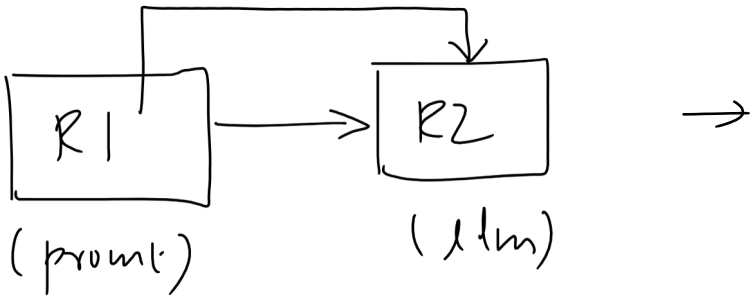
✅ **Examples**:

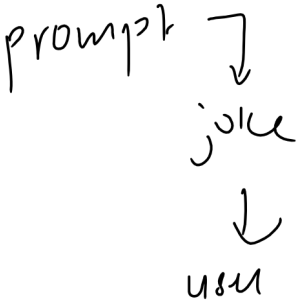
- `RunnableSequence` → Runs steps **in order** (`|` operator).
- `RunnableParallel` → Runs multiple steps **simultaneously**.
- `RunnableMap` → Maps the same input across multiple functions.
- `RunnableBranch` → Implements **conditional execution** (if-else logic).
- `RunnableLambda` → Wraps **custom Python functions** into Runnables.
- `RunnablePassthrough` → Just forwards input as output (acts as a placeholder).

1. RunnableSequence

20 March 2025 12:10

RunnableSequence is a sequential chain of runnablesin LangChain that executes each step one after another, passing the output of one step as the input to the next.
It is useful when you need to compose multiple runnables togetherin a structured workflow.



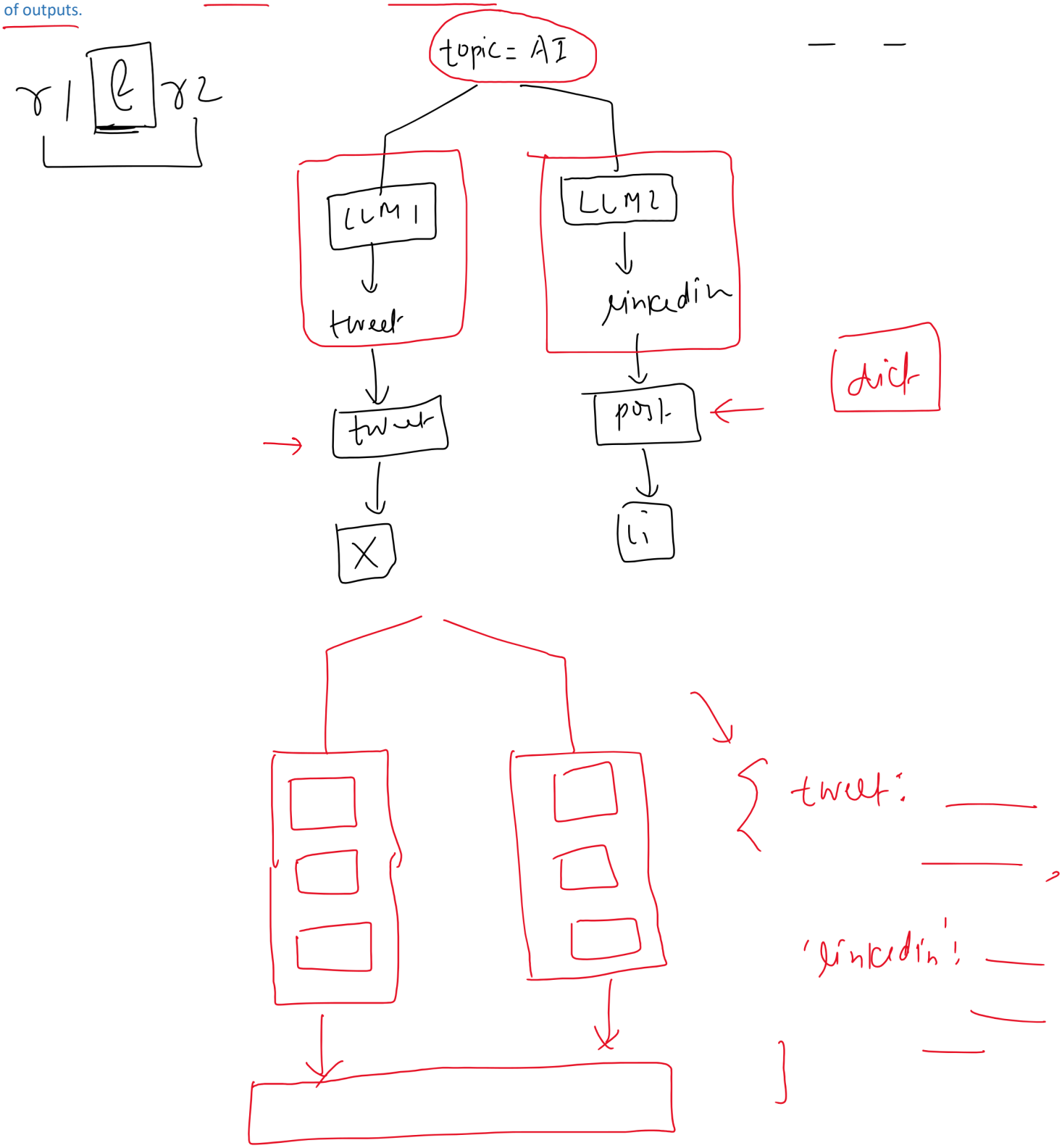


2. RunnableParallel

20 March 2025 18:33

RunnableParallelis a runnable primitivethat allows multiple runnables to execute in parallel.

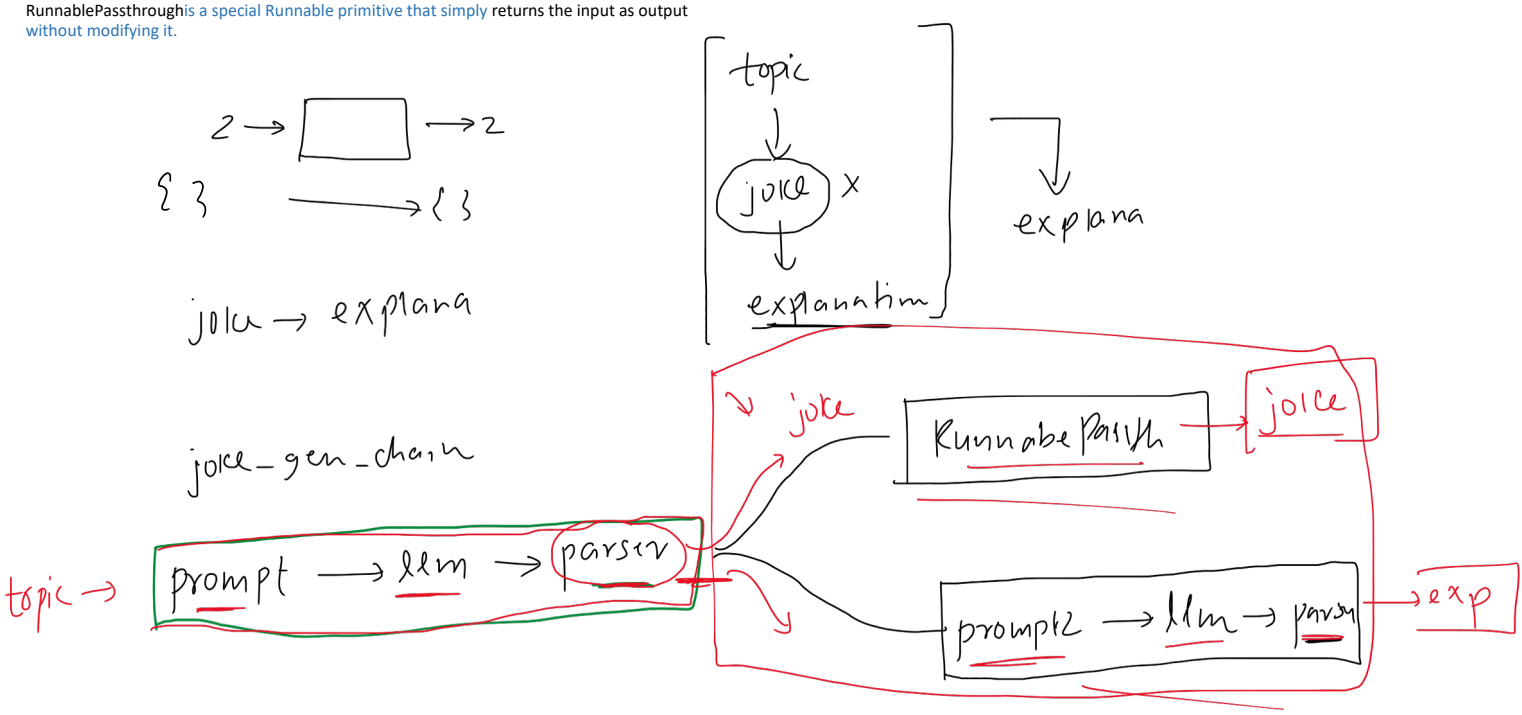
Each runnable receives the same inputand processes it independently, producing a dictionary of outputs.



3. RunnablePassthrough

20 March 2025 22:34

RunnablePassthroughis a special Runnable primitive that simply returns the input as output without modifying it.

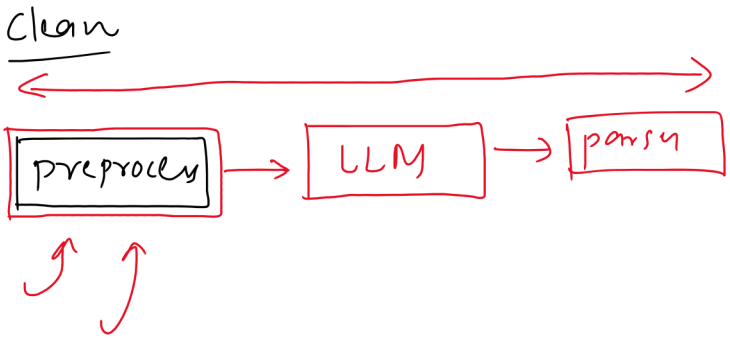


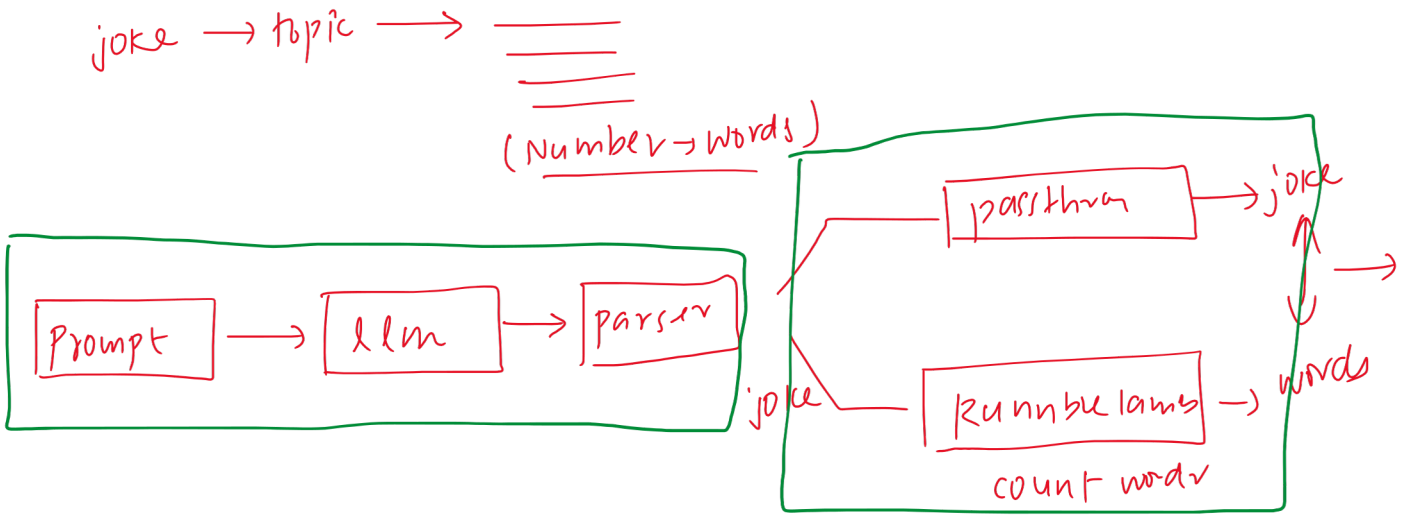
4. RunnableLambda

20 March 2025 23:18

RunnableLambdais a runnable primitive that allows you to apply custom Python functions within an AI pipeline.

It acts as a middlewarebetween different AI components, enabling preprocessing, transformation, API calls, filtering, and post-processingin a LangChain workflow.



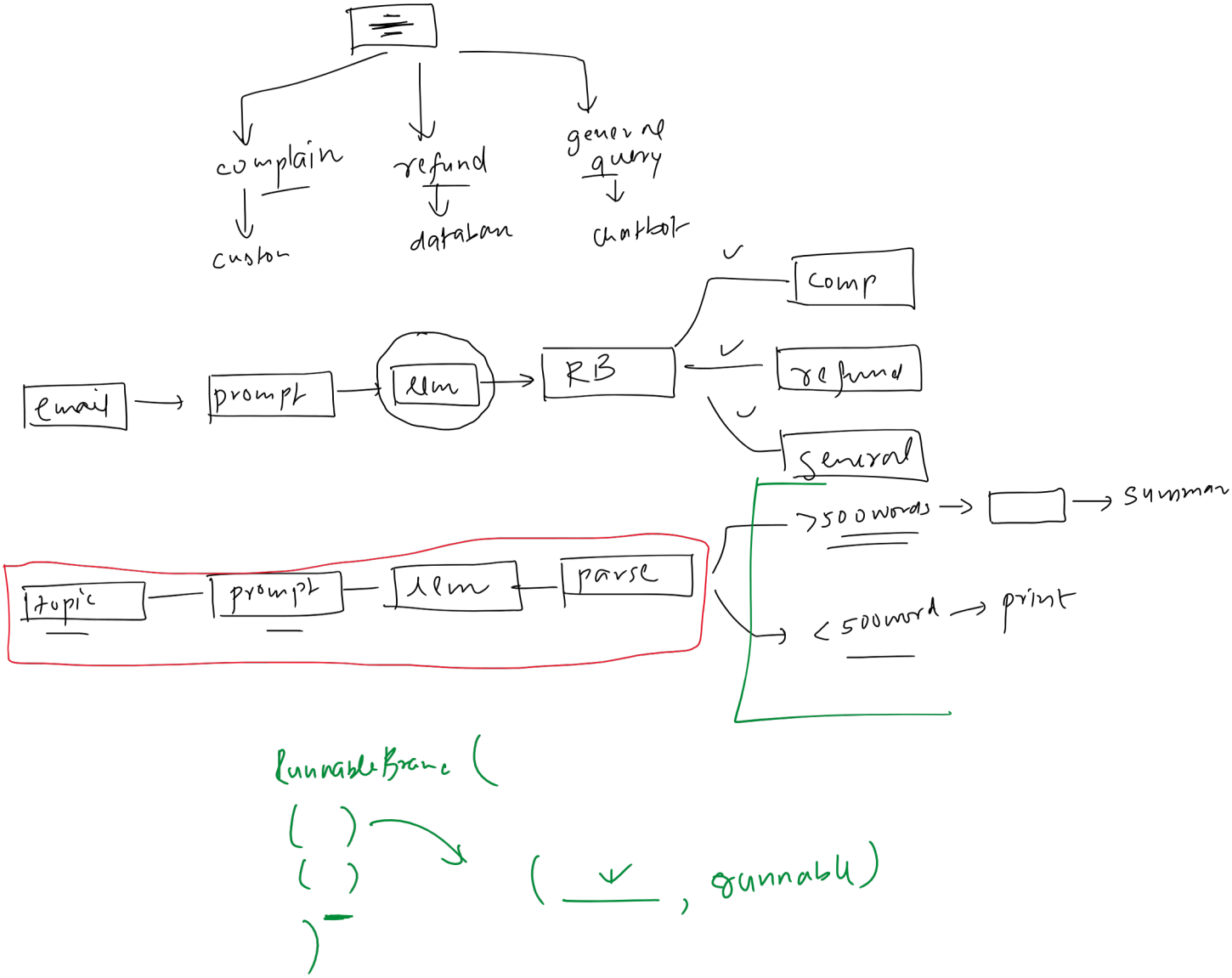


5. RunnableBranch → conditional chains

21 March 2025 08:02

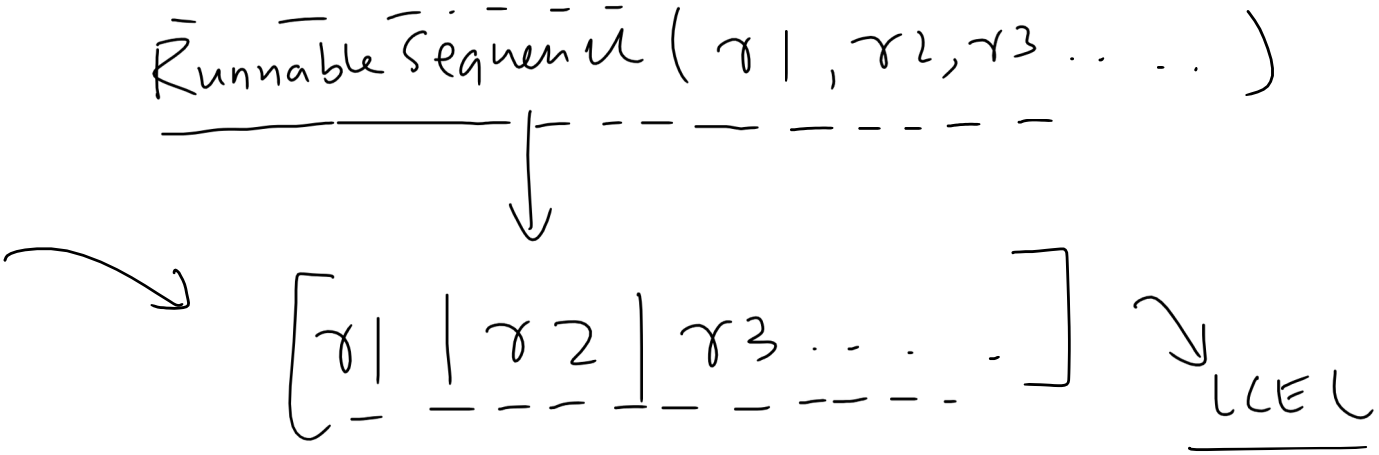
RunnableBranchis a control flow componentin LangChain that allows you to conditionally route input data to different chains or runnablesbased on custom logic.

It functions like an if/elif/elseblock for chains —where you define a set of condition functions, each associated with a runnable (e.g., LLM call, prompt chain, or tool). The first matching condition is executed. If no condition matches, a default runnableis used (if provided).



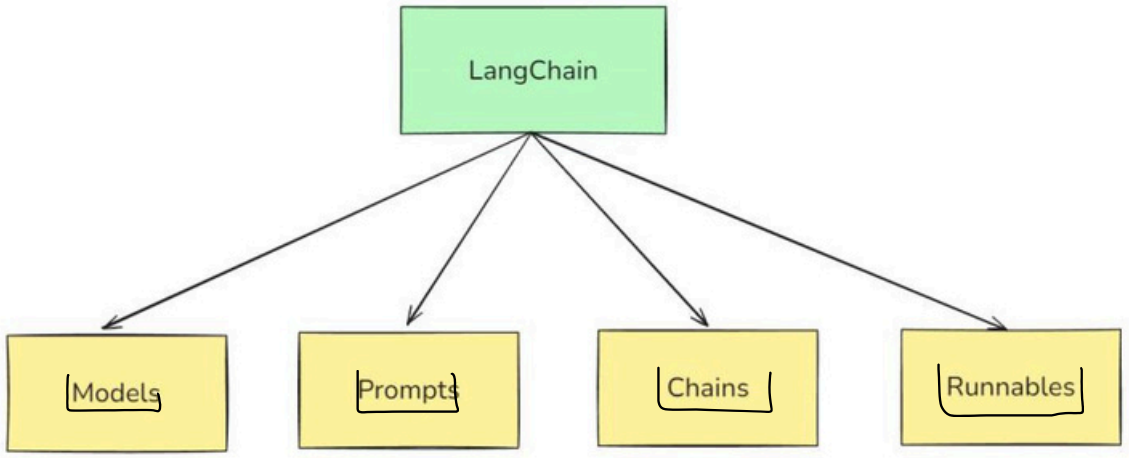
LCEL

21March 2025 08:39



Plan of Action

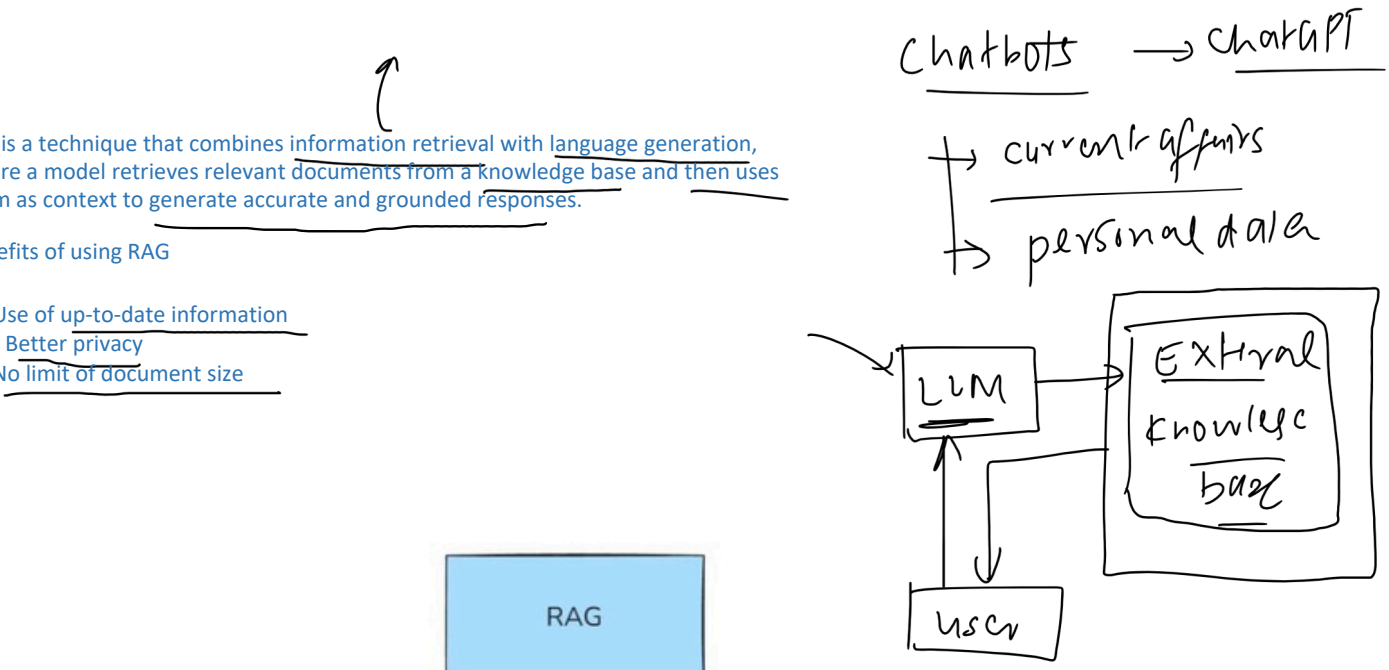
27 March 2025 10:59

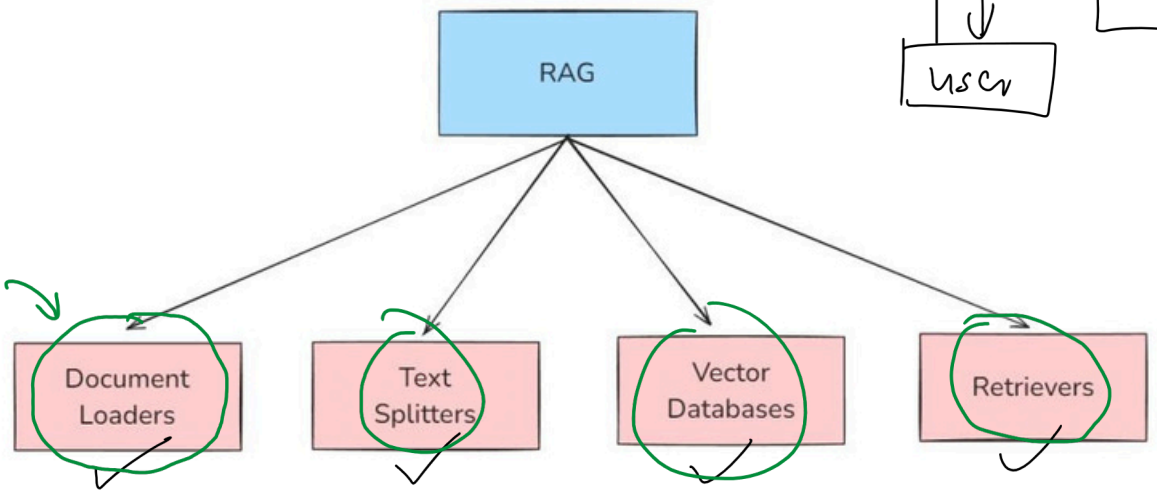


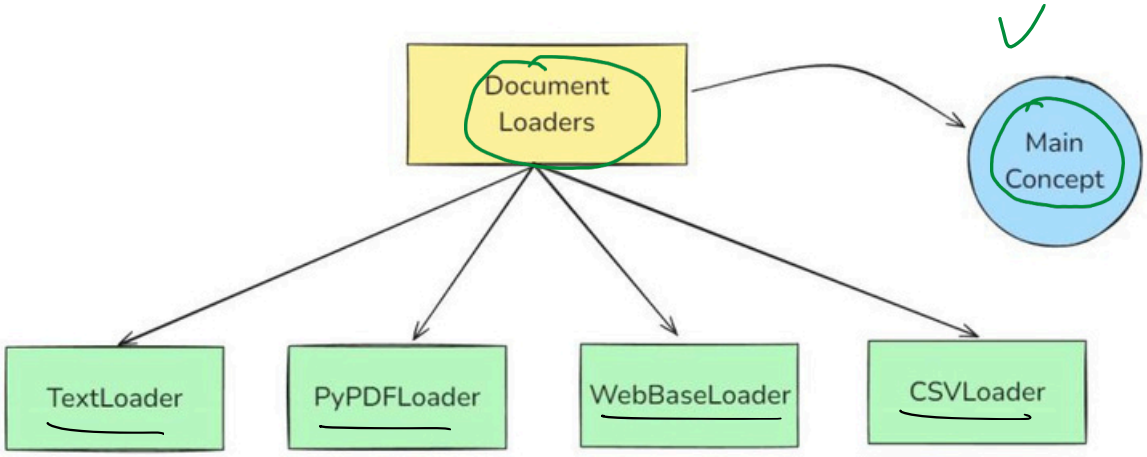
RAGis a technique that combines information retrieval with language generation, where a model retrieves relevant documents from a knowledge base and then uses them as context to generate accurate and grounded responses.

Benefits of using RAG

1.Use of up-to-date information
2. Better privacy
3.No limit of document size





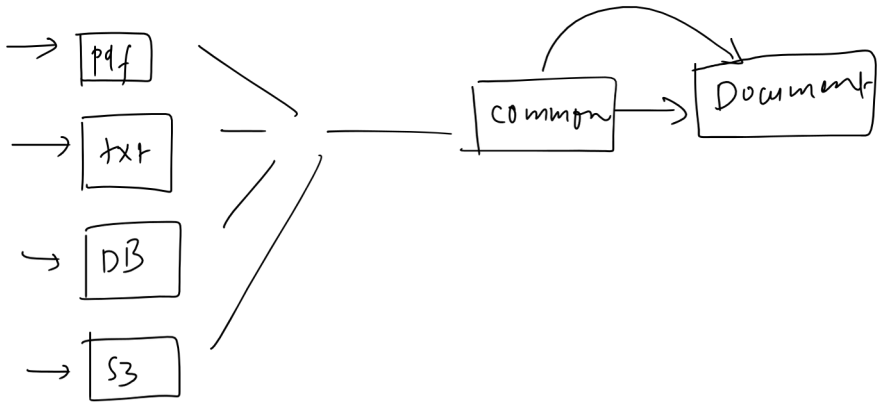

Document Loaders
Main Concept
TextLoader
PyPDFLoader
WebBaseLoader
CSVLoader

Document Loaders in LangChain

27 March 2025 16:20

Document loadersare components in LangChain used to load data from various sourcesinto a standardized format (usually as Documentobjects), which can then be used for chunking, embedding, retrieval, and generation.

```
Document(
    page_content="The actual text content",
    metadata={"source": "filename.pdf", ...}
)
```



TextLoader

27 March 2025 16:50

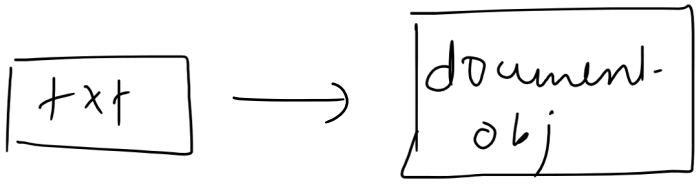
TextLoaderis a simple and commonly used document loaderin LangChain that reads plain text (.txt) filesand converts them into LangChain Documentobjects.

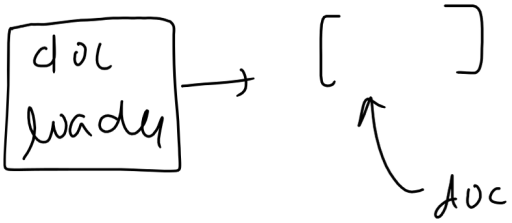
Use Case

- Ideal for loading chat logs, scraped text, transcripts, code snippets, or any plain text data into a LangChain pipeline.

Limitation

- Works only with .txt files






```python
from langchain_community.document_loaders import TextLoader
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI()

prompt = PromptTemplate(
    template='Write a summary for the following poem - \n {poem}',
    input_variables=['poem']
)

parser = StrOutputParser()

loader = TextLoader('cricket.txt', encoding='utf-8')

docs = loader.load()

print(type(docs))

print(len(docs))

print(docs[0].page_content)

print(docs[0].metadata)

chain = prompt | model | parser

print(chain.invoke({'poem':docs[0].page_content}))
```

PyPDFLoader → text → scanned pdf

27 March 2025 17:50

25 → PyPdf → 25 document

[—, —, —, —]

PyPDFLoaderis a document loaderin LangChain used to load content from PDF filesand convert each page into a Documentobject.

```
[
    Document(page_content="Text from page 1", metadata={"page": 0, "source": "file.pdf"}),
    Document(page_content="Text from page 2", metadata={"page": 1, "source": "file.pdf"}),
    ...
]
```

Limitations:

- It uses the PyPDFlibrary under the hood —not great with scanned PDFs or complex layouts.

Use Case	Recommended Loader
Simple, clean PDFs	`PyPDFLoader`
PDFs with tables/columns	`PDFPlumberLoader`
Scanned/image PDFs	`UnstructuredPDFLoader` or `AmazonTextractPDFLoader`
Need layout and image data	`PyMuPDFLoader`
Want best structure extraction	`UnstructuredPDFLoader`

```python
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader('dl-curriculum.pdf')

docs = loader.load()

print(len(docs))

print(docs[0].page_content)
print(docs[1].metadata)
```

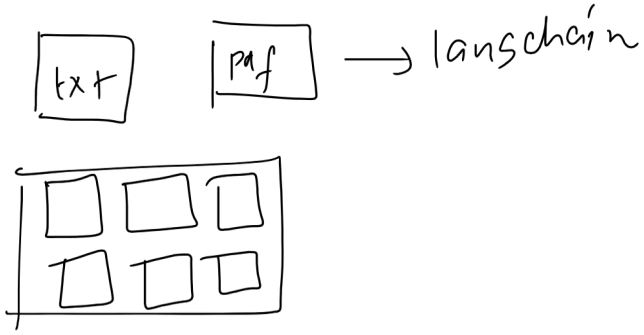
DirectoryLoader

27 March 2025 18:44

DirectoryLoaderis a document loader that lets you load multiple documentsfrom a directory (folder)of files.

Glob Pattern	What It Loads
`"**/*.txt"`	All `.txt` files in all subfolders
`"*.pdf"`	All `.pdf` files in the root directory
`"data/*.csv"`	All `.csv` files in the `data/` folder
`"**/*"`	All files (any type, all folders)

`**` = recursive search through subfolders




```python
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader

loader = DirectoryLoader(
    path='books',
    glob='*.pdf',
    loader_cls=PyPDFLoader
)

docs = loader.lazy_load()

for document in docs:
    print(document.metadata)
```

Load vs Lazy load → 500 pdf → generator of docs →

27 March 2025 23:51

eager loading

✅ `load()`

- **Eager Loading** (loads everything at once).
- Returns: A **list of** `Document` **objects**.
- Loads all documents **immediately** into memory.
- Best when:
  - The number of documents is small.
  - You want everything loaded upfront.

🌀 `lazy_load()`

- **Lazy Loading** (loads on demand).
- Returns: A **generator** of `Document` objects.
- Documents are **not all loaded at once**; they're fetched one at a time as needed.
- Best when:
  - You're dealing with **large documents or lots of files**.
  - You want to **stream** processing (e.g., chunking, embedding) without using lots of memory.

WebBaseLoader

28 March 2025 00:34

WebBaseLoaderis a document loaderin LangChain used to load and extract text content from web pages (URLs).
It uses BeautifulSoupunder the hood to parse HTML and extract visible text.

When to Use:

- For blogs, news articles, or public websites where the content is primarily text-based and static.

Limitations:

- Doesn't handle JavaScript-heavy pages well (use SeleniumURLLoaderfor that).
- Loads only static content (what's in the HTML, not what loads after the page renders).

```python
from langchain_community.document_loaders import WebBaseLoader
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI()

prompt = PromptTemplate(
    template='Answer the following question \n {question} from the following text - \n {text}',
    input_variables=['question','text']
)

parser = StrOutputParser()

url = 'https://www.flipkart.com/apple-macbook-air-m2-16-gb-256-gb-ssd-macos-sequoia-mc7x4hn-a/p/itmdc5308fa78421'
loader = WebBaseLoader(url)

docs = loader.load()


chain = prompt | model | parser

print(chain.invoke({'question':'What is the prodcut that we are talking about?', 'text':docs[0].page_content}))
```

CSVLoader

28 March 2025 01:48

CSVLoaderis a document loader used to load CSV filesinto LangChain Documentobjects —one per row, by default.

```python
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader(file_path='Social_Network_Ads.csv')

docs = loader.load()

print(len(docs))
print(docs[1])
```

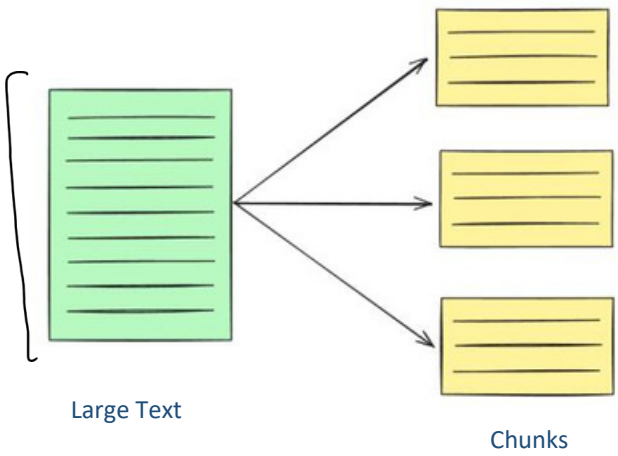
Other Document Loaders

28 March 2025 01:58

Text Splitting

01 April 2025 18:10

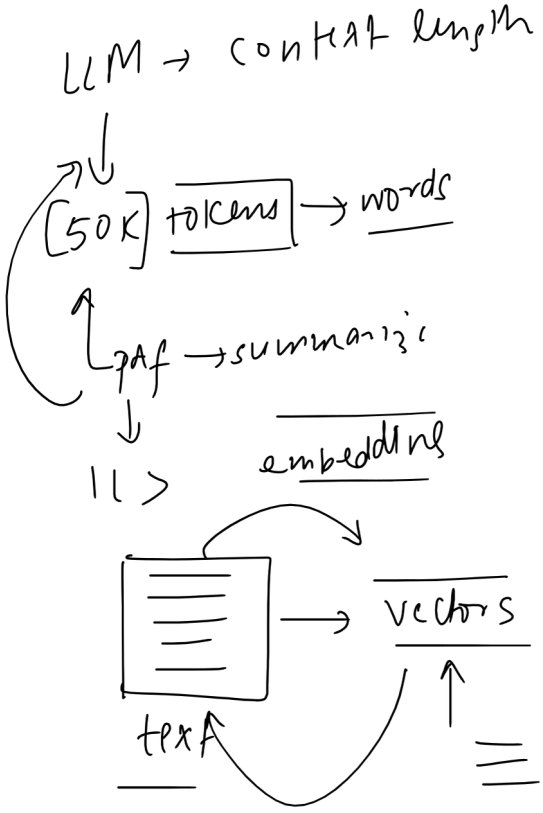
Text Splitting is the process of breaking large chunks of text (like articles, PDFs, HTML pages, or books) into smaller, manageable pieces (chunks)that an LLM can handle effectively.

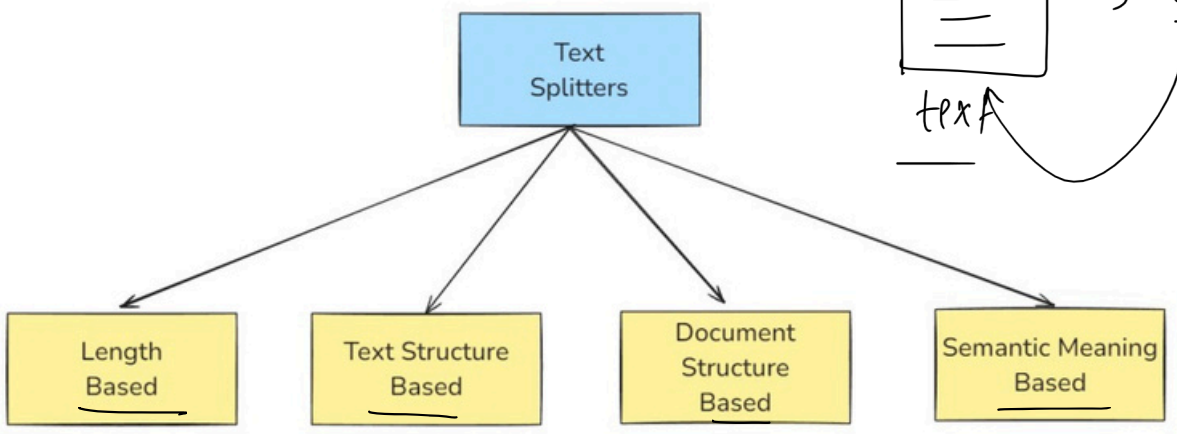


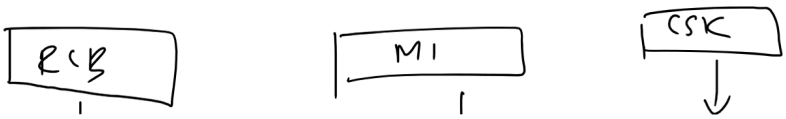
- Overcoming model limitations: Many embedding models and language models have maximum input size constraints. Splitting allows us to process documents that would otherwise exceed these limits.
- Downstream tasks-Text Splitting improves nearly every LLM powered task

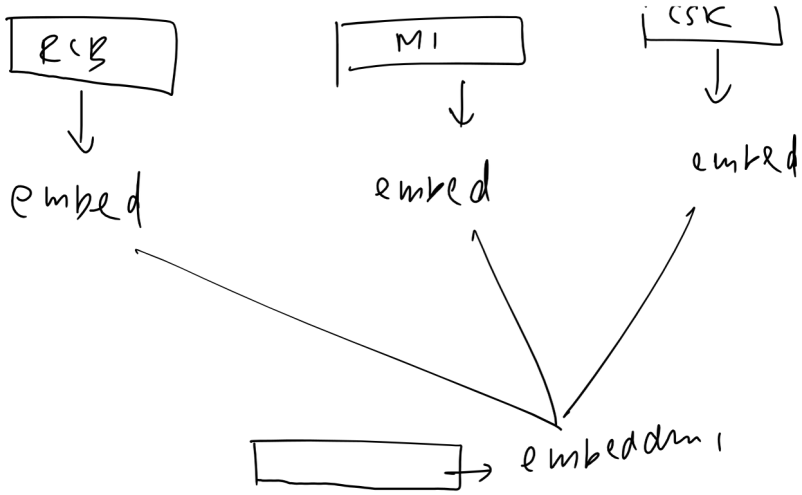
Task	Why Splitting Helps
Embedding	Short chunks yield more accurate vectors
Semantic Search	Search results point to focused info, not noise
Summarization	Prevents hallucination and topic drift

- Optimizing computational resources: Working with smaller chunks of text can be more memory-efficient and allow for better parallelization of processing tasks.








RCB
MI
CSK
embed
embed
embed
embeddings

1. Length Based Text Splitting

01 April 2025 18:10

Space exploration has led to incredible scientific discoveries. From landing on the Moon to exploring Mars, humanity continues to push the boundaries of what's possible beyond our planet.

These missions have not only expanded our knowledge of the universe but have also contributed to advancements in technology here on Earth. Satellite communications, GPS, and even certain medical imaging techniques trace their roots back to innovations driven by space programs.

Space exploration has led to incredible scientific discoveries. From landing on the Moon to explorin → c1

g Mars, humanity continues to push the boundaries of what's possible beyond our planet. These missi → c2

ons have not only expanded our knowledge of the universe but have also contributed to advancements in → c3

n technology here on Earth. Satellite communications, GPS, and even certain medical imaging techniqu → c4

es trace their roots back to innovations driven by space programs. → c5

chunks → size

chunk

100 characters

chars

tokens

```python
from langchain.text_splitter import CharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader('dl-curriculum.pdf')

docs = loader.load()

splitter = CharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=0,
    separator=''
)

result = splitter.split_documents(docs)

print(result[1].page_content)
```

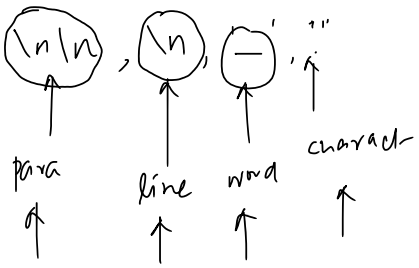
2. Text-Structured Based

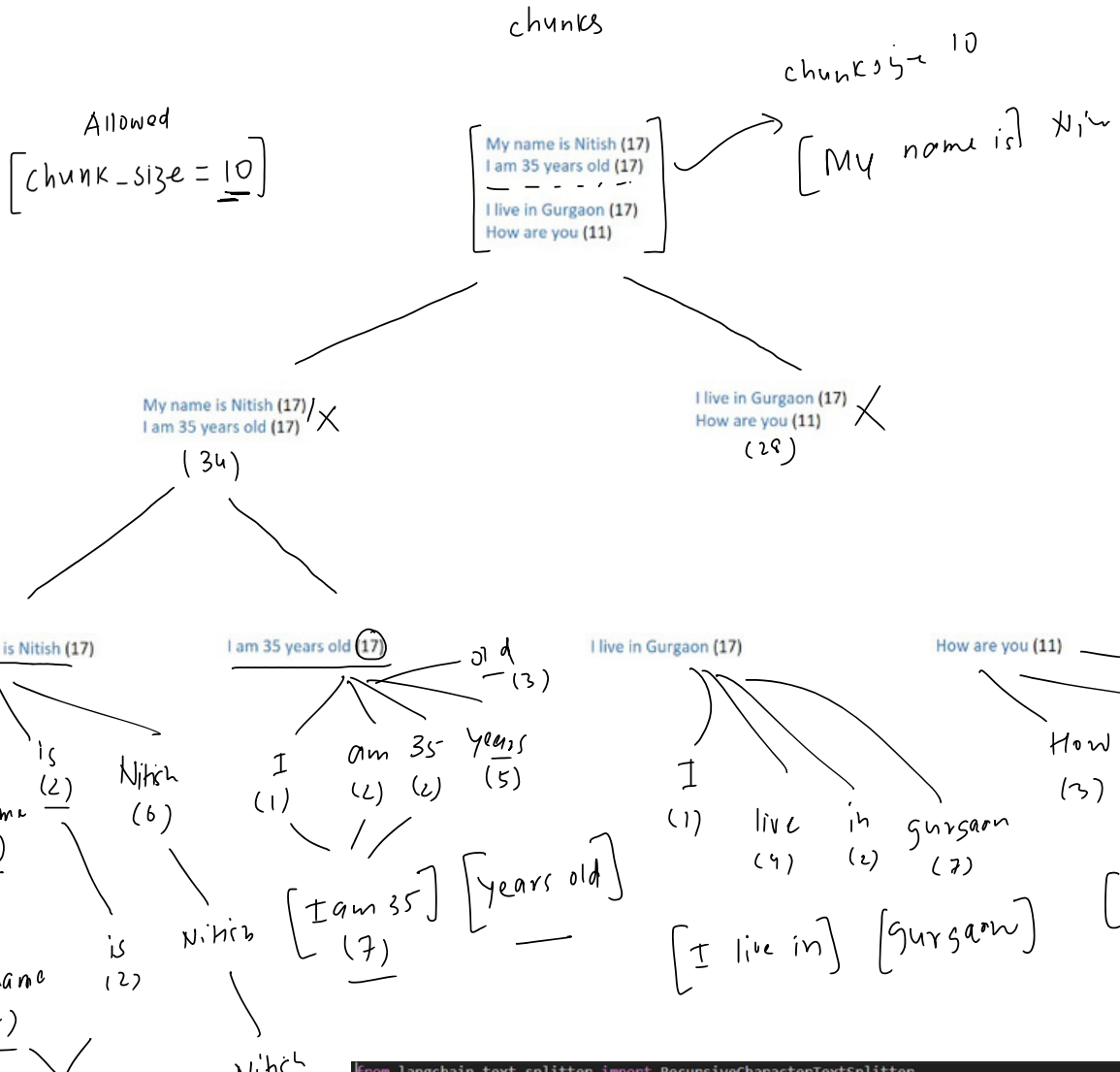
01 April 2025 18:10

My name is Nitish
Iam35 yearsold

I live in Gurgaon
How are you

structure



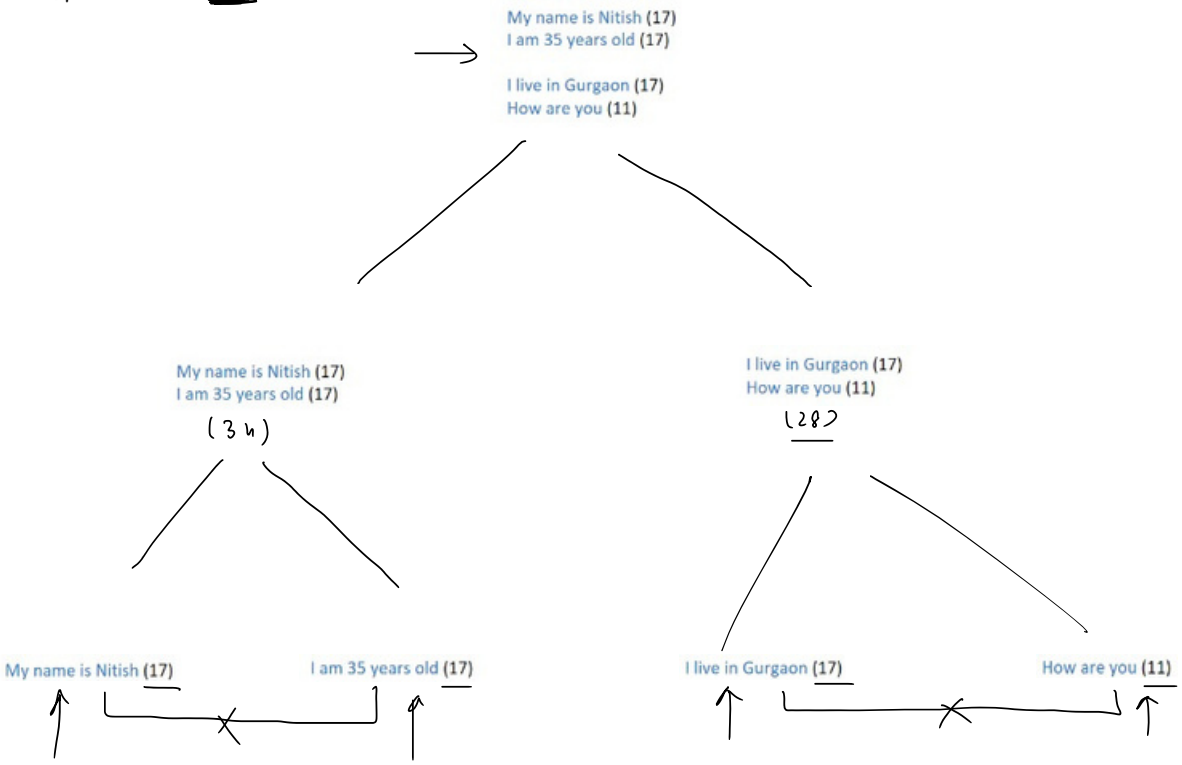



```python
from langchain.text_splitter import RecursiveCharacterTextSplitter

text = """
Space exploration has led to incredible scientific discoveries. From landing on the Moon to exploring Mars, humanity conti

These missions have not only expanded our knowledge of the universe but have also contributed to advancements in technolog
"""

# Initialize the splitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=0,
)

# Perform the split
chunks = splitter.split_text(text)

print(len(chunks))
print(chunks)
```

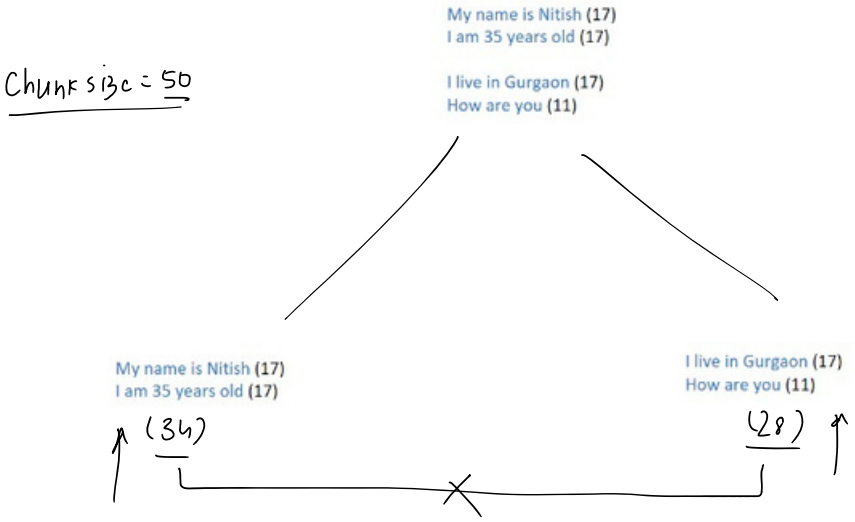
Chunk size = 25

My name is Nitish (17)
I am 35 years old (17)

Chunk size = 25



Chunk size = 50



3. Document-Structured Based

01 April 2025 18:11

markdown → LLM

```
# 🟦 Project Name: Smart Student Tracker

A simple Python-based project to manage and track student data,

---

## 🔍 Features

- Add new students with relevant info
- View student details
- Check if a student is passing
- Easily extendable class-based design

---

## 🛠 Tech Stack

- Python 3.10+
- No external dependencies
```

code →

```python
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade  # Grade is a float (like 8.5 or 9.2)

    def get_details(self):
        return f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}"

    def is_passing(self):
        return self.grade >= 6.0


# Example usage
student1 = Student("Aarav", 20, 8.2)
print(student1.get_details())

if student1.is_passing():
    print("The student is passing.")
else:
    print("The student is not passing.")
```

```
# First, try to split along Markdown headings (starting with level 2)
"\n#{1,6} ",
# Note the alternative syntax for headings (below) is not handled here
# Heading level 2
# ---------------
# End of code block
"```\n",
# Horizontal lines
"\n\\*\\*\\*+\n",
"\n---+\n",
"\n___+\n",
# Note that this splitter doesn't handle horizontal lines defined
# by *three or more* of ***, ---, or ___, but this is not handled
"\n\n",
"\n",
" ",
"",
```

```
# First, try to split along class definitions
"\nclass ",
"\ndef ",
"\n\tdef ",
# Now split by the normal type of lines
"\n\n",
"\n",
" ",
"",
```

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter,Langu
text = """
# Project Name: Smart Student Tracker

A simple Python-based project to manage and track student data, includin


## Features

- Add new students with relevant info
- View student details
- Check if a student is passing
- Easily extendable class-based design


## 🛠 Tech Stack

- Python 3.10+
- No external dependencies


## Getting Started

1. Clone the repo
   ```bash
   git clone https://github.com/your-username/student-tracker.git

"""

# Initialize the splitter
splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.MARKDOWN,
    chunk_size=200,
    chunk_overlap=0,
)

# Perform the split
chunks = splitter.split_text(text)

print(len(chunks))
print(chunks[0])
```

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter,Language
text = """
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade  # Grade is a float (like 8.5 or 9.2)

    def get_details(self):
        return self.name"

    def is_passing(self):
        return self.grade >= 6.0


# Example usage
student1 = Student("Aarav", 20, 8.2)
print(student1.get_details())

if student1.is_passing():
    print("The student is passing.")
else:
    print("The student is not passing.")

"""

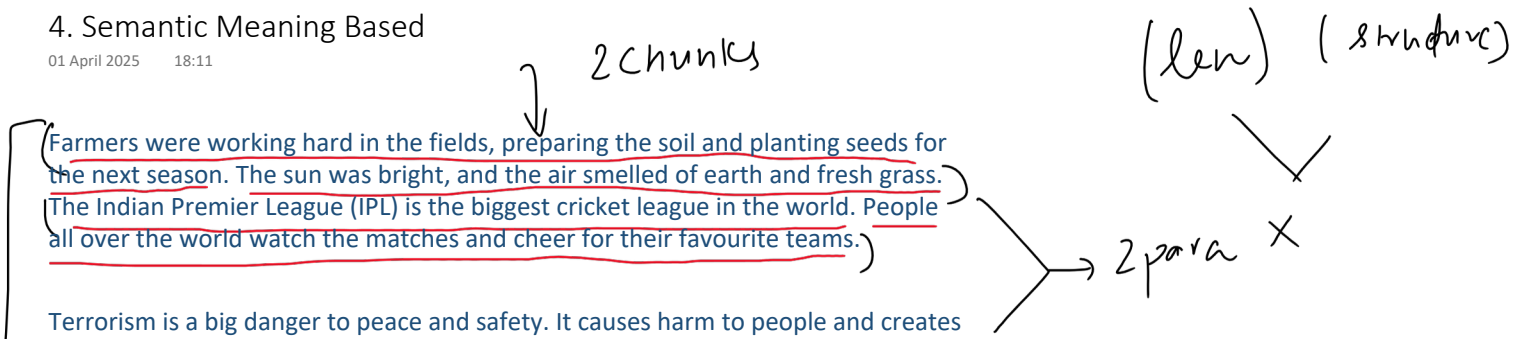
# Initialize the splitter
splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=300,
    chunk_overlap=0,
)

# Perform the split
chunks = splitter.split_text(text)

print(len(chunks))
print(chunks[1])
```
```

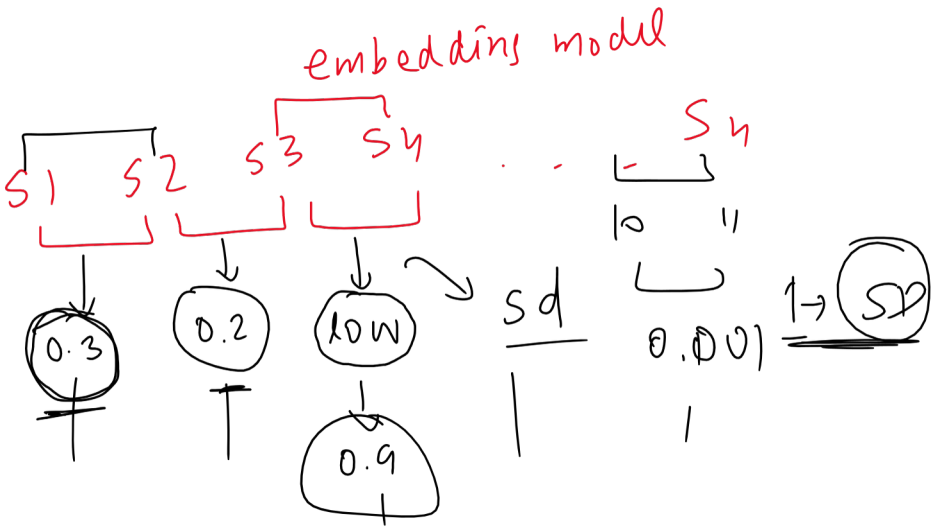
4. Semantic Meaning Based

01 April 2025 18:11



Farmers were working hard in the fields, preparing the soil and planting seeds for the next season. The sun was bright, and the air smelled of earth and fresh grass. The Indian Premier League (IPL) is the biggest cricket league in the world. People all over the world watch the matches and cheer for their favourite teams.

Terrorism is a big danger to peace and safety. It causes harm to people and creates fear in cities and villages. When such attacks happen, they leave behind pain and sadness. To fight terrorism, we need strong laws, alert security forces, and support from people who care about peace and safety.

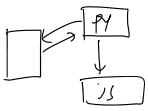



```python
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai.embeddings import OpenAIEmbeddings
from dotenv import load_dotenv

load_dotenv()

text_splitter = SemanticChunker(
    OpenAIEmbeddings(), breakpoint_threshold_type="standard_deviation",
    breakpoint_threshold_amount=3
)

sample = """
Farmers were working hard in the fields, preparing the soil and planting seeds for the next season. The sun was bright, and


Terrorism is a big danger to peace and safety. It causes harm to people and creates fear in cities and villages. When such
"""

docs = text_splitter.create_documents([sample])
print(len(docs))
print(docs)
```

Why Vector Stores?

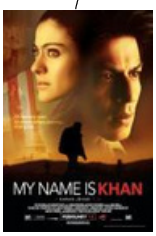
05 April 2025 17:40

Movie id	Movie name	Director	Actor	Genre	Release Date	Outcome
M001	3 Idiots	Raju Hirani	Aamir Khan	Drama, Romance	2009	Super Hit
M002	Chennai Express	Rohit Shetty	ShahRukh Khan	Romance, Comedy	2014	Super Hit
M003	Inception	C Nolan	L Di Caprio	Thriller, Sci-Fi	2009	Blockbuster Hit
M004	Stree	Amar Kaushik	Rajkumar Rao	Horror, Comedy	2019	



keyword match

m1 m2



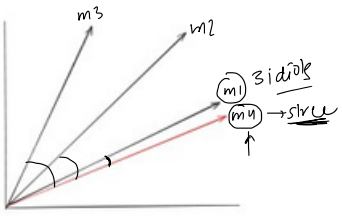
Movie id	Plot
M001	In the present day, Farhan receives a call from Chatur, saying that Ranchi is coming. Farhan is so excited that he fakes a heart attack to get off a flight and picks up Raju from his home (who forgets to wear his pants). Farhan and Raju meet Chatur at the water tower of their college ICE (Imperial College of Engineering), where Chatur informs them that he has found Rancho. Chatur taunts Farhan and Raju that now he has a mansion worth $3.5 million in the US, with a heated swimming pool, a maple wood floor living room and a Lamborghini for a car. Chatur reveals that Rancho is in Shimla...
M002	Rahul Mithaiwala (Shahrukh Khan) is a forty-year old bachelor who lives in Mumbai. His parents died in a car accident when he was eight years old and was brought up by grandparents. His grandfather has a sweet-selling chain store -Y.Y. Mithaiwala. Before his birth centenary celebration, two of Rahul's friends suggest a vacation in Goa which he accepts. On the eve of the celebration, his grandfather dies whilst watching a cricket match. His grandmother tells him that his grandfather desired to have his ashes divided into two parts -to be immersed in the Ganges River and Rameswaram respectively. She requests Rahul to go to Rameswaram and immerse them. Rahul reluctantly accepts her request but was also eager to attend the Goa trip...
M003	Dominick "Dom" Cobb (Leonardo DiCaprio) and business partner Arthur (Joseph Gordon-Levitt) are "extractors", people who perform corporate espionage using an experimental military technology to infiltrate the subconscious of their targets and extract information while experiencing shared dreaming. Their latest target is Japanese businessman Saito (Ken Watanabe). The extraction from Saito fails when sabotaged by a memory of Cobb's deceased wife Mal (Marion Cotillard). After Cobb's and Arthur's associate sells them out, Saito reveals that he was actually auditioning the team to perform the difficult act of "inception": planting an idea in a person's subconscious...
M004	In the peculiar town of Chanderi, India, the residents believe in the myth of an angry woman ghost, referred to as "Stree" (Hindi for woman) (Flora Saini), who stalks men during Durga Puja festival. This is explained by the sudden disappearance of these men, leaving their clothes behind. She is said to stalk the men of the town, whispering their names and causing disappearances if they look back at her. The whole town protects itself from Stree during the 4 nights of Durga Puja by writing "OO Stree, Kal Aana" on their walls. Additionally, men are advised to avoid going out alone after 10 PM during the festival and to move in groups for safety. This practice reflects a societal parallel to the precautions typically advised to women for their own protection...

512 dim

0.32	0.63	0.13	0.45	0.0	0.1	...	...	0.78	0.99
0.44	0.61	0.0	0.91	0.0	0.99	...	...	0.91	0.99
0.21	0.0	0.0	0.01	0.0	0.77	...	...	0.0	0.99
0.33	0.03	0.10	0.22	0.99	0.77	...	...	0.22	0.88

compare similarity score →

512 dim

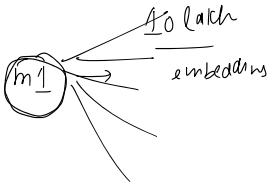


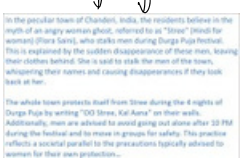
C1 → gen embedding vectors

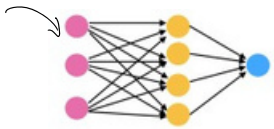
C2 → storage

C3 → semantic search intelligence

↳ Vector store







vectors

0.33	0.03	0.10	0.22	0.99	0.77	...	...	0.22	0.88

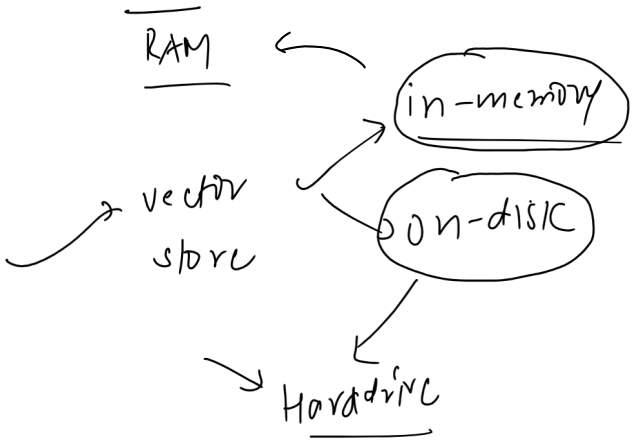
What are Vector Stores

05 April 2025 17:38

A vector storeis a system designed to store and retrieve data represented as numerical vectors.

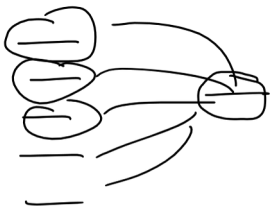
Key Features

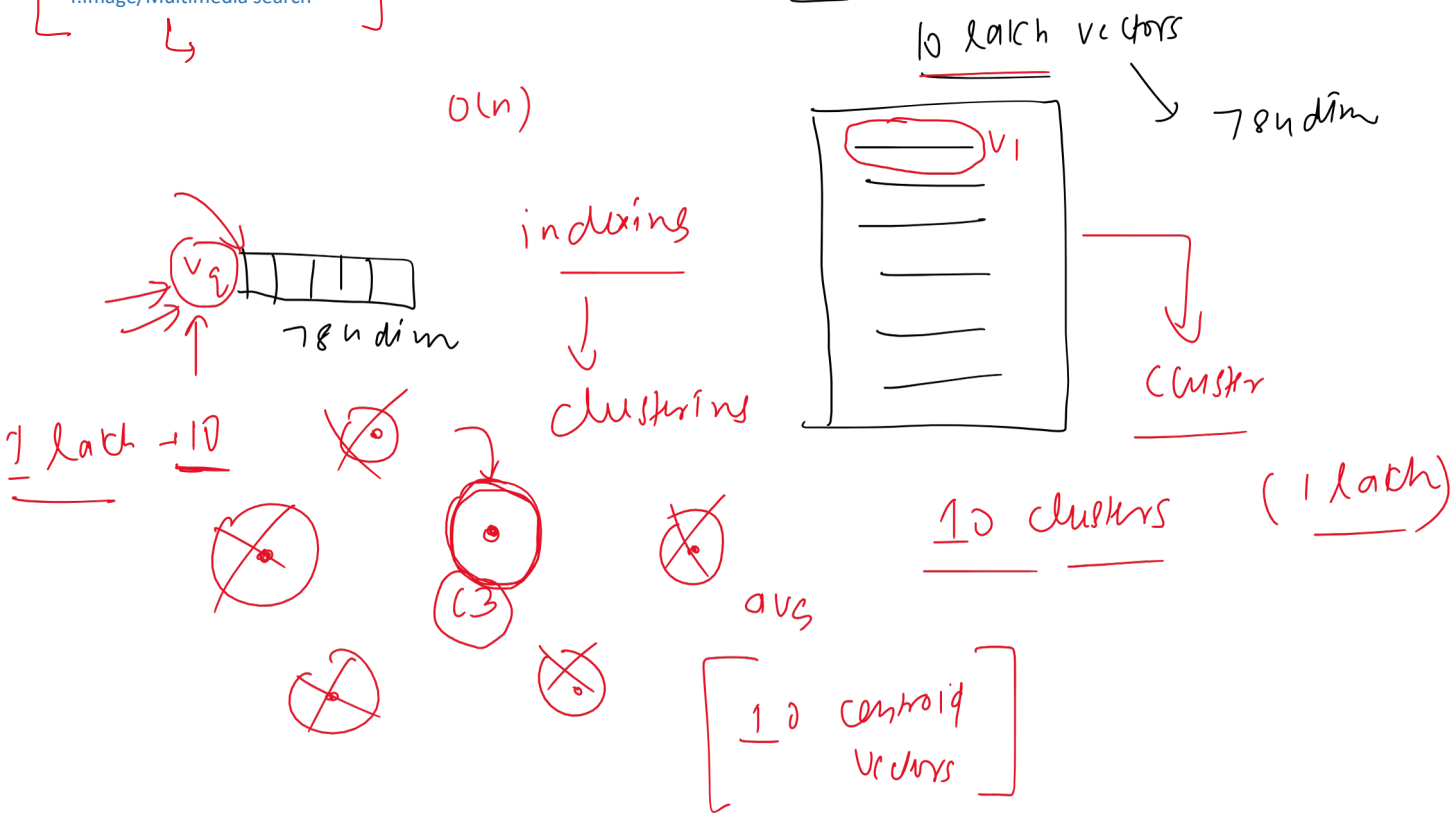
1. Storage–Ensures that vectors and their associated metadata are retained, whether in-memoryfor quick lookups or on-diskfor durability and large-scale use.
2. Similarity Search-Helps retrieve the vectors most similar to a query vector.
3. Indexing-Provide a data structure or method that enables fast similarity searcheson high-dimensional vectors (e.g., approximate nearest neighbor lookups).
4. CRUD Operations-Manage the lifecycle of data—adding new vectors, reading them, updating existing entries, removing outdated vectors.



Use-cases

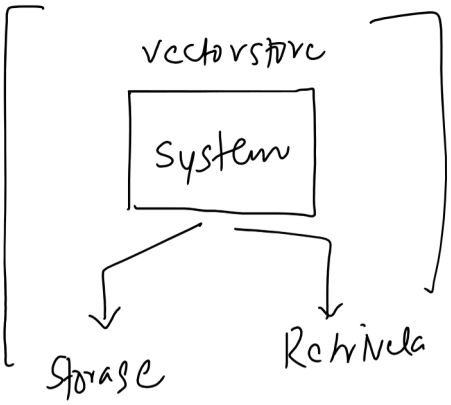
1. Semantic Search
2. RAG
3. Recommender Systems
4. Image/Multimedia search



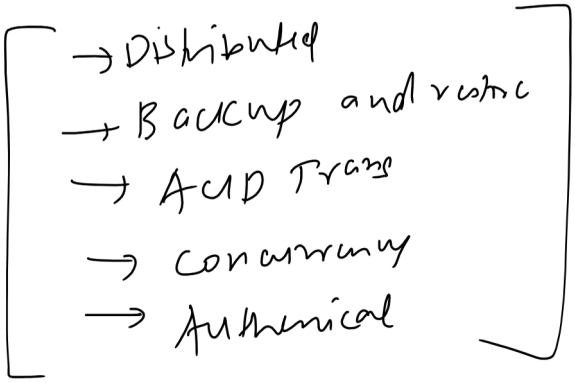


Vector Store Vs Vector Database

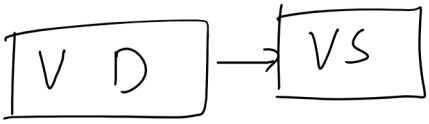
05 April 2025 17:40



- Vector Store
  - Typically refers to a lightweight library or servicethat focuses on storing vectors (embeddings) and performing similarity search.
  - May not include many traditional database features like transactions, rich query languages, or role-based access control.
  - Ideal for prototyping, smaller-scale applications
  - Examples: FAISS (where you store vectors and can query them by similarity, but you handle persistence and scaling separately).



- Vector Database
  - A full-fledged database systemdesigned to store and query vectors.
  - Offers additional "database-like" features:
    - Distributed architecture for horizontal scaling
    - Durability and persistence (replication, backup/restore)
    - Metadata handling (schemas, filters)
    - Potential for ACIDor near-ACID guarantees
    - Authentication/authorization and more advanced security
  - Geared for productionenvironments with significant scaling, large datasets
  - Examples: Milvus, Qdrant, Weaviate. Pinecone

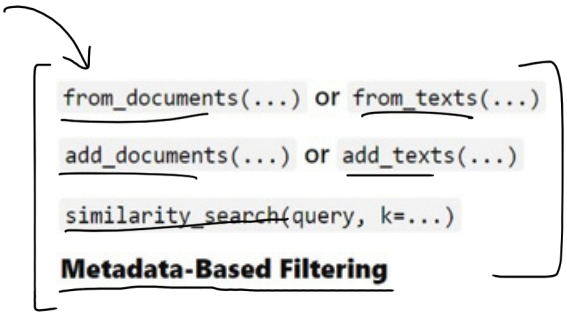


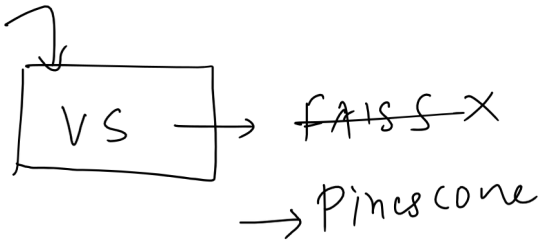
A vector databaseis effectively a vector storewith extra database features (e.g., clustering, scaling, security, metadata filtering, and durability)

Vector Stores in LangChain

05 April 2025 17:41

- Supported Stores: LangChain integrates with multiple vector stores(FAISS, Pinecone, Chroma, Qdrant, Weaviate, etc.), giving you flexibility in scale, features, and deployment.
- Common Interface: A uniform Vector StoreAPI lets you swap outone backend (e.g., FAISS) for another (e.g., Pinecone) with minimal code changes.
- Metadata Handling: Most vector stores in LangChain allow you to attach metadata(e.g., timestamps, authors) to each document, enabling filter-basedretrieval.

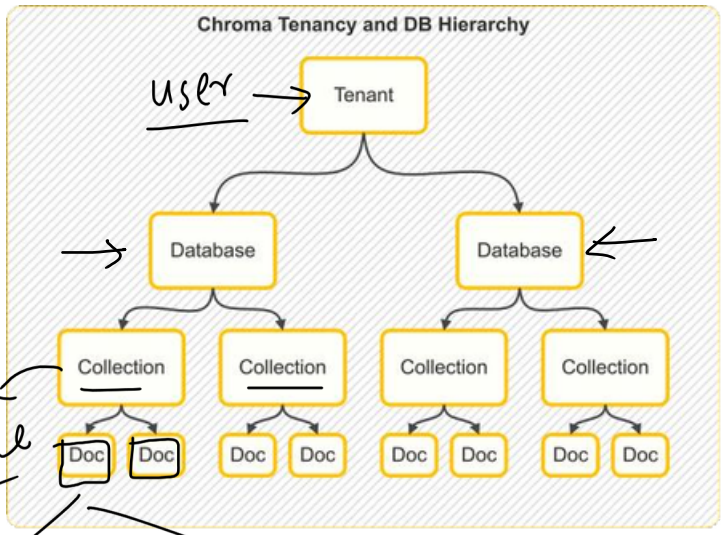




Chroma Vector Store

05 April 2025 17:41

Chroma is a lightweight, open-source vector databasethat is especially friendly for local developmentand small-to medium-scaleproduction needs.



Code

05 April 2025 17:41

```
from langchain_openai import OpenAIEmbeddings
from langchain.vectorstores import Chroma
```

```
from langchain.schema import Document

# Create LangChain documents for IPL players

doc1 = Document(
        page_content="Virat Kohli is one of the most successful and consistent batsmen in IPL history. Known for his aggres
        metadata={"team": "Royal Challengers Bangalore"}
    )
doc2 = Document(
        page_content="Rohit Sharma is the most successful captain in IPL history, leading Mumbai Indians to five titles. He
        metadata={"team": "Mumbai Indians"}
    )
doc3 = Document(
        page_content="MS Dhoni, famously known as Captain Cool, has led Chennai Super Kings to multiple IPL titles. His fin
        metadata={"team": "Chennai Super Kings"}
    )
doc4 = Document(
        page_content="Jasprit Bumrah is considered one of the best fast bowlers in T20 cricket. Playing for Mumbai Indians,
        metadata={"team": "Mumbai Indians"}
    )
doc5 = Document(
        page_content="Ravindra Jadeja is a dynamic all-rounder who contributes with both bat and ball. Representing Chennai
        metadata={"team": "Chennai Super Kings"}
    )
```

```
docs = [doc1, doc2, doc3, doc4, doc5]
```

```
vector_store = Chroma(
    embedding_function=OpenAIEmbeddings(),
    persist_directory='my_chroma_db',
    collection_name='sample'
)
```

```
# add documents
vector_store.add_documents(docs)
```

```
['09a39dc6-3ba6-4ea7-927e-fdda591da5e4',
 '8b561bf2-72ce-4295-8097-27e6f3bcd582',
 'aa800a1a-4b4f-4e58-8fce-ae2279eb385b',
 'eb519603-4222-46d5-a596-11797a4d39b6',
 '1182b187-1e5b-4e2d-a076-0119742940cc']
```

```python
# view documents
vector_store.get(include=['embeddings','documents', 'metadatas'])
```

```
{'ids': ['09a39dc6-3ba6-4ea7-927e-fdda591da5e4',
  '8b561bf2-72ce-4295-8097-27e6f3bcd582',
  'aa800a1a-4b4f-4e58-8fce-ae2279eb385b',
  'eb519603-4222-46d5-a596-11797a4d39b6',
  '1182b187-1e5b-4e2d-a076-0119742940cc'],
 'embeddings': array([[-0.00210453, -0.00214285,  0.0268    , ..., -0.01707893,
         -0.00366616,  0.01357884],
        [-0.00268021, -0.00010323,  0.02815653, ..., -0.01501936,
          0.00590092, -0.01164922],
        [ 0.00092799, -0.00476   ,  0.0124662 , ..., -0.01731381,
          0.00075886,  0.00296567],
        [-0.02714536,  0.00885395,  0.02699314, ..., -0.02592762,
          0.00900617, -0.01999116],
        [-0.01810451,  0.01281202,  0.0347942 , ..., -0.03034012,
         -0.00595078,  0.00521716]]),
 'documents': ['Virat Kohli is one of the most successful and consistent batsmen
  "Rohit Sharma is the most successful captain in IPL history, leading Mumbai Ind
  'MS Dhoni, famously known as Captain Cool, has led Chennai Super Kings to multi
  'Jasprit Bumrah is considered one of the best fast bowlers in T20 cricket. Play
  'Ravindra Jadeja is a dynamic all-rounder who contributes with both bat and bal
 'uris': None,
 'included': ['embeddings', 'documents', 'metadatas'],
 'data': None,
 'metadatas': [{'team': 'Royal Challengers Bangalore'},
  {'team': 'Mumbai Indians'},
  {'team': 'Chennai Super Kings'},
  {'team': 'Mumbai Indians'},
  {'team': 'Chennai Super Kings'}]}
```

```python
# search documents
vector_store.similarity_search(
    query='Who among these are a bowler?',
    k=2
)
```

```
[Document(metadata={'team': 'Mumbai Indians'}, page_cont
 Document(metadata={'team': 'Chennai Super Kings'}, page_
```

```python
# search with similarity score
vector_store.similarity_search_with_score(
    query='Who among these are a bowler?',
    k=2
)
```

```
[(Document(metadata={'team': 'Mumbai Indians'}, page_con
  0.35445845127105713),
 (Document(metadata={'team': 'Chennai Super Kings'}, pag
  0.4085893929004669)]
```

```python
# meta-data filtering
vector_store.similarity_search_with_score(
    query="",
    filter={"team": "Chennai Super Kings"}
)
```

```
[(Document(metadata={'team': 'Chennai Super Kings'}, pag
  0.6488258242607117),
 (Document(metadata={'team': 'Chennai Super Kings'}, pag
```

```python
# update documents
updated_doc1 = Document(
    page_content="Virat Kohli, the former captain of Royal Challengers Bangalore (RCB), is renowned for his aggressive leadership and
    metadata={"team": "Royal Challengers Bangalore"}
)

vector_store.update_document(document_id='09a39dc6-3ba6-4ea7-927e-fdda591da5e4', document=updated_doc1)
```

```python
# view documents
vector_store.get(include=['embeddings','documents', 'metadatas'])
```

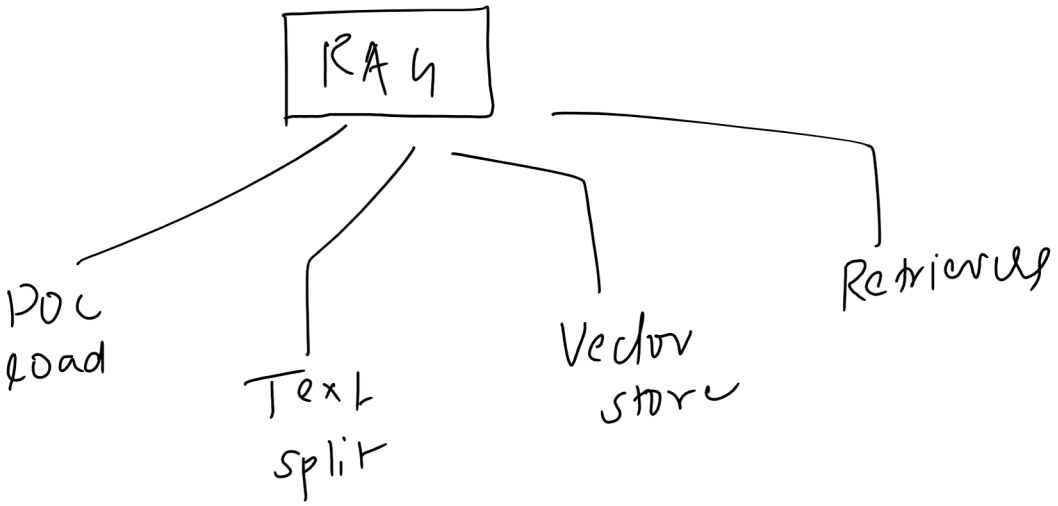
```python
# delete document
vector_store.delete(ids=['09a39dc6-3ba6-4ea7-927e-fdda591da5e4'])
```

```python
# view documents
vector_store.get(include=['embeddings','documents', 'metadatas'])
```

```
'ids': ['8b561bf2-72ce-4295-8097-27e6f3bcd582',
 'aa800a1a-4b4f-4e58-8fce-ae2279eb385b',
 'eb519603-4222-46d5-a596-11797a4d39b6',
 '1182b187-1e5b-4e2d-a076-0119742940cc'],
'embeddings': array([[-0.00268021, -0.00010323,  0.02815653, ..., -0.01501936,
        0.00590092, -0.01164922],
       [ 0.00092799, -0.00476   ,  0.0124662 , ..., -0.01731381,
```

Recap

12 April 2025 13:51



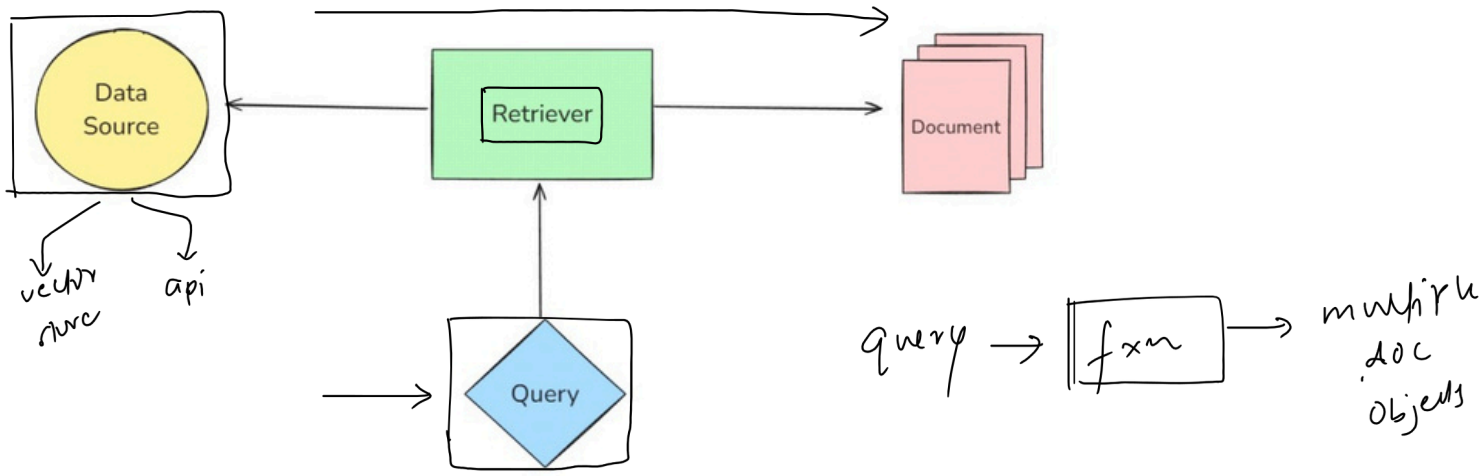
What are Retrievers

10 April 2025 07:56

A retrieveris a component in LangChain that fetches relevant documentsfrom a data source in response to a user's query.

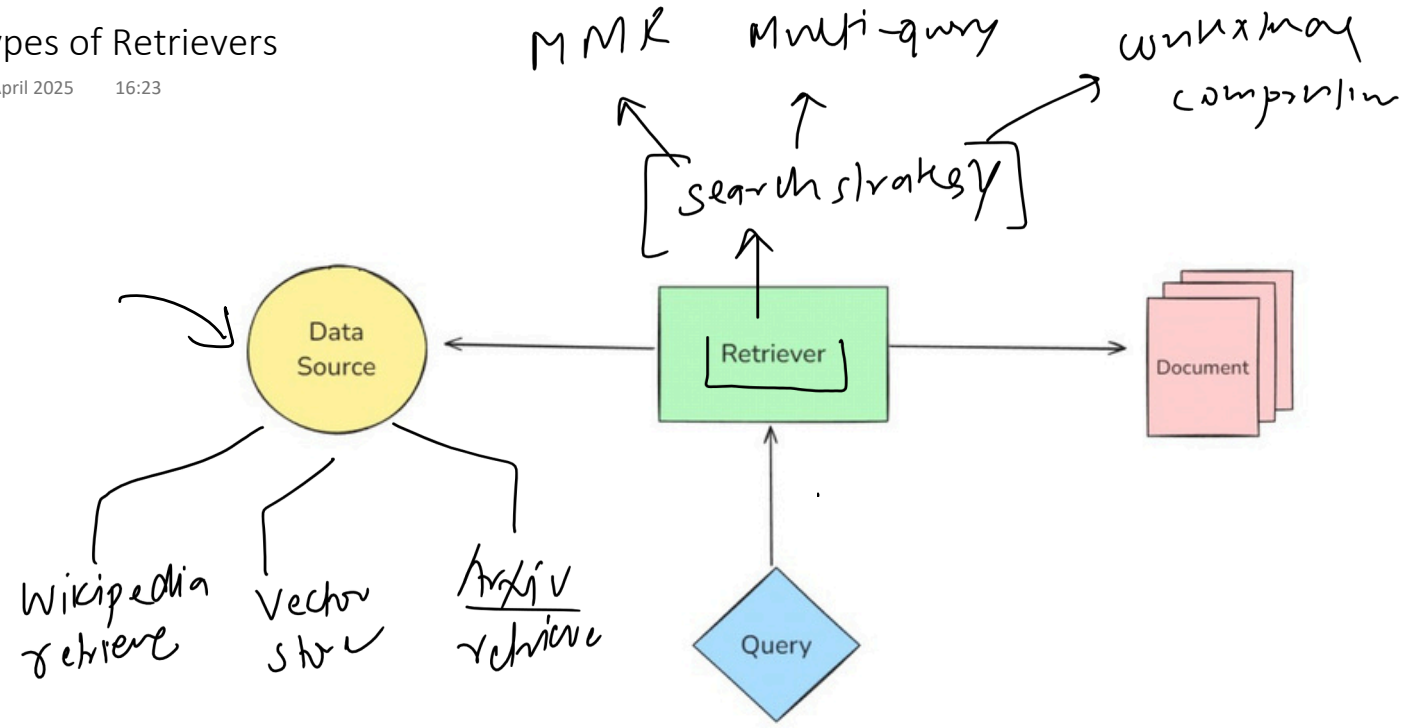
→ There are multiple types of retrievers

→ All retrievers in LangChain are runnables



Types of Retrievers

10 April 2025 16:23



Wikipedia Retriever → doc loader

10 April 2025 16:23

A Wikipedia Retrieveris a retriever that queries the Wikipedia APIto fetch relevant content for a given query.

ipl → API → doc

How It Works

1. You give it a query (e.g., "Albert Einstein")
2. It sends the query to Wikipedia's API
3. It retrieves the **most relevant articles**
4. It returns them as LangChain `Document` objects

Wikipedia Retriever

```
from langchain_community.retrievers import WikipediaRetriever
```

```
# Initialize the retriever (optional: set language and top_k)
retriever = WikipediaRetriever(top_k_results=2, lang="en")
```

```
# Define your query
query = "the geopolitical history of india and pakistan from the perspective of a chinese"

# Get relevant Wikipedia documents
docs = retriever.invoke(query)
```

```
docs
```

```
[Document(metadata={'title': 'United States aid to Pakistan', 'summary': 'The United States had be
 Document(metadata={'title': 'Foreign relations of Pakistan', 'summary': "The Islamic Republic of
```

```
# Print retrieved content
for i, doc in enumerate(docs):
    print(f"\n--- Result {i+1} ---")
    print(f"Content:\n{doc.page_content}...")  # truncate for display
```

Vector Store Retriever

10 April 2025 16:24

A Vector Store Retrieverin LangChain is the most common type of retriever that lets you search and fetch documents from a vector store based on semantic similarityusing vector embeddings.

How It Works

1. You store your documents in a **vector store** (like FAISS, Chroma, Weaviate)
2. Each document is converted into a **dense vector** using an **embedding model**
3. When the user enters a query:
    - It's also turned into a vector
    - The retriever compares the query vector with the stored vectors
    - It retrieves the top-k most similar ones

Vector Store Retriever

```python
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_core.documents import Document
```

```python
# Step 1: Your source documents
documents = [
    Document(page_content="LangChain helps developers build LLM applications easily."),
    Document(page_content="Chroma is a vector database optimized for LLM-based search."),
    Document(page_content="Embeddings convert text into high-dimensional vectors."),
    Document(page_content="OpenAI provides powerful embedding models."),
]
```

```python
# Step 2: Initialize embedding model
embedding_model = OpenAIEmbeddings()

# Step 3: Create Chroma vector store in memory
vectorstore = Chroma.from_documents(
    documents=documents,
    embedding=embedding_model,
    collection_name="my_collection"
)
```

```python
# Step 4: Convert vectorstore into a retriever
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})
```

```python
query = "What is Chroma used for?"
results = retriever.invoke(query)
```

```python
for i, doc in enumerate(results):
    print(f"\n--- Result {i+1} ---")
    print(doc.page_content)
```

```
-- Result 1 ---
hroma is a vector database optimized for LLM-based search.

-- Result 2 ---
angChain helps developers build LLM applications easily.
```

```python
results = vectorstore.similarity_search(query, k=2)
```

```python
for i, doc in enumerate(results):
    print(f"\n--- Result {i+1} ---")
    print(doc.page_content)
```

```
-- Result 1 ---
hroma is a vector database optimized for LLM-based search.

-- Result 2 ---
angChain helps developers build LLM applications easily.
```

Maximal Marginal Relevance (MMR)

10 April 2025 16:24

> "How can we pick results that are not only relevant to the query but also different from each other?"

MMR is an information retrieval algorithm designed to reduce redundancyin the retrieved results while maintaining high relevanceto the query.

🤔 Why MMR Retriever?

In regular similarity search, you may get documents that are:

- All very similar to each other
- Repeating the same info
- Lacking diverse perspectives

MMR Retriever avoids that by:

- Picking the **most relevant document** first
- Then picking the next most relevant **and least similar** to already selected docs
- And so on...

This helps especially in RAG pipelines where:

- You want your context window to contain **diverse but still relevant information**
- Especially useful when documents are semantically overlapping

Doc ID	Content
D1	"Climate change is causing glaciers to melt rapidly in the Arctic region."
D2	"Glaciers in the Arctic are melting at an alarming rate due to rising temperatures."
D3	"Deforestation in the Amazon is accelerating global climate change."
D4	"Climate change is increasing the frequency of wildfires in California."
D5	"Rising sea levels due to climate change threaten coastal cities like Mumbai and New York."

-diverse perspective

```
Top 3 results:
1. D1: Arctic glaciers melting
2. D2: Arctic glaciers melting
3. D3: Deforestation in Amazon
```

```
Top 3 results:
1. D1: Arctic glaciers melting
2. D4: Wildfires in California
3. D5: Rising sea levels in coastal cities
```

MMR

```python
# Sample documents
docs = [
    Document(page_content="LangChain makes it easy to work with LLMs."),
    Document(page_content="LangChain is used to build LLM based applications."),
    Document(page_content="Chroma is used to store and search document embeddings."),
    Document(page_content="Embeddings are vector representations of text."),
    Document(page_content="MMR helps you get diverse results when doing similarity search."),
    Document(page_content="LangChain supports Chroma, FAISS, Pinecone, and more."),
]
```

```python
from langchain_community.vectorstores import FAISS

# Initialize OpenAI embeddings
embedding_model = OpenAIEmbeddings()

# Step 2: Create the FAISS vector store from documents
vectorstore = FAISS.from_documents(
    documents=docs,
    embedding=embedding_model
)
```

Generate + Code + Markdown

```python
# Enable MMR in the retriever
retriever = vectorstore.as_retriever(
    search_type="mmr",                  # <-- This enables MMR
    search_kwargs={"k": 3, "lambda_mult": 0.5}  # k = top results, lambda_mult = relevance-diversity balance
)
```

```python
query = "What is langchain?"
results = retriever.invoke(query)
```

```python
for i, doc in enumerate(results):
    print(f"\n--- Result {i+1} ---")
    print(doc.page_content)
```

Multi-Query Retriever

10 April 2025 16:26

> **Sometimes a single query might not capture all the ways information is phrased in your documents.**

For example:

Query:

"How can I stay healthy?"

Could mean:

- What should I eat?
- How often should I exercise?
- How can I manage stress?

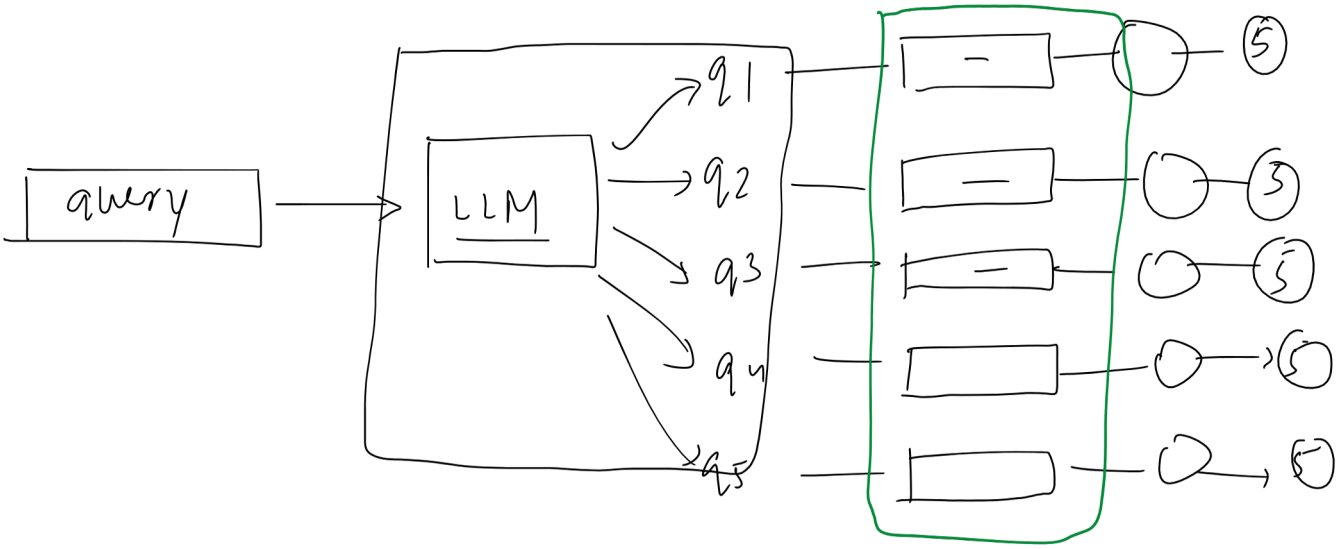
query → vector store

"How can I stay healthy?"

1. "What are the best foods to maintain good health?"
2. "How often should I exercise to stay fit?"
3. "What lifestyle habits improve mental and physical wellness?"
4. "How can I boost my immune system naturally?"
5. "What daily routines support long-term health?"

A simple similarity search might **miss documents** that talk about those things but don't use the word "healthy."

1. **Takes your original query**
2. **Uses an LLM (e.g., GPT-3.5)** to generate multiple semantically different versions of that query
3. **Performs retrieval for each sub-query**
4. **Combines and deduplicates the results**



Multiquery Retriever

```python
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_core.documents import Document
from langchain_openai import ChatOpenAI
from langchain.retrievers.multi_query import MultiQueryRetriever
```

```python
# Relevant health & wellness documents
all_docs = [
    Document(page_content="Regular walking boosts heart health and can reduce symptoms of depression.", metadata={"source": "H1"}),
    Document(page_content="Consuming leafy greens and fruits helps detox the body and improve longevity.", metadata={"source": "H2"}),
    Document(page_content="Deep sleep is crucial for cellular repair and emotional regulation.", metadata={"source": "H3"}),
    Document(page_content="Mindfulness and controlled breathing lower cortisol and improve mental clarity.", metadata={"source": "H4"}),
    Document(page_content="Drinking sufficient water throughout the day helps maintain metabolism and energy.", metadata={"source": "H5"}),
    Document(page_content="The solar energy system in modern homes helps balance electricity demand.", metadata={"source": "I1"}),
    Document(page_content="Python balances readability with power, making it a popular system design language.", metadata={"source": "I2"}),
    Document(page_content="Photosynthesis enables plants to produce energy by converting sunlight.", metadata={"source": "I3"}),
    Document(page_content="The 2022 FIFA World Cup was held in Qatar and drew global energy and excitement.", metadata={"source": "I4"}),
    Document(page_content="Black holes bend spacetime and store immense gravitational energy.", metadata={"source": "I5"}),
]
```

```python
# Initialize OpenAI embeddings
embedding_model = OpenAIEmbeddings()

# Create FAISS vector store
vectorstore = FAISS.from_documents(documents=all_docs, embedding=embedding_model)
```

```python
# Create retrievers
similarity_retriever = vectorstore.as_retriever(search_type="similarity", search_kwargs={"k": 5})
```

Generate + Code + Markdown

```python
multiquery_retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(search_kwargs={"k": 5}),
    llm=ChatOpenAI(model="gpt-3.5-turbo")
)
```

```python
# Query
query = "How to improve energy levels and maintain balance?"
```

```python
# Retrieve results
similarity_results = similarity_retriever.invoke(query)
multiquery_results= multiquery_retriever.invoke(query)
```

```python
for i, doc in enumerate(similarity_results):
    print(f"\n--- Result {i+1} ---")
    print(doc.page_content)

print("*"*150)

for i, doc in enumerate(multiquery_results):
    print(f"\n--- Result {i+1} ---")
    print(doc.page_content)
```

Contextual Compression Retriever

10 April 2025 16:29

The Contextual Compression Retrieverin LangChain is an advanced retriever that improves retrieval quality by compressing documents after retrieval—keeping only the relevant contentbased on the user's query.

❓ **Query:**

"What is photosynthesis?"

Retrieved Document (by a traditional retriever):

"The Grand Canyon is a famous natural site.
Photosynthesis is how plants convert light into energy.
Many tourists visit every year."

❌ **Problem:**

- The retriever returns the **entire paragraph**
- Only **one sentence** is actually relevant to the query
- The rest is **irrelevant noise** that wastes context window and may confuse the LLM

✅ **What Contextual Compression Retriever does:**

Returns only the relevant part, e.g.

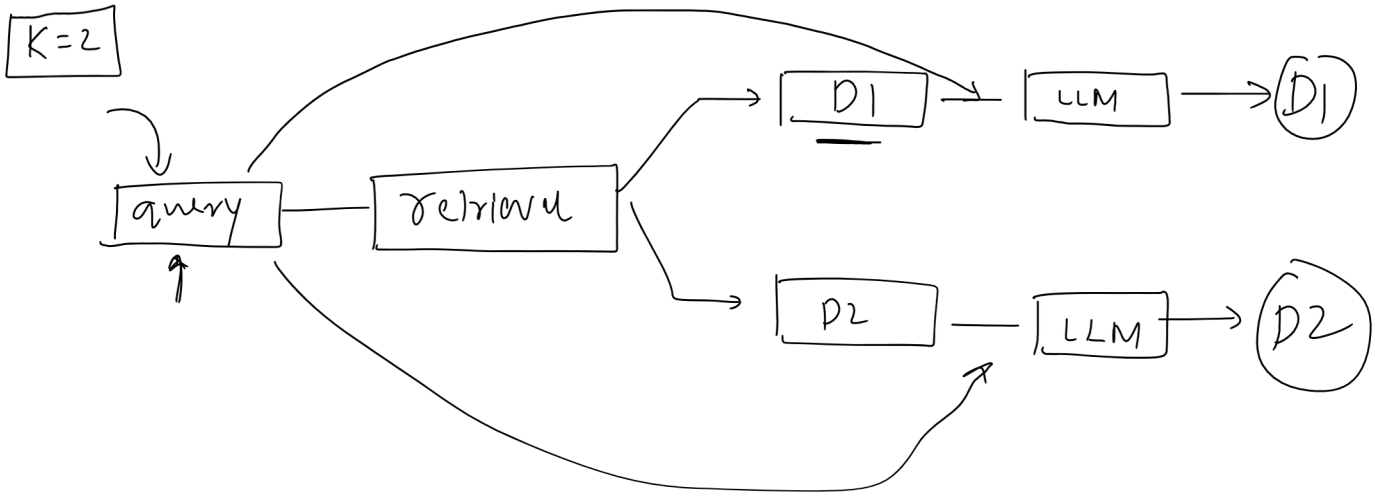
```
"Photosynthesis is how plants convert light into energy."
```

⚙️ **How It Works**

1. **Base Retriever** (e.g., FAISS, Chroma) retrieves N documents.
2. A **compressor** (usually an LLM) is applied to each document.
3. The compressor keeps **only the parts relevant to the query**.
4. Irrelevant content is discarded.

✅ **When to Use**

- Your documents are **long and contain mixed information**
- You want to **reduce context length** for LLMs
- You need to **improve answer accuracy** in RAG pipelines



ContextualCompressionRetriever

```python
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain.retrievers.contextual_compression import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor
from langchain_core.documents import Document
```

```python
# Recreate the document objects from the previous data
docs = [
    Document(page_content=(
        """The Grand Canyon is one of the most visited natural wonders in the world.
        Photosynthesis is the process by which green plants convert sunlight into energy.
        Millions of tourists travel to see it every year. The rocks date back millions of years."""
    ), metadata={"source": "Doc1"}),

    Document(page_content=(
        """In medieval Europe, castles were built primarily for defense.
        The chlorophyll in plant cells captures sunlight during photosynthesis.
        Knights wore armor made of metal. Siege weapons were often used to breach castle walls."""
    ), metadata={"source": "Doc2"}),

    Document(page_content=(
        """Basketball was invented by Dr. James Naismith in the late 19th century.
        It was originally played with a soccer ball and peach baskets. NBA is now a global league."""
    ), metadata={"source": "Doc3"}),

    Document(page_content=(
        """The history of cinema began in the late 1800s. Silent films were the earliest form.
        Thomas Edison was among the pioneers. Photosynthesis does not occur in animal cells.
        Modern filmmaking involves complex CGI and sound design."""
    ), metadata={"source": "Doc4"})
]
```

```python
# Create a FAISS vector store from the documents
embedding_model = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(docs, embedding_model)
```

```python
base_retriever = vectorstore.as_retriever(search_kwargs={"k": 5})
```

```python
# Set up the compressor using an LLM
llm = ChatOpenAI(model="gpt-3.5-turbo")
compressor = LLMChainExtractor.from_llm(llm)
```

```python
# Create the contextual compression retriever
compression_retriever = ContextualCompressionRetriever(
    base_retriever=base_retriever,
    base_compressor=compressor
)
```

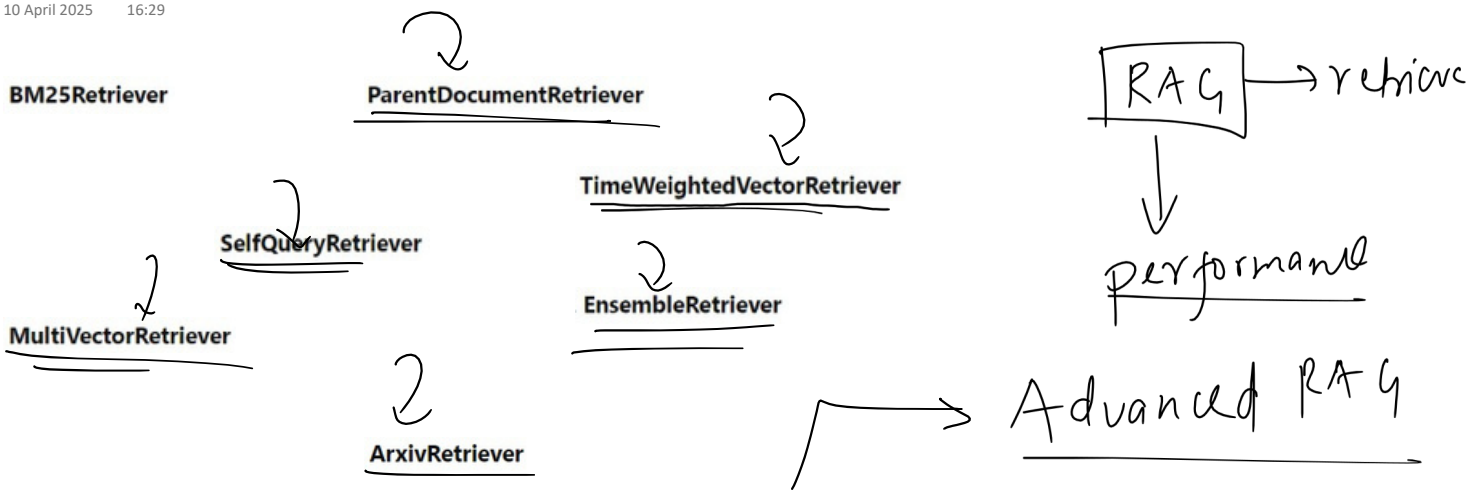
```python
# Query the retriever
query = "What is photosynthesis?"
compressed_results = compression_retriever.invoke(query)
```

```python
for i, doc in enumerate(compressed_results):
    print(f"\n--- Result {i+1} ---")
    print(doc.page_content)
```

```
--- Result 1 ---
Photosynthesis is the process by which green plants convert sunlight into energy.

--- Result 2 ---
The chlorophyll in plant cells captures sunlight during photosynthesis.
```

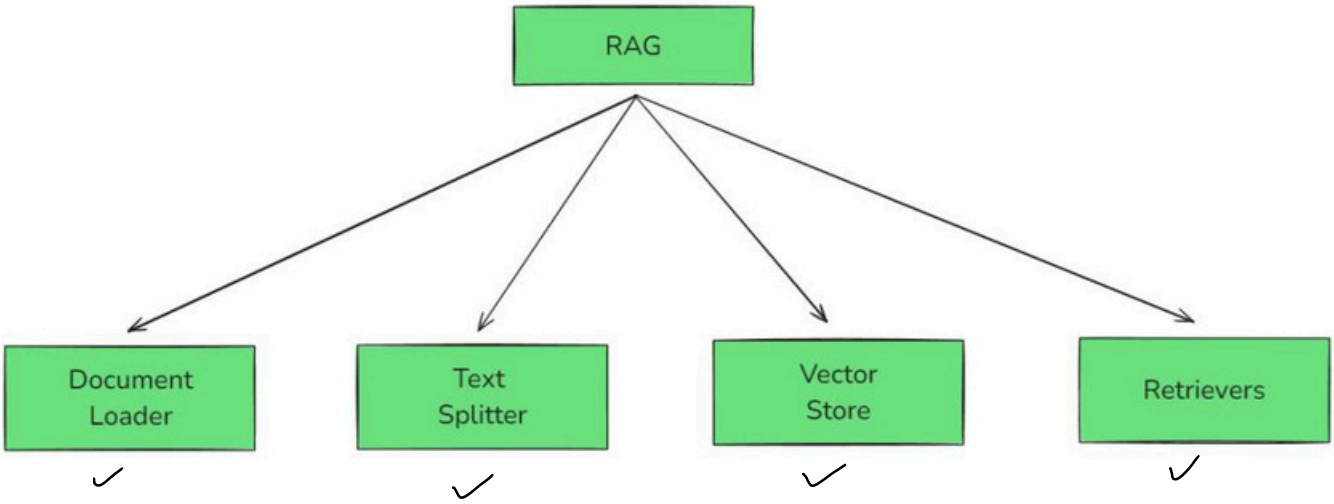
More Retrievers

10 April 2025 16:29



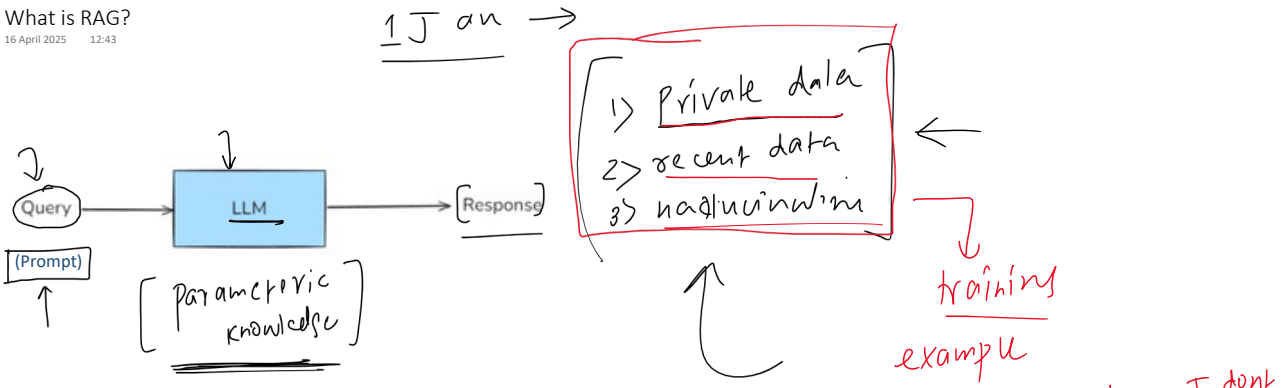
Recap

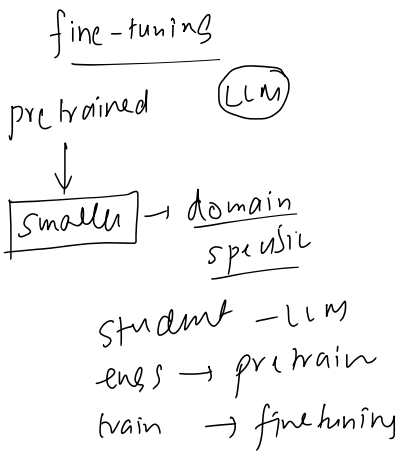
16 April 2025 12:45



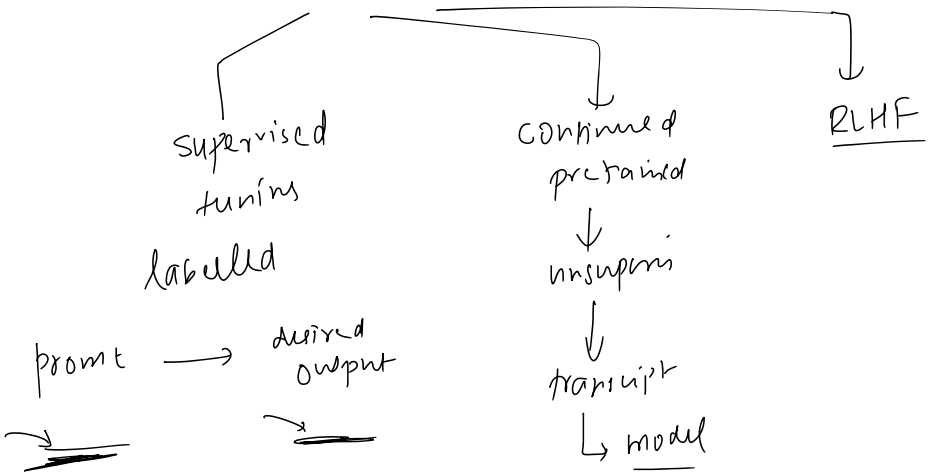
What is RAG?

16 April 2025 12:43





1. Collect data	A few hundred – few hundred-thousand carefully curated examples (prompts → desired outputs).
2. Choose a method	Full-parameter FT, LoRA/QLoRA, or parameter-efficient adapters.
3. Train for a few epochs	You keep the base weights frozen or partially frozen and update only a small subset (LoRA) or all weights (full FT).
4. Evaluate & safety-test	Measure exact-match, factuality, and hallucination rate against held-out data; red-team for safety.



1) LLM → train
2) technical expertise
3)

In context learning

In-Context Learning is a core capability of Large Language Models (LLMs) like GPT-3/4, Claude, and Llama, where the model learns to solve a task purely by seeing examples in the prompt—without updating its weights.

LLM → emergent property

```
Below are examples of texts labeled with their sentiment.
Use these examples to determine the sentiment of the final text.

Text: I love this phone. It's so smooth. → Positive
Text: This app crashes a lot. → Negative
Text: The camera is amazing! → Positive
```

LLM → emergent property

```
Use these examples to determine the sentiment of the final text.

Text: I love this phone. It's so smooth. → Positive
Text: This app crashes a lot. → Negative
Text: The camera is amazing! → Positive

Text: I hate the battery life. →
```

An **emergent property** is a behaviour or ability that suddenly appears in a system when it reaches a certain scale or complexity—even though it was not explicitly programmed or expected from the individual components.

GPT-1
GPT-2
→ prompts

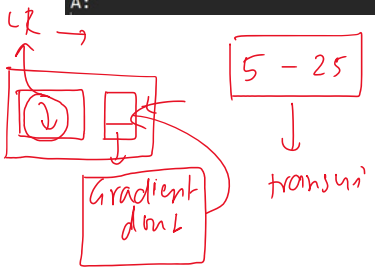
GPT-3 (175B)

```
Label the named entities in the sentences (Person, Location, Organization):

Sentence: Elon Musk founded SpaceX.
Entities: [Elon Musk → Person], [SpaceX → Organization]

Sentence: The Eiffel Tower is in Paris.
Entities: [Eiffel Tower → Location], [Paris → Location]

Sentence: Sundar Pichai leads Google.
Entities:
```

```
Solve the math problems step by step:

Q: John has 3 apples. He buys 4 more. How many apples does he have now?
A: John had 3 apples. He bought 4 more. 3 + 4 = 7. → 7

Q: Sarah has 10 pens. She gives away 3. How many does she have left?
A: Sarah had 10 pens. She gave away 3. 10 - 3 = 7. → 7

Q: A pizza is cut into 8 slices. Alex eats 3 slices. How many are left?
A:
```

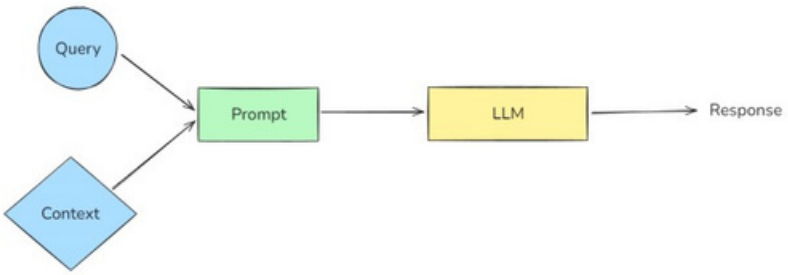
context

- Instead of just *example tasks*, retrieve **background information, facts, documents, product manuals,** etc.
- Inject that into the prompt to **augment the model's knowledge**



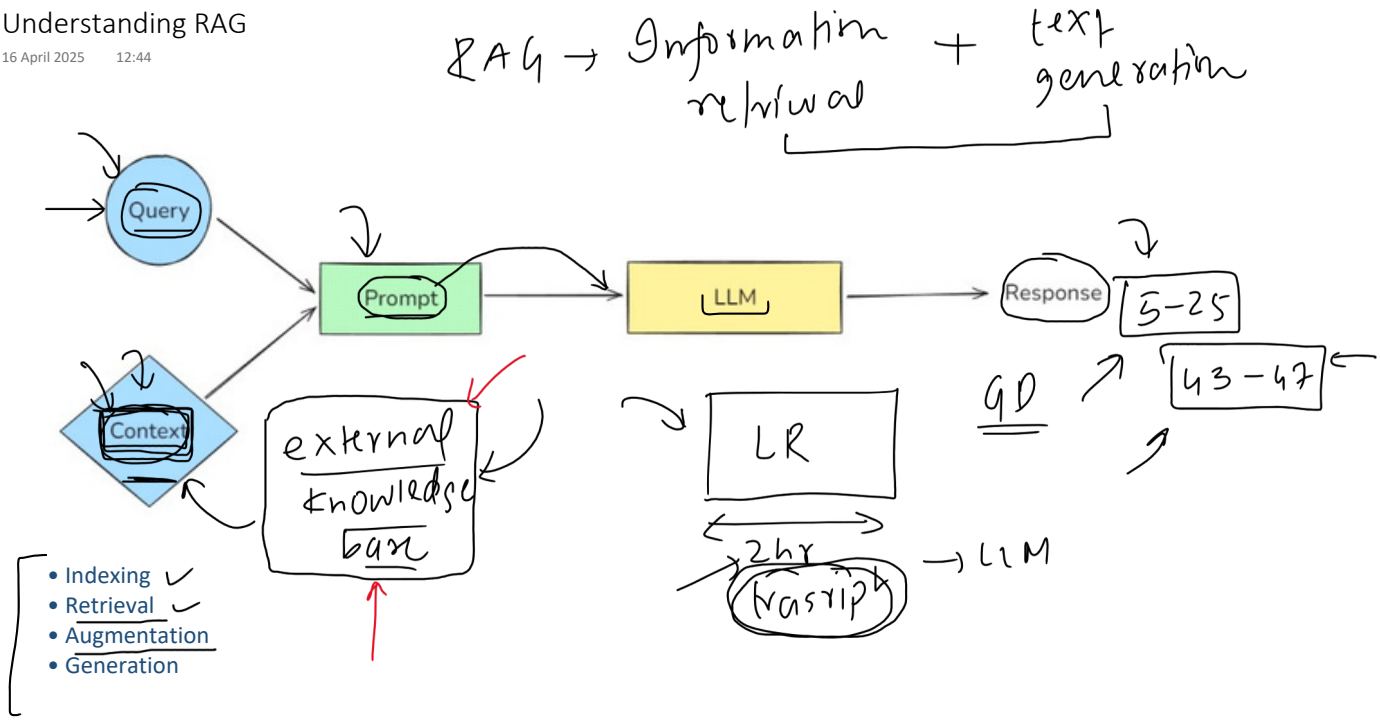

```
"""You are a helpful assistant.
Answer the question ONLY from the provided context.
If the context is insufficient, just say you don't know.

{context}
Question: {question}"""
```

RAG is a way to make a language model (like ChatGPT) smarter by giving it extra information at the time you ask your question.



Understanding RAG

16 April 2025 12:44



Indexing- Indexingis the process of **preparing your knowledge base so that it can be efficiently searched**at query time. This steps consists of 4 sub-steps.

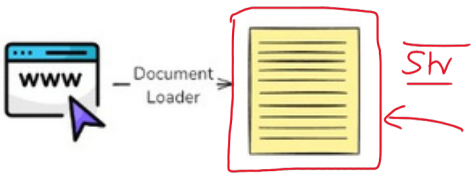
1.Document Ingestion -You load your source knowledge into memory.

Examples:

- PDF reports, Word documents
- YouTube transcripts, blog pages
- GitHub repos, internal wikis
- SQL records, scraped webpages

Tools:

- LangChain loaders (`PyPDFLoader`, `YoutubeLoader`, `WebBaseLoader`, `GitLoader`, etc.)



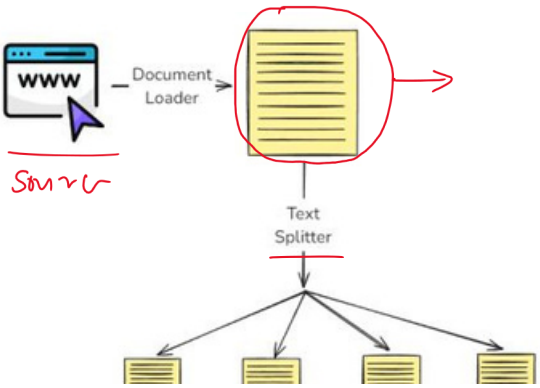
2.Text Chunking -Break large documents into small, semantically meaningful **chunks**

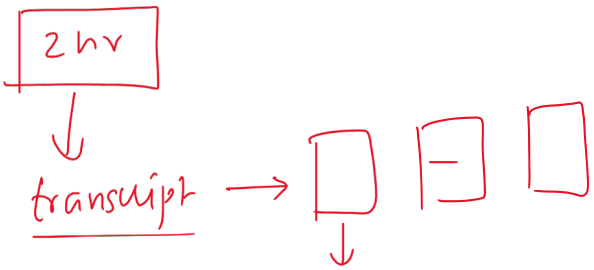
Why chunk?

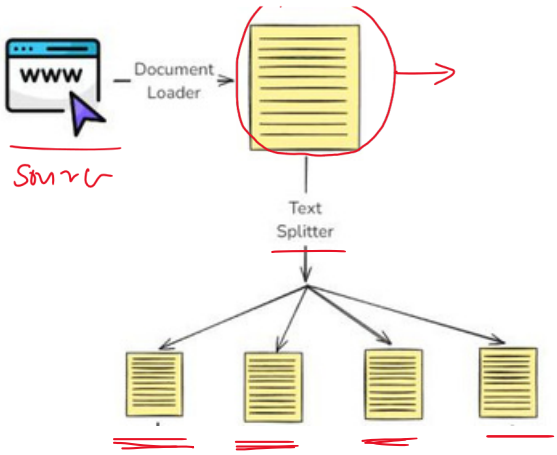
- LLMs have context limits (e.g., 4K–32K tokens)
- Smaller chunks are more focused → better semantic search

Tools:

- `RecursiveCharacterTextSplitter`, `MarkdownHeaderTextSplitter`, `SemanticChunker`







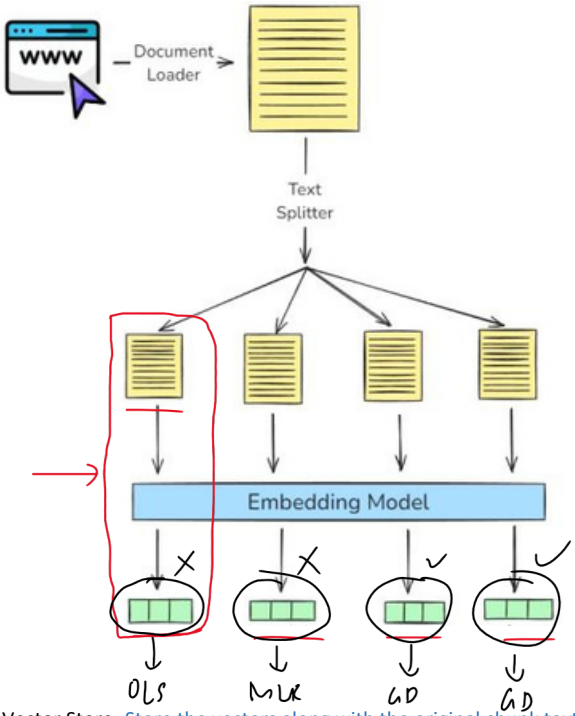
3.Embedding Generation -Convert each chunk into a **dense vector**(embedding) that captures its meaning.

🔍 Why embeddings?

- Similar ideas land close together in vector space
- Allows fast, fuzzy semantic search

🔧 Tools:

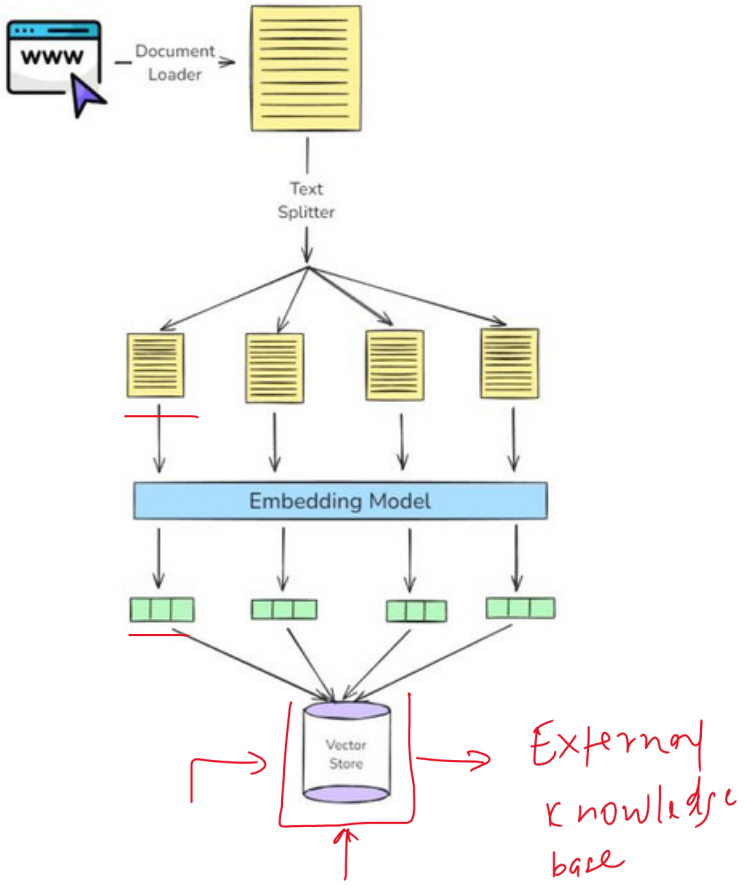
- `OpenAIEmbeddings`, `SentenceTransformerEmbeddings`, `InstructorEmbeddings`, etc.



4.Storage in a Vector Store -Store the vectors along with the original chunk text + metadata in a **vector database**.

🔧 Vector DB options:

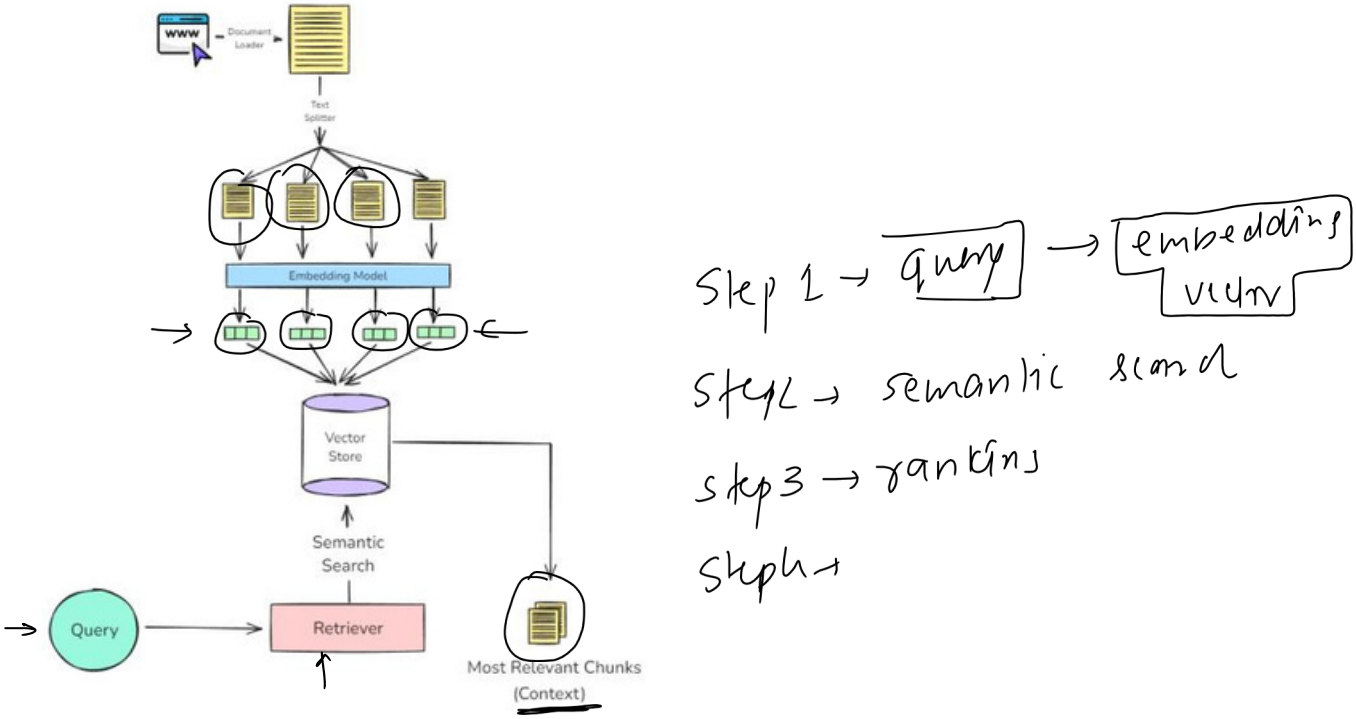
- Local: `FAISS`, `Chroma`
- Cloud: `Pinecone`, `Weaviate`, `Milvus`, `Qdrant`



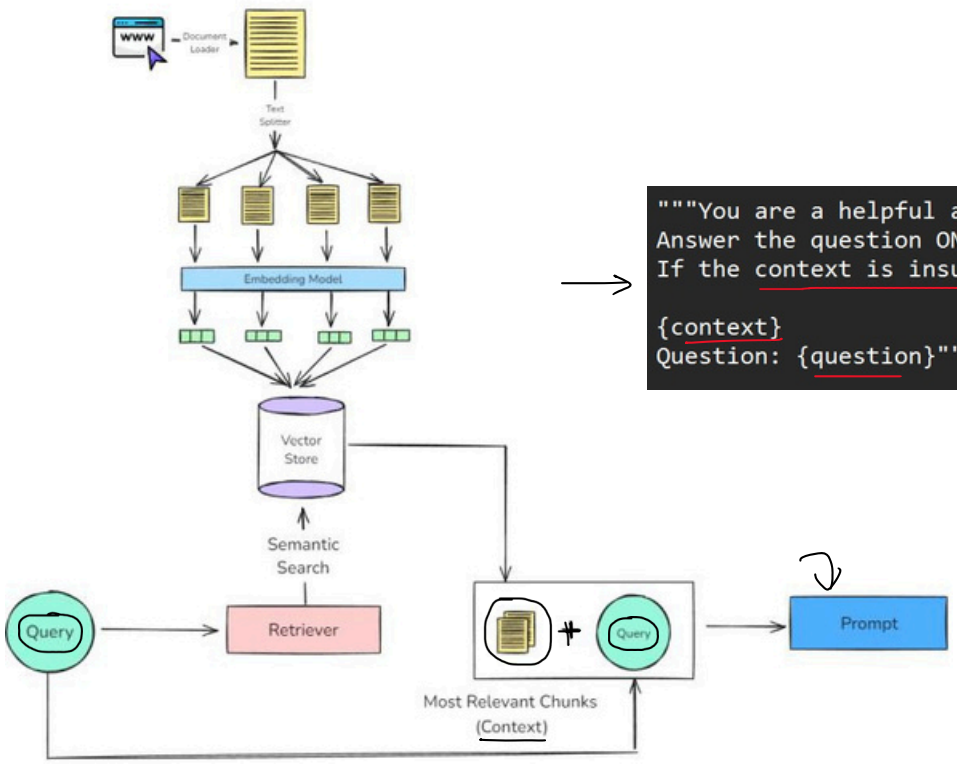
Retrieval-**Retrieval**is the *real-time*process of **finding the most relevant pieces of information**from a pre-built index (created during indexing) based on the user's question.

It's like asking:

> "From all the knowledge I have, which 3–5 chunks are most helpful to answer this query?"

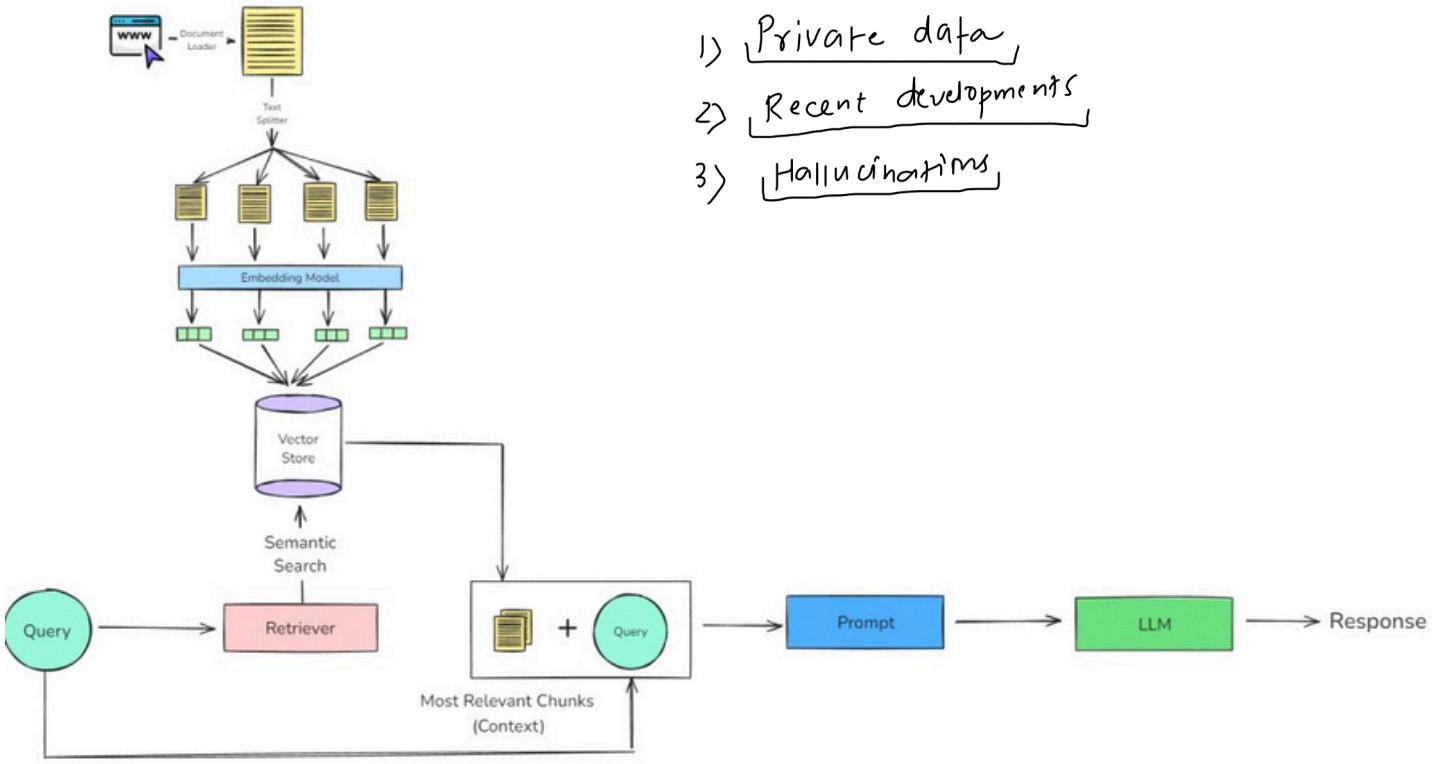


Augmentation-**Augmentation**refers to the step where the **retrieved documents**(chunks of relevant context) are **combined with the user's query**to form a new, enriched prompt for the LLM.




```
"""You are a helpful assistant.
Answer the question ONLY from the provided context.
If the context is insufficient, just say you don't know.

{context}
Question: {question}"""
```

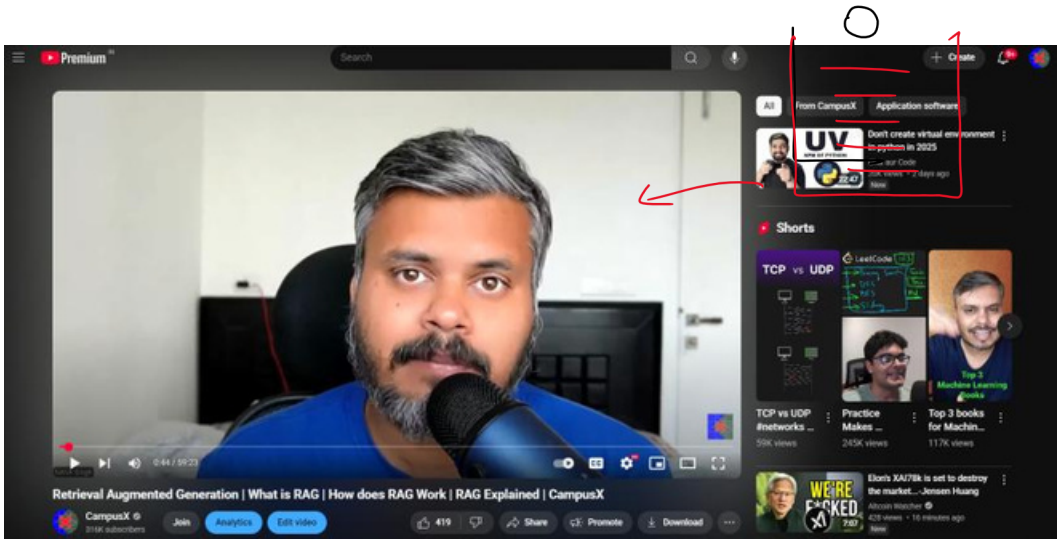
Generation-**Generation**is the final step where a **Large Language Model (LLM)**uses the **user's query**and the **retrieved & augmented context**to generate a response.

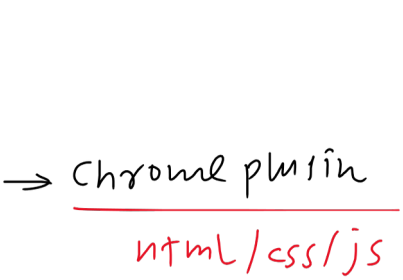


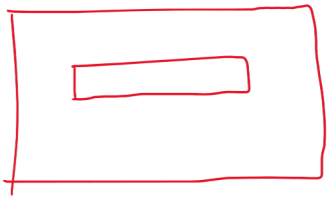
1) Private data
2) Recent developments
3) Hallucinations

Problem Statement

21 April 2025 08:55

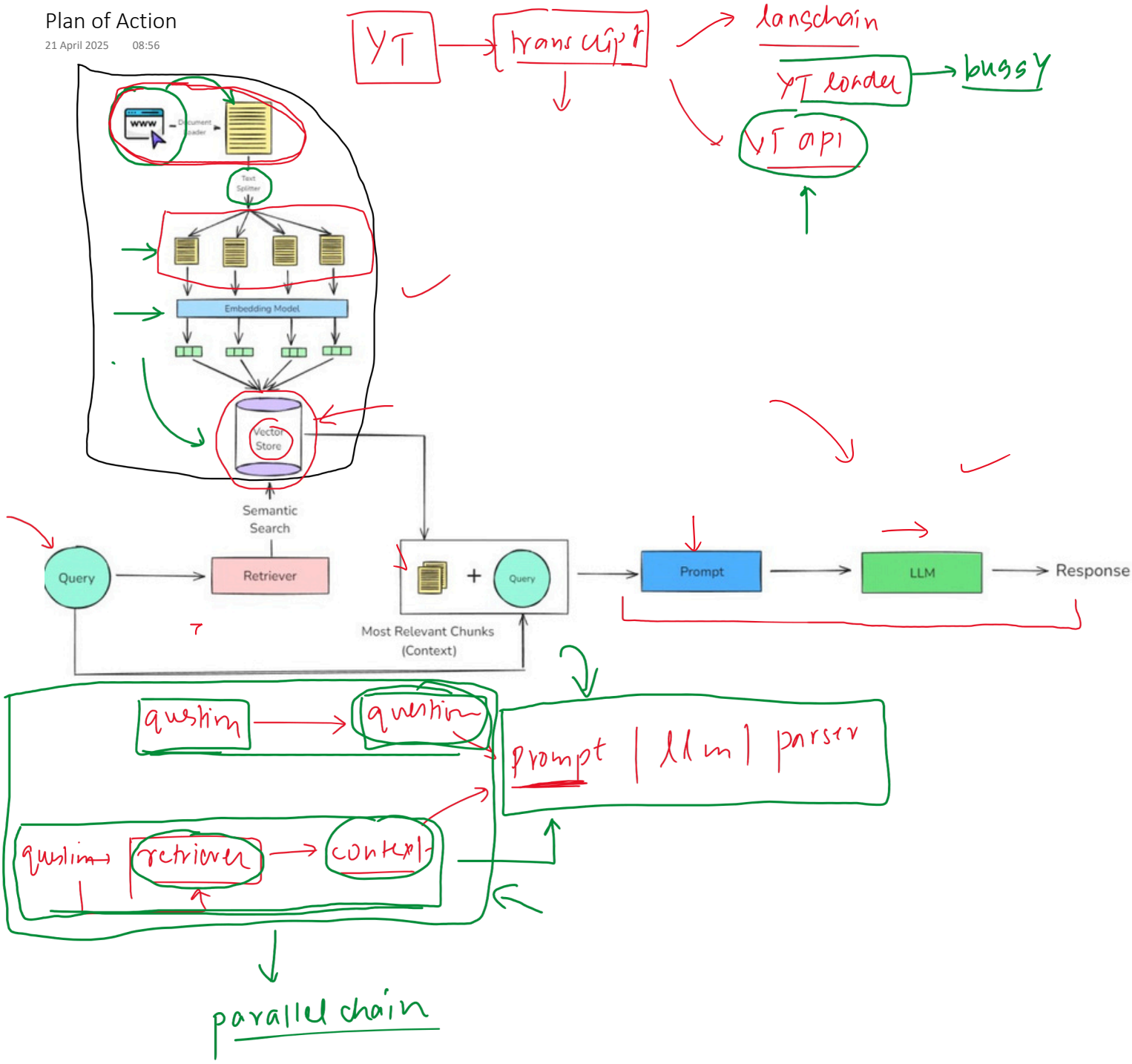






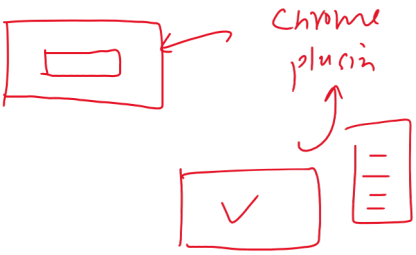
Plan of Action

21 April 2025 08:56



Improvements

22 April 2025 08:50

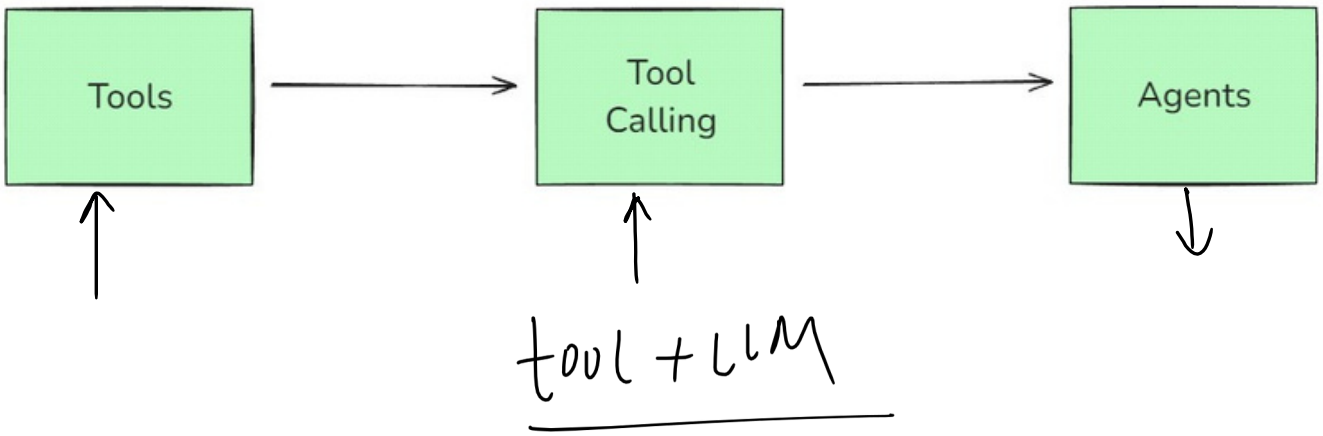


1. UI based enhancements
2. Evaluation
   a. Ragas
   b. LangSmith
3. Indexing
   a. Document Ingestion
   b. Text Splitting
   c. Vector Store → Pine Cone
4. Retrieval
   a. Pre-Retrieval
      i. Query rewriting using LLM
      ii. Multi-query generation
      iii. Domain aware routing
   b. DuringRetrieval
      i. MMR
      ii. HybridRetrieval
      iii. Reranking
   c. Post-Retrieval
      i. Contextual Compression
5. Augmentation
   a. Prompt Templating
   b. Answer grounding
   c. Context window optimization
6. Generation
   a. Answer with Citation
   b. Guard railing
7. System Design
   a. Multimodal
   b. Agentic
   c. Memory based

Metric	What It Measures
faithfulness	Is the answer grounded in the retrieved context?
answer_relevancy	Is the answer relevant to the user's question?
context_precision	How much of the retrieved context is actually useful?
context_recall	Did we retrieve all necessary information?

Overview

23 April 2025 15:51

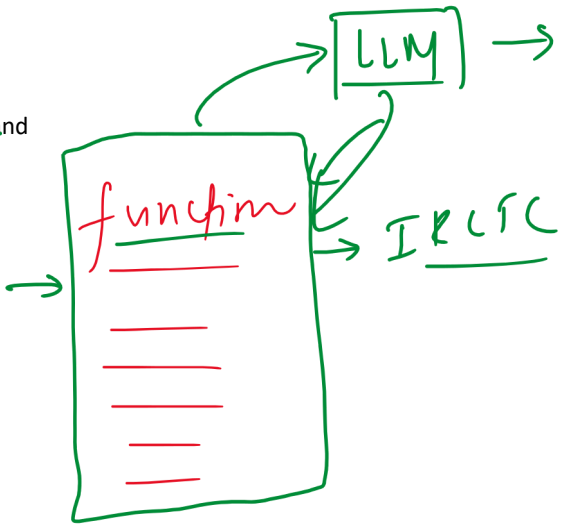


tool + LLM

What is a Tool?

23 April 2025 15:45

A toolis just a Python function (or API) that is packaged in a way the LLM can understand and callwhen needed.

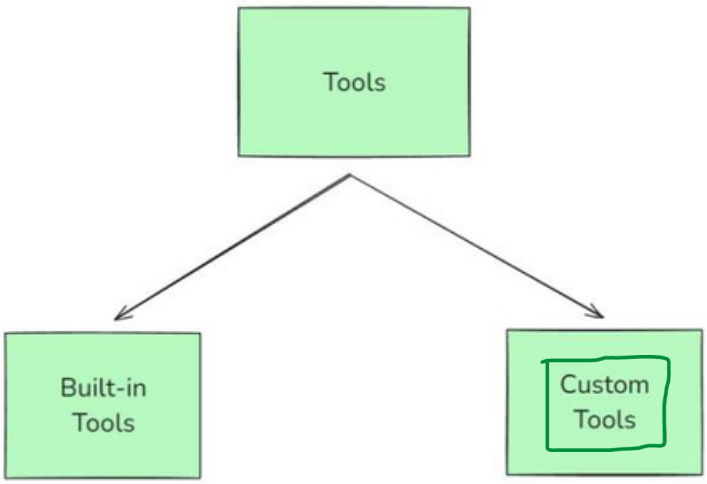


LLMs (like GPT) are great at:

- Reasoning (think)
- Language generation (speak)

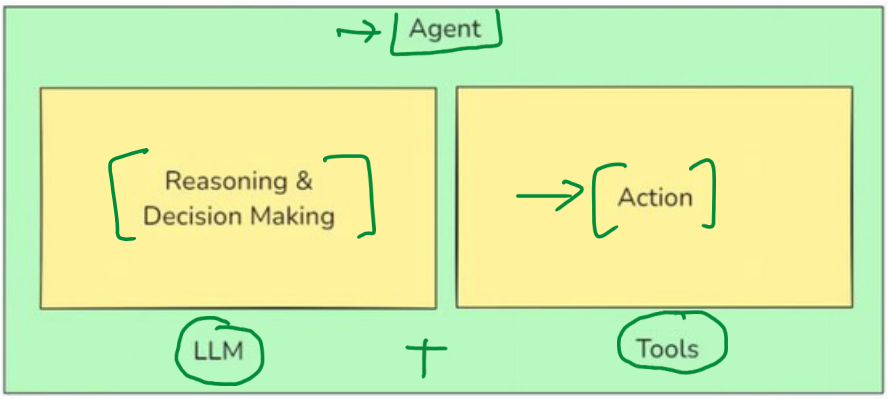
But they **can't do things like:**

- Access live data (weather, news)
- Do reliable math
- Call APIs
- Run code
- Interact with a database



How Tools fits into the Agent ecosystem

An AI agentis an LLM-powered system that can autonomously think, decide, and take actions using external tools or APIs to achieve a goal.



Built-in Tools

23 April 2025 15:46

A built-in toolis a tool that LangChain already provides for you —it's pre-built, production-ready, and requires minimal or no setup.

You don't have to write the function logic yourself —you just import and use it.

`DuckDuckGoSearchRun`	Web search via DuckDuckGo
`WikipediaQueryRun`	Wikipedia summary
`PythonREPLTool`	Run raw Python code
`ShellTool`	Run shell commands
`RequestsGetTool`	Make HTTP GET requests
`GmailSendMessageTool`	Send emails via Gmail
`SlackSendMessageTool`	Post message to Slack
`SQLDatabaseQueryTool`	Run SQL queries

```python
from langchain_community.tools import DuckDuckGoSearchRun

search_tool = DuckDuckGoSearchRun()

results = search_tool.invoke('top news in india today')

print(results)
```

Built-in Tool - Shell Tool

```python
from langchain_community.tools import ShellTool

shell_tool = ShellTool()

results = shell_tool.invoke('ls')

print(results)
```

```
Executing command:
 ls
sample_data

/usr/local/lib/python3.11/dist-packages/langchain_community/tools/shell/tool.py:33: UserW
  warnings.warn(
```

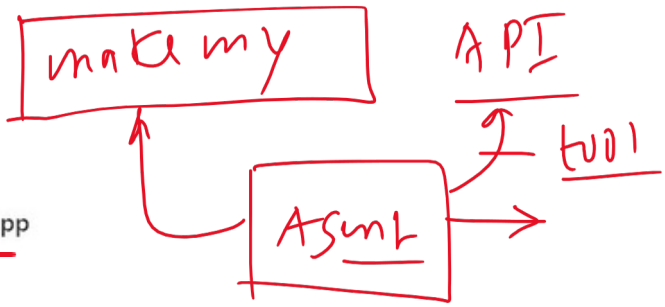
Custom Tools

23 April 2025 15:46

A custom toolis a tool that you define yourself.

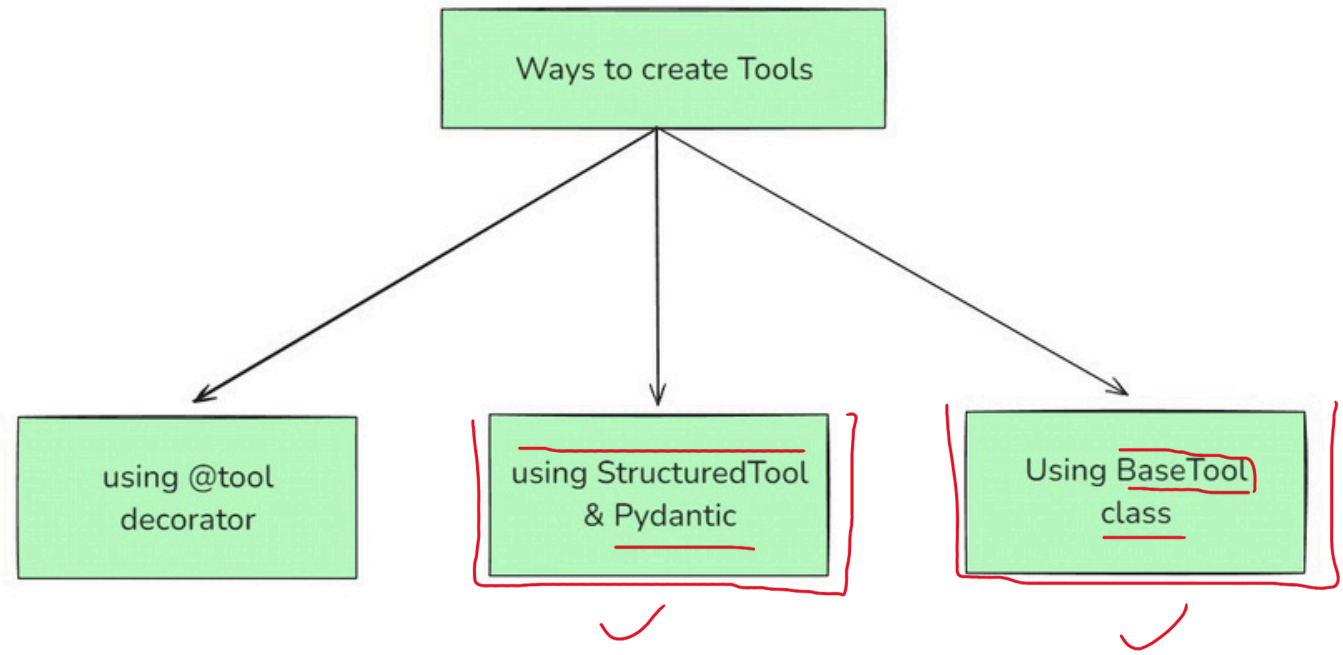
Use them when:

- You want to call **your own APIs**
- You want to **encapsulate business logic**
- You want the LLM to interact with **your database, product, or app**



Ways to create Custom Tools

23 April 2025 15:46



A **Structured Tool**in LangChain is a special type of toolwhere the input to the tool follows a structured schema, typically defined using a Pydantic model.

BaseToolis the abstract base classfor all tools in LangChain.
It defines the core structure and interfacethat any tool must follow, whether it's a simple one-liner or a fully customized function.

All other tool types like **@tool, StructuredTool**are built on top of BaseTool

Custom Tools

```python
from langchain_core.tools import tool


# Step 1 - create a function

def multiply(a, b):
    """Multiply two numbers"""
    return a*b


# Step 2 - add type hints

def multiply(a: int, b:int) -> int:
    """Multiply two numbers"""
    return a*b


# Step 3 - add tool decorator

@tool
def multiply(a: int, b:int) -> int:
    """Multiply two numbers"""
    return a*b


result = multiply.invoke({"a":3, "b":5})
```

```python
print(multiply.name)
print(multiply.description)
print(multiply.args)
```

```
ultiply
ultiply two numbers
{'a': {'title': 'A', 'type': 'integer'}, 'b': {'title': 'B', 'type': 'integer'}}
```

```python
print(multiply.args_schema.model_json_schema())
```

```
{'description': 'Multiply two numbers', 'properties': {'a': {'title': 'A', 'type': 'integer'}, 'b': {'title': 'B', 'type': 'integer'}},
```

Method 2 - Using StructuredTool

```python
from langchain.tools import StructuredTool
from pydantic import BaseModel, Field
```

```python
class MultiplyInput(BaseModel):
    a: int = Field(required=True, description="The first number to add")
    b: int = Field(required=True, description="The second number to add")
```

```python
def multiply_func(a: int, b: int) -> int:
    return a * b
```

```python
multiply_tool = StructuredTool.from_function(
    func=multiply_func,
    name="multiply",
    description="Multiply two numbers",
    args_schema=MultiplyInput
)
```

```python
result = multiply_tool.invoke({'a':3, 'b':3})

print(result)
print(multiply_tool.name)
print(multiply_tool.description)
print(multiply_tool.args)
```

```
9
multiply
Multiply two numbers
{'a': {'description': 'The first number to add', 'required': True, 'title': 'A', 'type': 'integer'}, 'b': {'description':
```

Method 3 - Using BaseTool Class

```python
from langchain.tools import BaseTool
from typing import Type
```

```python
# arg schema using pydantic

class MultiplyInput(BaseModel):
    a: int = Field(required=True, description="The first number to add")
    b: int = Field(required=True, description="The second number to add")
```

```python
class MultiplyTool(BaseTool):
    name: str = "multiply"
    description: str = "Multiply two numbers"

    args_schema: Type[BaseModel] = MultiplyInput

    def _run(self, a: int, b: int) -> int:
        return a * b
```

```python
multiply_tool = MultiplyTool()
```

```python
result = multiply_tool.invoke({'a':3, 'b':3})

print(result)
print(multiply_tool.name)
print(multiply_tool.description)

print(multiply_tool.args)
```

```
9
multiply
Multiply two numbers
{'a': {'description': 'The first number to add', 'required': True, 'title': 'A', 'type': 'integer'}, 'b': {'description': 'The second number to
```

Toolkits

23 April 2025 15:47

A toolkitis just a collection (bundle) of related toolsthat serve a common purpose — packaged together for convenience and**reusability**.

In LangChain:

- A toolkitmight be: GoogleDriveToolKit
- And it can contain the following tools

- `GoogleDriveCreateFileTool` : Upload a file
- `GoogleDriveSearchTool` : Search for a file by name/content
- `GoogleDriveReadFileTool` : Read contents of a file

Toolkit

```python
from langchain_core.tools import tool

# Custom tools
@tool
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b
```

```python
class MathToolkit:
    def get_tools(self):
        return [add, multiply]
```

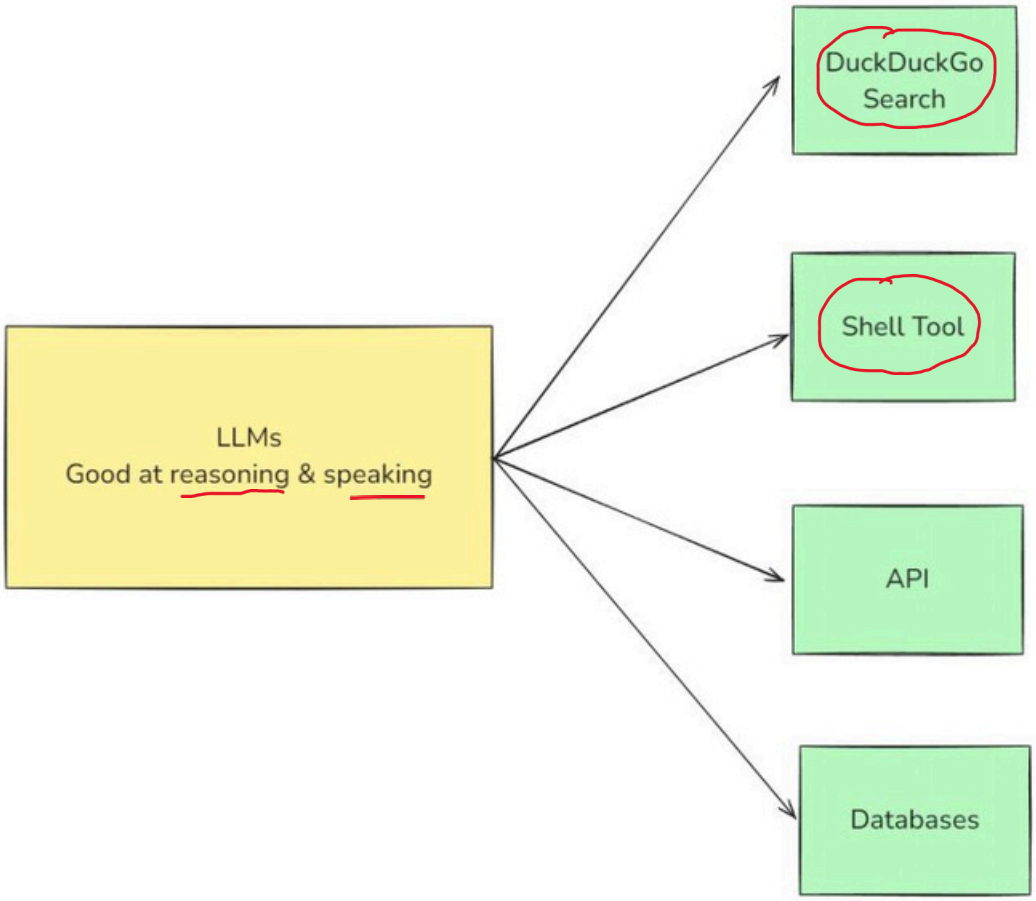
```python
toolkit = MathToolkit()
tools = toolkit.get_tools()

for tool in tools:
    print(tool.name, "=>", tool.description)
```

```
add => Add two numbers
multiply => Multiply two numbers
```

Quick Revision

25 April 2025 16:18

TOOLS

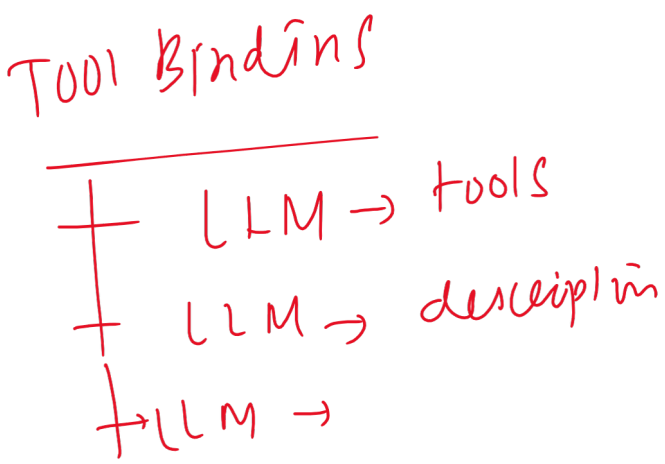


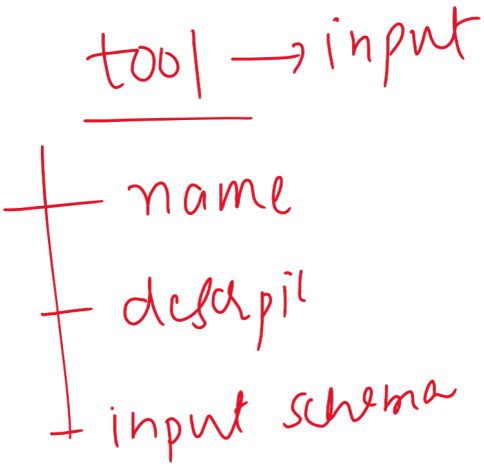
Tool Binding

25 April 2025 16:45

Tool Binding is the step where you **register tools with a Language Model (LLM)** so that:

1. The LLM knows **what tools are available**
2. It knows **what each tool does** (via description)
3. It knows **what input format to use** (via schema)






```
llm_with_tools = llm.bind_tools([multiply])
```

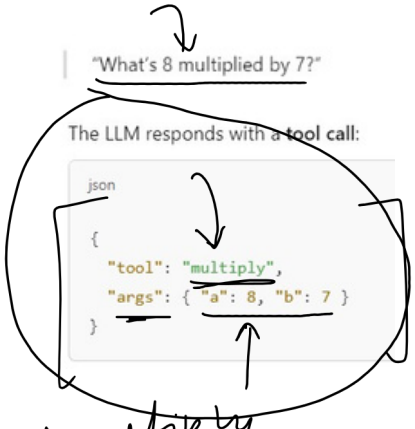
```
llm_with_tools.invoke('Hi how are you')
```

Tool Calling

25 April 2025 16:54

Tool Calling is the process where the **LLM (language model)** decides, during a conversation or task, that it needs to **use a specific tool (function)** — and generates a structured output with:

- the **name of the tool**
- and the **arguments** to call it with



⚠️ The LLM **does not actually run the tool** — it just **suggests** the tool and the input arguments. The **actual execution is handled by LangChain or you**.

Tool creation
Tool bindings

name : multiply
schema

Tool Execution

25 April 2025 17:12

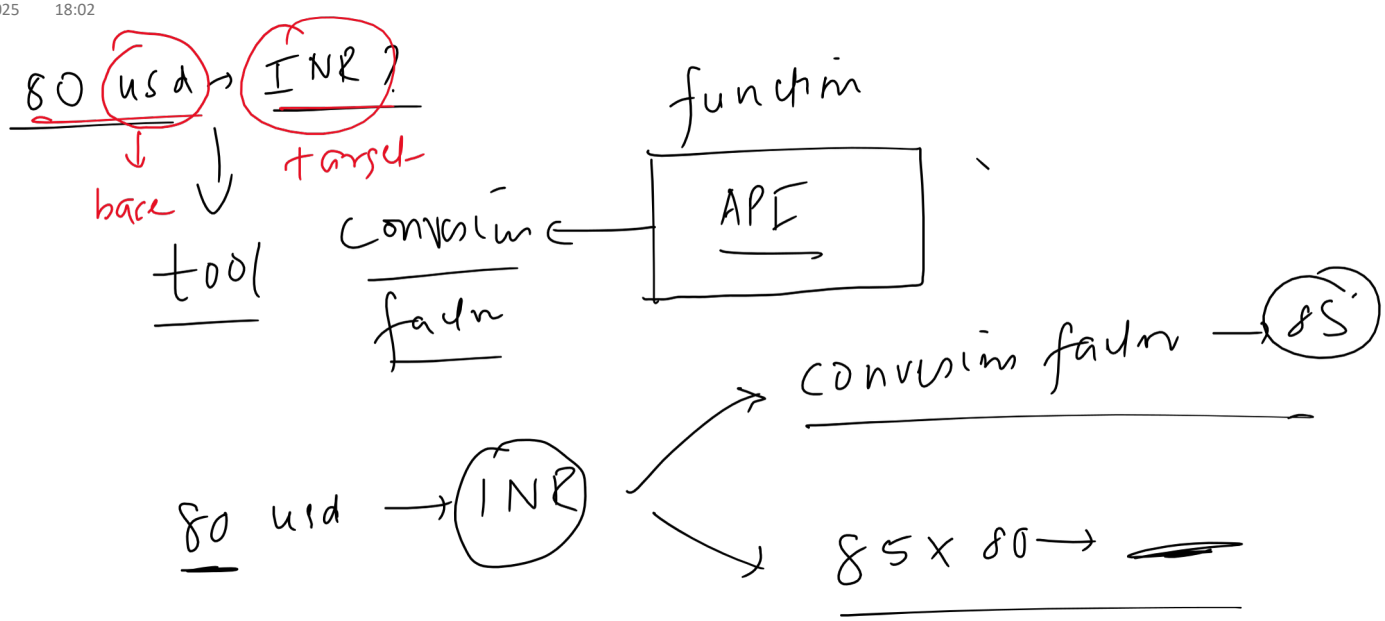
Tool Execution is the step where the **actual Python function (tool)** is run using the input arguments that the **LLM suggested during tool calling**.

In simpler words:

> 🧠 The LLM says:
> *"Hey, call the `multiply` tool with a=8 and b=7."*
>
> ⚙️ **Tool Execution** is when *you or LangChain actually run:*
> `multiply(a=8, b=7)`
> → and get the result: `56`

Currency Conversion Tool

25 April 2025 18:02



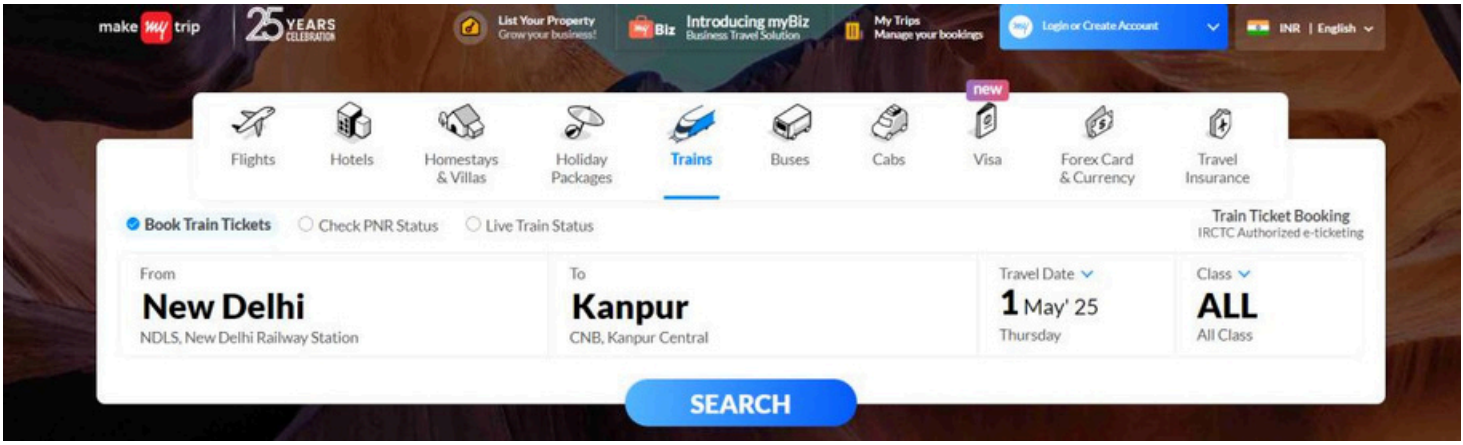
> ❌ "LLM, do **not** try to fill this argument." ✅ "I (the developer/runtime) will inject this value after running earlier tools."

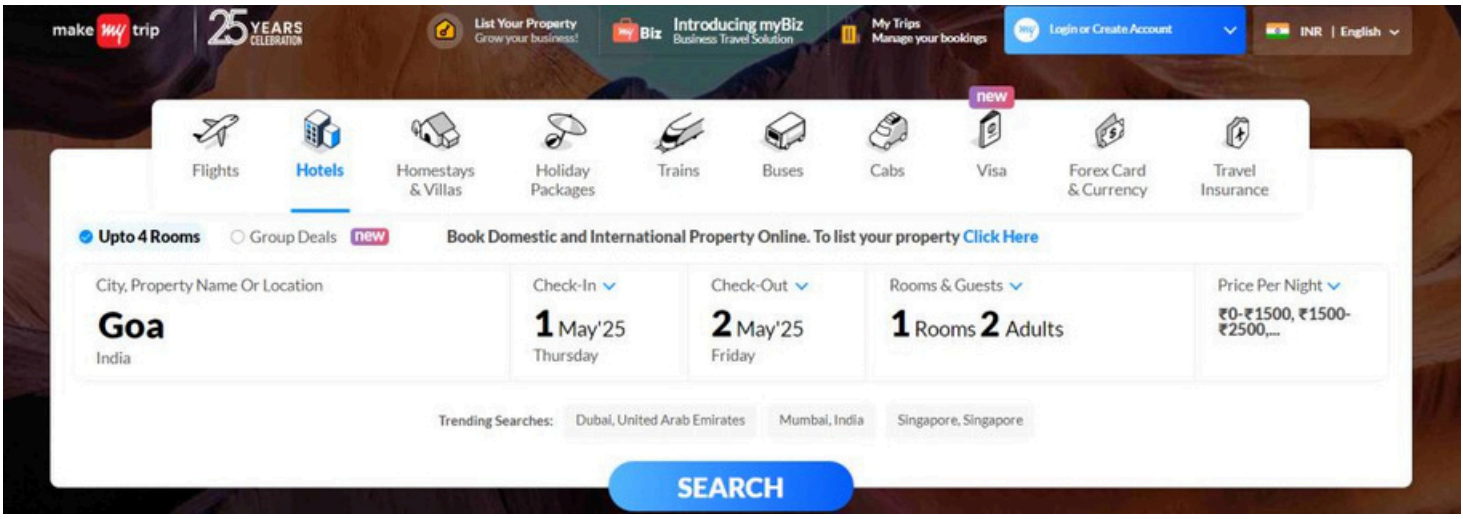
AI Agent → tools / tool calling (X)

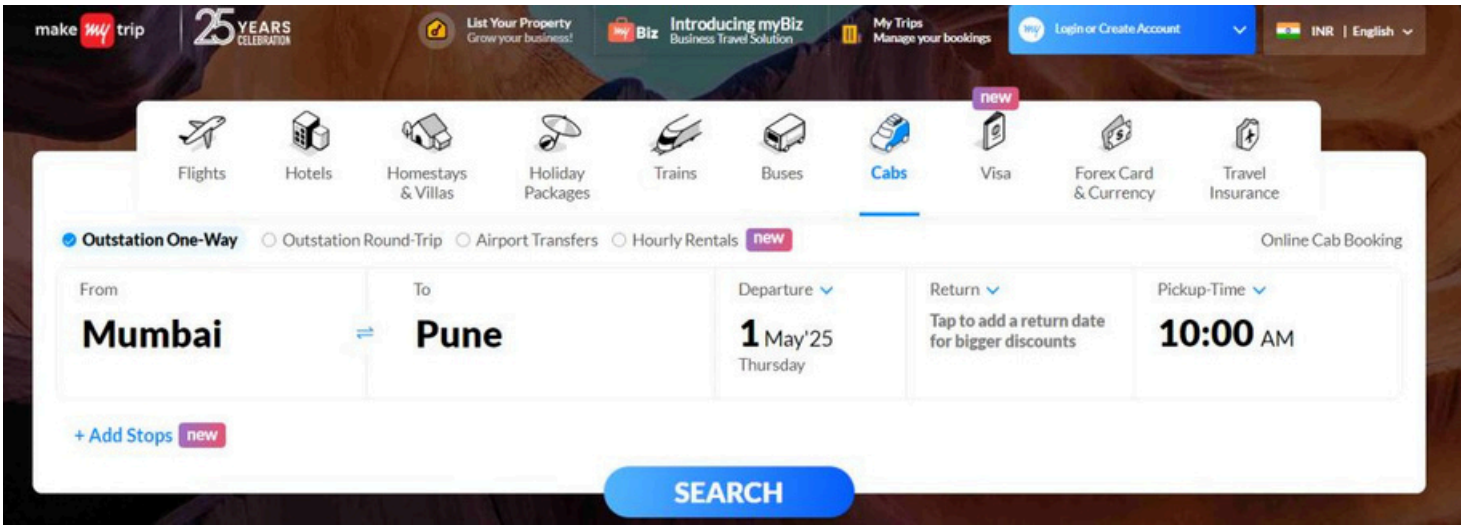
1. **User says:** "Convert 10 USD to INR."
2. **LLM thinks:** "I don't know the rate. First, let me call `get_conversion_factor`."
3. **Tool result comes:** 85.3415
4. **LLM looks at result, THINKS again:** "Now I know the rate, next I should call `convert` with 10 and 85.3415."
5. **Tool result comes:** 853.415 INR
6. **LLM summarizes:** "10 USD is 853.415 INR at current rate."

What are AI Agents

29 April 2025 22:53







✅ **User enters query:**

> "Can you create a budget travel itinerary from Delhi to Goa from 1st to 7th May?"

🧠 1. Understanding your intent

The agent internally interprets:

- ✅ Origin: Delhi
- ✅ Destination: Goa
- ✅ Dates: May 1–7
- ✅ Preference: Budget travel
- ✅ Goal: Plan complete itinerary + optimize cost

Agent thinks: *"I need to plan affordable travel, stay, local movement, and activities over 7 days."*

🔍 2. Research transport options

Agent (internally):

- Calls **IRCTC/train API** and **Flight APIs**
- Compares price, duration, and availability

Agent (to user):

> "The cheapest and most available option is the Goa Express from Delhi on April 30th night (sleeper class ₹800 or 3AC ₹1500).
> It arrives in Goa on May 1st evening. Would you like to go ahead?"

✅ User selects "3AC".

🏨 3. Search for affordable stay

Agent (internally):

- Uses hotel API or scraping tools
- Filters by:
  - Price < ₹700/night
  - Close to Baga or Anjuna beach
  - User reviews

Agent (to user):

> "I found a dorm room at The Hosteller, Baga for ₹650/night with good ratings. Shall I book this for 6 nights (May 1–7)?"

✅ User agrees.

🛵 4. **Plan local travel**

Agent (to user):

> "Scooter rentals are the most cost-effective in Goa.
> Should I pre-book a scooty for ₹300/day? (~₹1800 for 6 days)"

✅ User says yes.

🎯 5. **Generate activity plan**

Agent (internally):

- Calls sightseeing API / uses static local dataset
- Maps popular attractions across North & South Goa
- Builds a balanced plan:
  - Culture, relaxation, nightlife, shopping

Agent (to user):

> "Here's a draft plan:

- **May 2**: Baga, Calangute, Aguada Fort
- **May 3**: Old Goa churches, Fontainhas, local lunch
- **May 4**: South Goa day trip: Palolem, Cabo de Rama
- **May 5**: Anjuna flea market + sunset at Chapora
- **May 6**: Free day + beach cafe dinner"

✅ User loves it.

🛬 6. **Plan return**

Agent (to user):

> "Return options on May 7:

1. Train (₹800–₹1500, overnight)
2. Early morning flight (₹2800, 6:00 AM via SpiceJet)

Choose your preference."

✅ User picks train.

7. **Budget summary**

Agent (to user):

Item	Cost
Train (round trip)	₹3,000
Stay (6 nights)	₹3,900
Scooter + fuel	₹2,200
Food (local + budget)	₹2,500
Sightseeing & entry	₹1,000
Shopping/misc	₹1,500
Total	**~₹14,100**

> "Total trip cost: ₹14,100. Would you like to finalize the bookings?"

✅ User confirms.

8. **Final Output**

Agent (to user):

> ✅ Bookings confirmed.
> 🧾 Receipt sent to your email.
> 📆 Calendar invite added for train departures.
> 📱 You'll get daily reminders and local recommendations during your stay.
> 💬 Need help anytime? Just ask!

> **An AI agent is an intelligent system that receives a high-level goal from a user, and autonomously plans, decides, and executes a sequence of actions by using external tools, APIs, or knowledge sources — all while maintaining context, reasoning over multiple steps, adapting to new information, and optimizing for the intended outcome.**

Goal-driven	You tell the agent *what you want*, not *how to do it*
Autonomous planning	Agent breaks down the problem and sequences tasks on its own
Tool-using	Agent calls APIs, calculators, search tools, etc.
Context-aware	Maintains memory across steps to inform future actions
Reasoning-capable	Makes decisions dynamically (e.g., "what to do next")
Adaptive	Rethinks plan when things change (e.g., API fails, no data)

Building an agent in LangChain

29 April 2025 22:53

Explanation

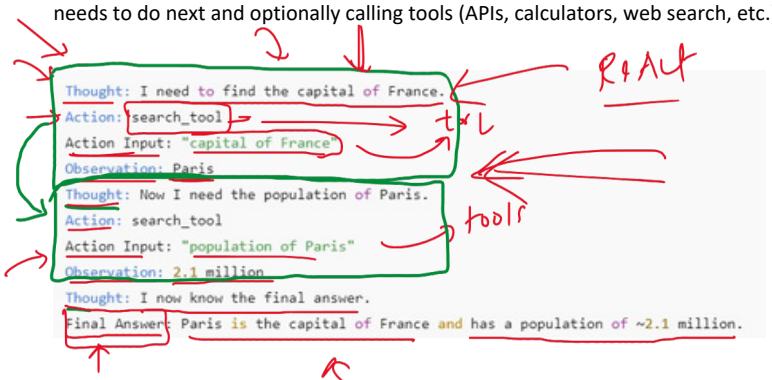
29 April 2025 22:53

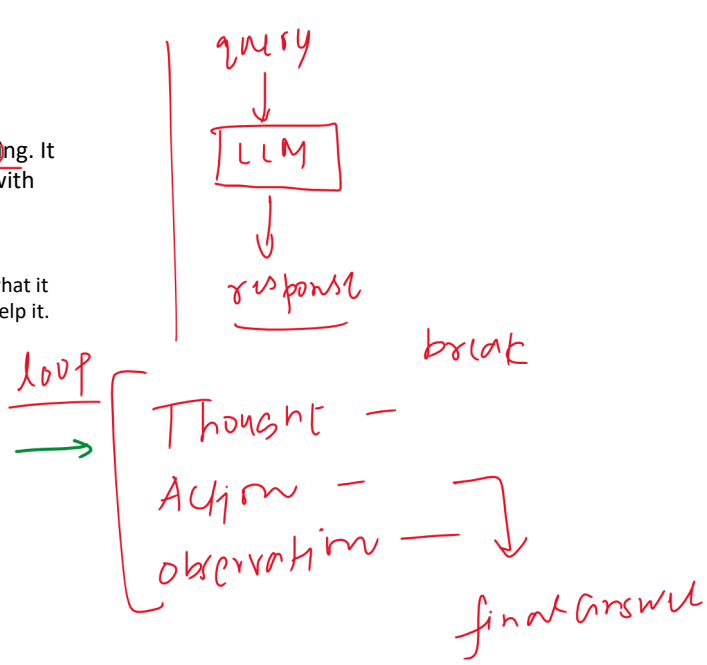
1. ReAct

01 May 2025 14:17

ReAct is a design pattern used in AI agents that stands for Reasoning + Acting. It allows a language model (LLM) to interleave internal reasoning (Thought)with external actions (like tool use)in a structured, multi-step process.

Instead of generating an answer in one go, the model thinks step by step, deciding what it needs to do next and optionally calling tools (APIs, calculators, web search, etc.) to help it.





ReAct is useful for:

- Multi-step problems
- Tool-augmented tasks (web search, database lookup, etc.)
- Making the agent's reasoning **transparent and auditable**

It was first introduced in the paper:

"*ReAct: Synergizing Reasoning and Acting in Language Models*" (Yao et al., 2022)

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao*,1, Jeffrey Zhao[2], Dian Yu[2], Nan Du[2], Izhak Shafran[2], Karthik Narasimhan[1], Yuan Cao[2]

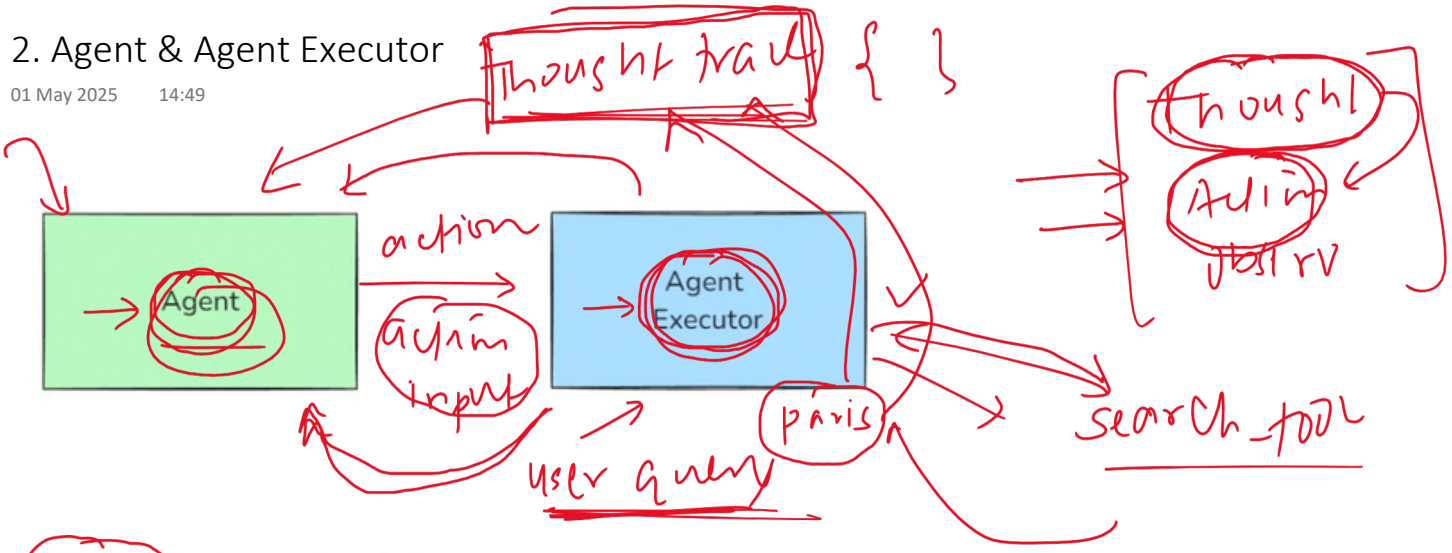
[1]Department of Computer Science, Princeton University
[2]Google Research, Brain team
[1]{shunyuy,karthikn}@princeton.edu
[2]{jeffreyzhao,dianyu,dunan,izhak,yuancao}@google.com

ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two: reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named ReAct, to a diverse set of language and decision making tasks and demonstrate its effectiveness over state-of-the-art baselines in addition to improved human interpretability and trustworthiness. Concretely, on question answering (HotpotQA) and fact verification (Fever), ReAct overcomes prevalent issues of hallucination and error propagation in chain-of-thought reasoning by interacting with a simple Wikipedia API, and generating human-like task-solving trajectories that are more interpretable than baselines without reasoning traces. Furthermore, on two interactive decision making benchmarks (ALFWorld and WebShop), ReAct outperforms imitation and reinforcement learning methods by an absolute success rate of 34% and 10% respectively, while being prompted with only one or two in-context examples.

2. Agent & Agent Executor

01 May 2025 14:49



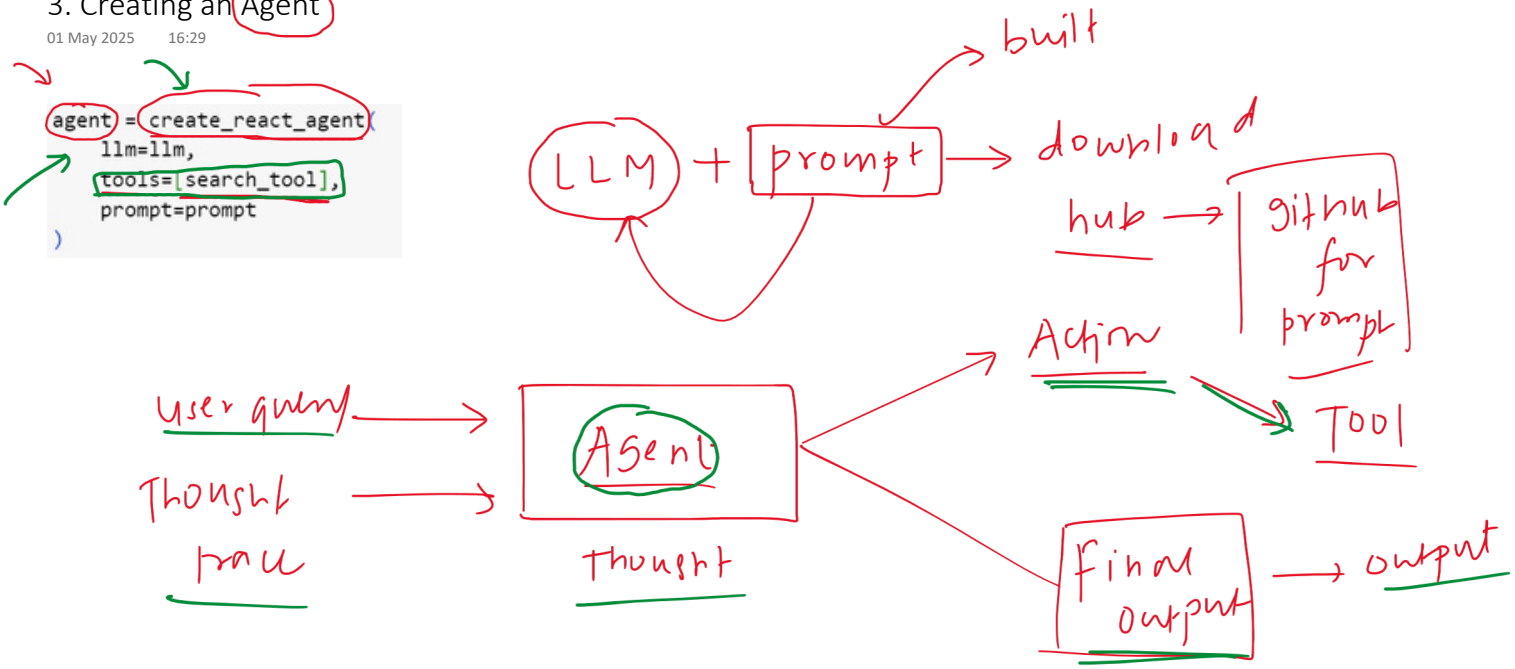
`AgentExecutor` orchestrates the **entire loop**:

1. Sends inputs and previous messages to the agent
2. Gets the next `action` from agent
3. Executes that tool with provided input
4. Adds the tool's `observation` back into the history
5. Loops again with updated history until the agent says `Final Answer`.

3. Creating an Agent

01 May 2025 16:29

```
agent = create_react_agent(
    llm=llm,
    tools=[search_tool],
    prompt=prompt
)
```




```
llm = ChatOpenAI()
```

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub
```

```
# Step 2: Pull the ReAct prompt from LangChain Hub
prompt = hub.pull("hwchase17/react")  # pulls the standard ReAct agent prompt
```

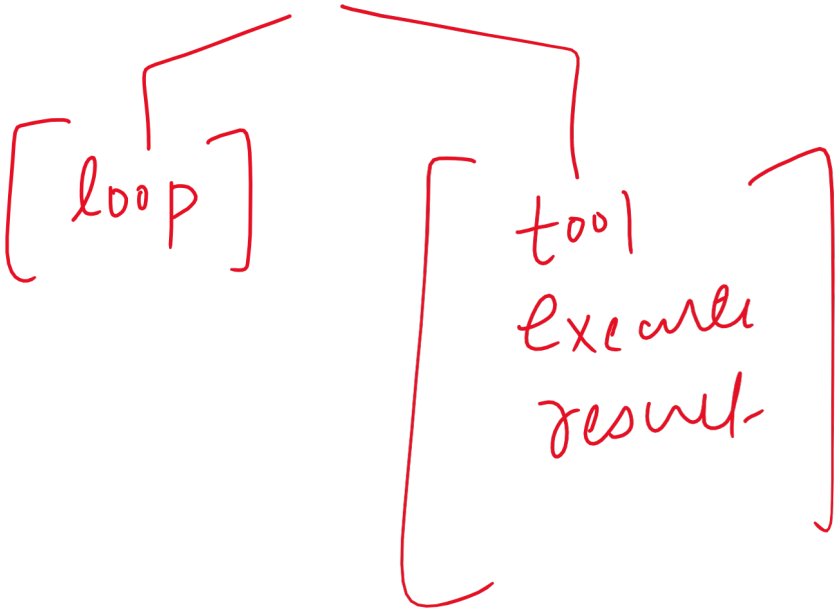
```
usr/local/lib/python3.11/dist-packages/langsmith/client.py:271: LangSmithMissingAPIKeyWa
 warnings.warn(
```

```
# Step 3: Create the ReAct agent manually with the pulled prompt
agent = create_react_agent(
    llm=llm,
    tools=[search_tool, get_weather_data],
    prompt=prompt
)
```

4. Creating an Agent Executor

01 May 2025 16:30

```
agent_executor = AgentExecutor(
    agent=agent,
    tools=[search_tool],
    verbose=True
)
```




```
# Step 4: Wrap it with AgentExecutor
agent_executor = AgentExecutor(
    agent=agent,
    tools=[search_tool, get_weather_data],
    verbose=True
)
```

```
# Step 5: Invoke
response = agent_executor.invoke({"input": "Find the capital of Madhya Pradesh, then find it's current weather condition"})
print(response)
```

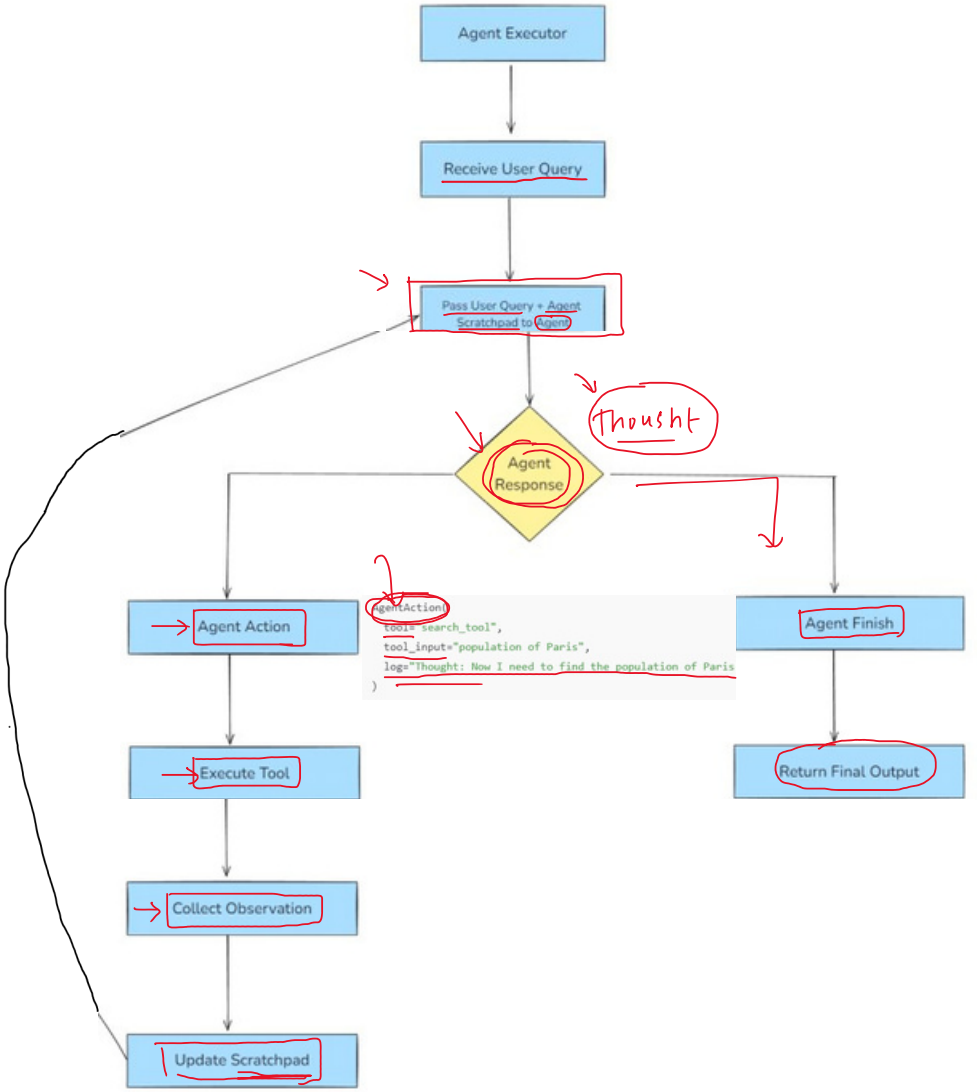
```
> Entering new AgentExecutor chain...
I should first find out the capital of Madhya Pradesh and then check the current weather condition for that city.
Action: duckduckgo_search
Action Input: "capital of Madhya Pradesh"Madhya Pradesh, state of India that is situated in the heart of the country. It has no coastline and no inte
Action: get_weather_data
Action Input: Bhopal{'request': {'type': 'City', 'query': 'Bhopal, India', 'Language': 'en', 'unit': 'm'}, 'Location': {'name': 'Bhopal', 'country':
Final Answer: The capital of Madhya Pradesh is Bhopal, and the current weather condition in Bhopal is partly cloudy with a temperature of 40°C.

> Finished chain.
{'input': "Find the capital of Madhya Pradesh, then find it's current weather condition", 'output': 'The capital of Madhya Pradesh is Bhopal, and the
```

```
response['output']
```

```
'The capital of Madhya Pradesh is Bhopal, and the current weather condition in Bhopal is partly cloudy with a temperature of 40°C.'
```

5. Flow Chart

01 May 2025 16:30



6. Example

01 May 2025 17:23

🧠 Input Query:

> "What is the capital of France and what is its population?"

```
Answer the following questions as best you can. You have access to the following tools:

search_tool: Useful for answering general knowledge questions by querying a search API.

Use the following format:

Question: the input question you must answer
Thought: you should always think about what to do
Action: the action to take, should be one of [search_tool]
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Action Input/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question

Begin!

Question: What is the capital of France and what is its population?
Thought:
```

```
Thought: I need to find the capital of France first.
Action: search_tool
Action Input: "capital of France"
```

```
AgentAction(
  tool="search_tool",
  tool_input="capital of France",
  log="Thought: I need to find the capital of France first."
)
```

```
observation = search_tool("capital of France")
```

→ `"Paris is the capital of France."`

```
Thought: I need to find the capital of France first.
Action: search_tool
Action Input: "capital of France"
Observation: Paris is the capital of France.
```

📌 Current state so far:

- **User Input:**

```
What is the capital of France and what is its population?
```

- **Agent Scratchpad after Step 2:**

text

```
Thought: I need to find the capital of France first.
Action: search_tool
Action Input: "capital of France"
Observation: Paris is the capital of France.
```

```
Answer the following questions as best you can. You have access to the following tools

search_tool: Useful for answering general knowledge questions by querying a search API.

Use the following format:

Question: the input question you must answer
Thought: you should always think about what to do
Action: the action to take, should be one of [search_tool]
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Action Input/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question

Begin!

Question: What is the capital of France and what is its population?
Thought: I need to find the capital of France first.
Action: search_tool
Action Input: "capital of France"
Observation: Paris is the capital of France.
Thought:
```

```
Thought: Now I need to find the population of Paris.
Action: search_tool
Action Input: "population of Paris"
```

```
AgentAction(
  tool="search_tool",
  tool_input="population of Paris",
```

```
AgentAction(
  tool="search_tool",
  tool_input="population of Paris",
  log="Thought: Now I need to find the population of Paris.
)
```

```
observation = search_tool("population of Paris")
```

```
"Paris has a population of approximately 2.1 million."
```

```
Thought: I need to find the capital of France first.
Action: search_tool
Action Input: "capital of France"
Observation: Paris is the capital of France.
Thought: Now I need to find the population of Paris.
Action: search_tool
Action Input: "population of Paris"
Observation: Paris has a population of approximately 2.1 million.
```

```
Answer the following questions as best you can. You have access to the following tools:

search_tool: Useful for answering general knowledge questions by querying a search API.

Use the following format:

Question: the input question you must answer
Thought: you should always think about what to do
Action: the action to take, should be one of [search_tool]
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Action Input/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question

Begin!

Question: What is the capital of France and what is its population?
Thought: I need to find the capital of France first.
Action: search_tool
Action Input: "capital of France"
Observation: Paris is the capital of France.
Thought: Now I need to find the population of Paris.
Action: search_tool
Action Input: "population of Paris"
Observation: Paris has a population of approximately 2.1 million.
Thought:
```

```
Thought: I now know the final answer.
Final Answer: Paris is the capital of France and has a population of approximately 2.1 million.
```

```
Thought: I now know the final answer.
Final Answer: Paris is the capital of France and has a population of approximately 2.1 million.
```

```
AgentFinish(
  return_values={"output": "The capital of France is Paris..."},
  log="Thought: I now know the final answer.\nFinal Answer: ..."
)
```

```
{
  "output": "Paris is the capital of France and has a population of approximately 2.1 million."
}
```